



Level Up Coding

You're reading for free via [Fareed Khan's](#) Friend Link. [Upgrade](#) to access the best of Medium.

★ Member-only story

Building 17 Agentic AI Patterns and Their Role in Large-Scale AI Systems

Ensembling, Meta-Control, ToT, Reflexive, PEV and more



Fareed Khan

Follow

77 min read · Sep 26, 2025



2.3K

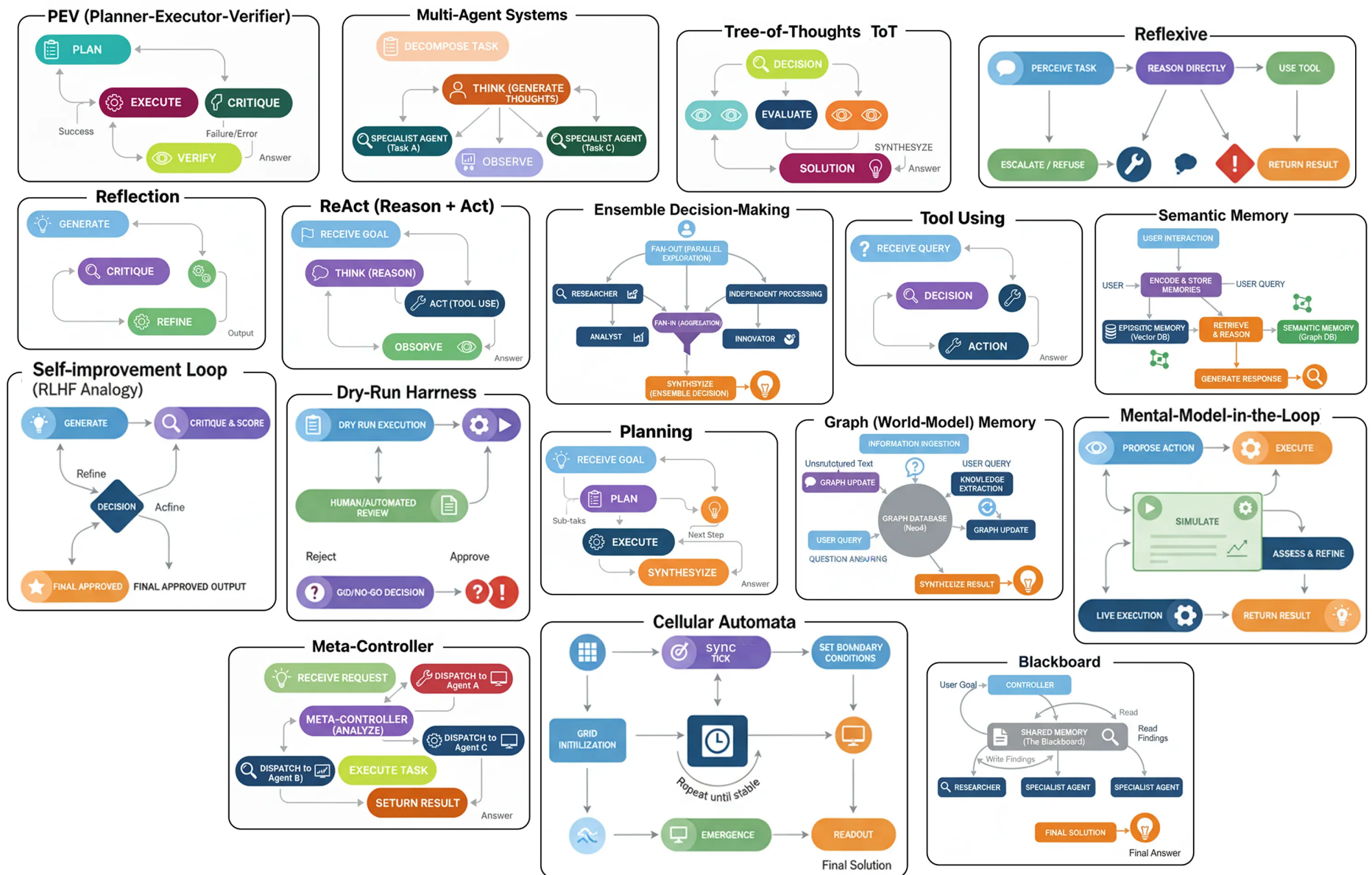


47



Read this story for free: [link](#)

When you build a large-scale AI system, you are really putting different **agentic design patterns** together. Each one has its own stage, build method, output, and evaluation. If we step back and group these patterns, they can be broken down into **17 high-level architectures** that capture the main shapes agentic systems can take ...



Agentic Architectures (Created by Fareed Khan)

Some of the patterns are:

- A **Multi-Agent System**, where several tools and agents work together to solve a problem.
- An **Ensemble Decision System**, where multiple agents each propose an answer and then vote on the best one.
- A **Tree-of-Thoughts**, where the agent explores many different reasoning paths before selecting the most promising direction.
- **Reflexive approach**, where the agent is able to recognize and acknowledge what it does not know.
- The **ReAct loop**, where the agent alternates between thinking, taking action, and then thinking again to refine its process.

And there are many more ...

In this blog, we are going to break down these different kinds of **agentic architectures** and show how each one plays a unique role in a complete AI system.

We will visually understand each architecture importance, code its workflow, and evaluate it to see whether it truly improves performance compared to a baseline.

All the code is available in my GitHub Repository:

**GitHub - FareedKhan-dev/all-agentic-architectures:
Implementation of 17+ agentic architectures...**

Implementation of 17+ agentic architectures designed for practical use across different stages of AI system...

[github.com](https://github.com/FareedKhan-dev/all-agentic-architectures)

Codebase is organized as follows:

```
all-agentic-architectures/  
├── 01_reflection.ipynb  
├── 02_tool_use.ipynb  
└── 03_ReAct.ipynb
```

```
...
├─ 06_PEV.ipynb
├─ 07_blackboard.ipynb
├─ 08_episodic_with_semantic.ipynb
├─ 09_tree_of_thoughts.ipynb
...
├─ 14_dry_run.ipynb
├─ 15_RLHF.ipynb
├─ 16_cellular_automata.ipynb
└─ 17_reflexive_metacognitive.ipynb
```

Table of contents

1. Setting up the Environment
2. Reflection
3. Tool Using
4. ReAct (Reason + Act)
5. Planning
6. PEV (Planner-Executor-Verifier)
7. Tree-of-Thoughts (ToT)
8. Multi-Agent Systems
9. Meta-Controller

10. Blackboard
11. Ensemble Decision-Making
12. Episodic + Semantic Memory
13. Graph (World-Model) Memory
14. Self-Improvement Loop (RLHF Analogy)
15. Dry-Run Harness
16. Simulator (Mental-Model-in-the-Loop)
17. Reflexive Metacognitive
18. Cellular Automata
19. Combining Architectures Together

Setting up the Environment

Before we start building each architecture, we need to set up the basics and be clear on what we are using and why the modules, the models, and how it all fits together.

We do know that LangChain, LangGraph, and LangSmith are pretty much the industry-standard modules out there for building any serious RAG or agentic system. They give us everything we need to build, orchestrate, and, most importantly, figure out what's going on inside our agents when things get complicated, so that's why we are sticking with these three.

The very first step is to get our core libraries imported. This way, we avoid repeating ourselves later and keep the setup nice and clean. Let's do that.

```
import os
from typing import List, Dict, Any, Optional, Annotated, TypedDict
from dotenv import load_dotenv # Load environment variables from .env file

# Pydantic for data modeling / validation
from pydantic import BaseModel, Field

# LangChain & LangGraph components
from langchain_nebius import ChatNebius # Nebius LLM wrapper
from langchain_tavily import TavilySearch # Tavily search tool integration
from langchain_core.prompts import ChatPromptTemplate # For structuring prompts
from langgraph.graph import StateGraph, END # Build a state machine graph
from langgraph.prebuilt import ToolNode, tools_condition # Prebuilt nodes & con

# For pretty printing output
from rich.console import Console # Console styling
from rich.markdown import Markdown # Render markdown in terminal
```

So, let's quickly break down why we are using these three.

- LangChain is our toolbox, giving us the core building blocks like prompts, tool definitions, and LLM wrappers.
- LangGraph is our orchestration engine, wiring everything together into complex workflows with loops and branches.

- LangSmith is our debugger, showing a visual trace of every step an agent takes so we can quickly spot and fix issues.

We will be working with open-source LLMs through providers like Nebius AI or Together AI. The nice part is they work just like the standard OpenAI module, so we don't have to change much to get started. And if we want to run things locally, we can just swap in something like Ollama.

To make sure our agents are not stuck with static data, we are giving them access to the Tavily API for live web searches (1000 credits/month enough for testing I think). That way, they can ...

actively go out and find information on their own,
which keeps the focus on real reasoning and tool use.

Next, we need to set up our environment variables. This is where we'll keep our sensitive info, like API keys. To do this, create a file called `.env` in the same directory and drop your keys inside it, for example:

```
# API key for Nebius LLM (used with ChatNebius)
```

```
NEBIUS_API_KEY="your_nebius_api_key_here"

# API key for LangSmith (LangChain's observability/telemetry platform)
LANGCHAIN_API_KEY="your_langsmith_api_key_here"

# API key for Tavily search tool (used with TavilySearch integration)
TAVILY_API_KEY="your_tavily_api_key_here"
```

Once the `.env` file is set up, we can easily pull those keys into our code using the `dotenv` module we imported earlier.

```
load_dotenv() # Load environment variables from .env file

# Enable LangSmith tracing for monitoring / debugging
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "Implementing 17 Agentic Architectures" # Project name

# Verify that all required API keys are available
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key): # If key not found in env vars
        print(f"{key} not found. Please create a .env file and set it.")
```

Now that our basic setup is ready, we can start building each architecture one by one to see how they perform and where they make the most sense in a large-scale AI system.

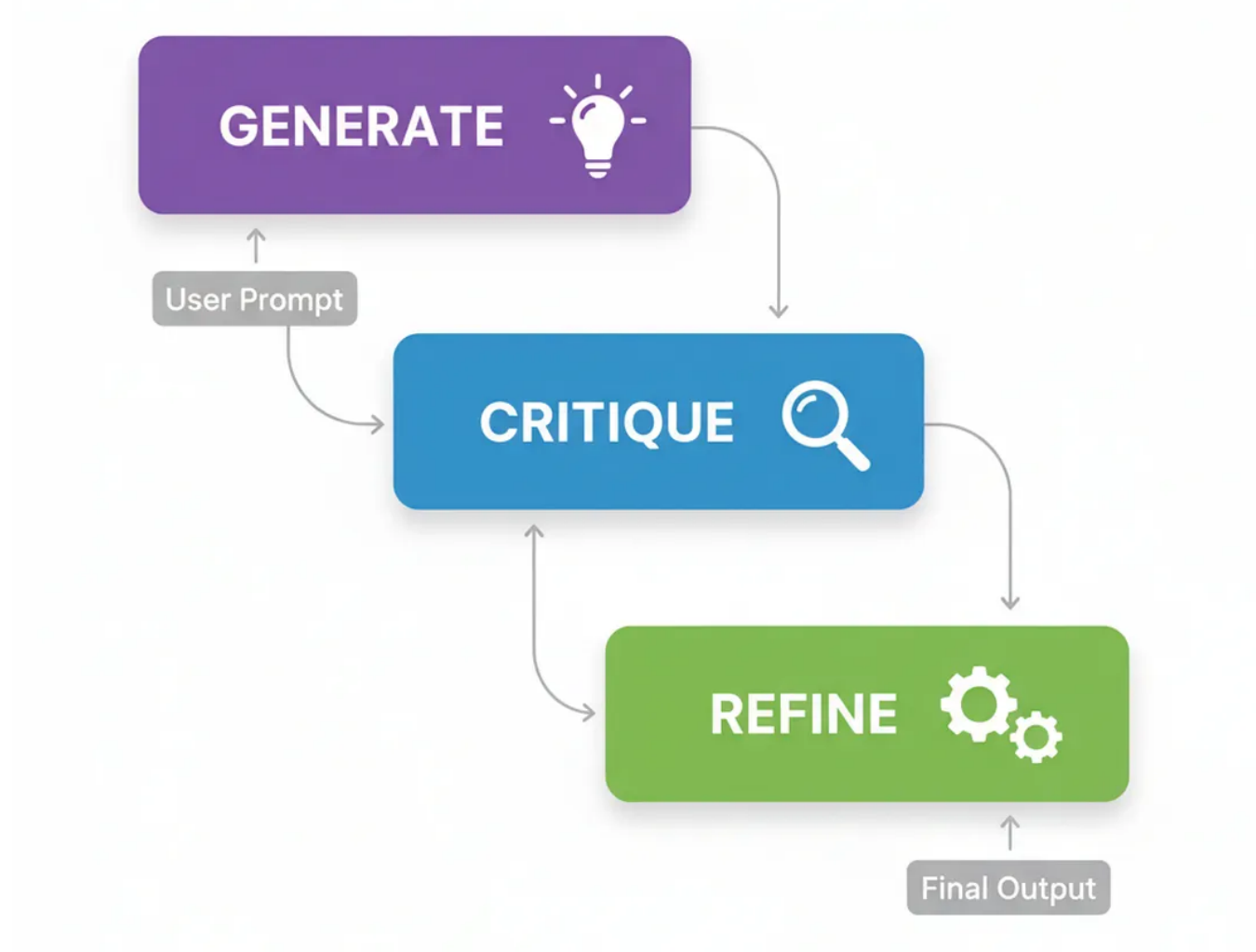
Reflection

So, the very first architecture we are going to look at is **Reflection**. This one is probably the most common and foundational pattern you see in agentic workflows.

It's all about giving an agent the ability to take a step back, look at its own work, and make it better.

In a large-scale AI system, this pattern fits perfectly in any stage **where the quality of a generated output is critical**. Think about tasks like generating complex code, writing detailed technical reports, anywhere a simple, first-draft answer just isn't good enough and could lead to real-world problems.

Let's understand how the process flows.



Reflection agent workflow (Created by [Fareed Khan](#))

1. **Generate:** The agent takes the user prompt and produces an initial draft

or solution. This is its first, unfiltered attempt.

2. **Critique:** The agent then switches roles and becomes its own critic. It analyzes the draft for flaws, asking questions like “Is this correct?”, “Is this efficient?”, or “What am I missing?”.
3. **Refine:** Finally with its own critique, the agent generates a final, improved version of the output, directly addressing the flaws it found.

Before we create the agent’s logic, we need to define the data structures it will work with. Using Pydantic models is a great way to force the LLM to give us clean, structured JSON output, which is critical for a multi-step process where the output of one step becomes the input for the next.

```
class DraftCode(BaseModel):  
    """Schema for the initial code draft generated by the agent."""  
    code: str = Field(description="The Python code generated to solve the user's  
    explanation: str = Field(description="A brief explanation of how the code wo  
  
class Critique(BaseModel):  
    """Schema for the self-critique of the generated code."""  
    has_errors: bool = Field(description="Does the code have any potential bugs  
    is_efficient: bool = Field(description="Is the code written in an efficient  
    suggested_improvements: List[str] = Field(description="Specific, actionable  
    critique_summary: str = Field(description="A summary of the critique.") # o  
  
class RefinedCode(BaseModel):
```

```
"""Schema for the final, refined code after incorporating the critique."""  
refined_code: str = Field(description="The final, improved Python code.") #  
refinement_summary: str = Field(description="A summary of the changes made b
```

With these schemas, we have a clear ‘contract’ for our LLM. The `critique` model is especially useful because it forces a structured review, asking for specific checks on errors and efficiency rather than just a vague "review the code" command.

First up is our `generator_node`. Its only job is to take the user request and produce that first draft.

```
def generator_node(state):  
    """Generates the initial draft of the code."""  
    console.print("--- 1. Generating Initial Draft ---")  
    # Initialize the LLM to return a structured DraftCode object  
    generator_llm = llm.with_structured_output(DraftCode)  
  
    prompt = f"""You are an expert Python programmer. Write a Python function to  
    Provide a simple, clear implementation and an explanation.  
  
    Request: {state['user_request']}  
    """  
  
    draft = generator_llm.invoke(prompt)  
    return {"draft": draft.model_dump()}
```

This node will give us our initial `draft` and its `explanation`, which we'll then pass on to the critic for review.

Now for the core of the reflection process the `critic_node`. This is where the agent steps into the role of a senior developer and gives its own work a tough code review.

```
def critic_node(state):  
    """Critiques the generated code for errors and inefficiencies."""  
    console.print("--- 2. Critiquing Draft ---")  
    # Initialize the LLM to return a structured Critique object  
    critic_llm = llm.with_structured_output(Critique)  
  
    code_to_critique = state['draft']['code']  
  
    prompt = f"""You are an expert code reviewer and senior Python developer. You  
  
Analyze the code for:  
1.  **Bugs and Errors:** Are there any potential runtime errors, logical flaws, or syntax errors?  
2.  **Efficiency and Best Practices:** Is this the most efficient way to solve the problem?  
  
Provide a structured critique with specific, actionable suggestions.  
  
Code to Review:  
```python  
{code_to_critique}
``` {data-source-line="87"}  
"""  
  
    critique = critic_llm.invoke(prompt)  
    return {"critique": critique.model_dump()}
```

The output here is a structured `Critique` object. This is way better than a vague **"this could be improved"** message because it gives our next step

specific, actionable feedback to work with.

The final piece of our logic is the `refiner_node` . It takes the original draft and the editor's feedback and creates the final, improved version.

```
def refiner_node(state):  
    """Refines the code based on the critique."""  
    console.print("--- 3. Refining Code ---")  
    # Initialize the LLM to return a structured RefinedCode object  
    refiner_llm = llm.with_structured_output(RefinedCode)  
  
    draft_code = state['draft']['code']  
    critique_suggestions = json.dumps(state['critique'], indent=2)  
  
    prompt = f"""You are an expert Python programmer tasked with refining a piece of code.  
  
Your goal is to rewrite the original code, implementing all the suggested improvements.  
  
**Original Code:**  
```python  
{draft_code}
``` {data-source-line="115"}  
  
**Critique and Suggestions:**  
{critique_suggestions}  
  
Please provide the final, refined code and a summary of the changes you made.  
"""  
  
    refined_code = refiner_llm.invoke(prompt)  
    return {"refined_code": refined_code.model_dump()}
```

Okay, we have our three logical pieces. Now, we need to wire them together

into a workflow. This is where LangGraph comes in. We'll define the `state` that gets passed between our nodes and then build the graph itself.

```
class ReflectionState(TypedDict):
    """Represents the state of our reflection graph."""
    user_request: str
    draft: Optional[dict]
    critique: Optional[dict]
    refined_code: Optional[dict]

# Initialize a new state graph
graph_builder = StateGraph(ReflectionState)

# Add the nodes to the graph
graph_builder.add_node("generator", generator_node)
graph_builder.add_node("critic", critic_node)
graph_builder.add_node("refiner", refiner_node)

# Define the workflow edges as a simple linear sequence
graph_builder.set_entry_point("generator")
graph_builder.add_edge("generator", "critic")
graph_builder.add_edge("critic", "refiner")
graph_builder.add_edge("refiner", END)

# Compile the graph into a runnable application
reflection_app = graph_builder.compile()
```

The flow is a simple, straight line: `generator -> critic -> refiner`. This is the

classic Reflection pattern, and now we are ready to test it.



Reflection Agent Architecture

Reflection Agent (Created by [Fareed Khan](#))

To test this workflow we will give it a classic coding problem where a naive first attempt is often inefficient, **finding the nth Fibonacci number**. This is a perfect test case because a simple recursive solution is easy to write but computationally expensive, leaving a lot of room for improvement.

```
user_request = "Write a Python function to find the nth Fibonacci number."
initial_input = {"user_request": user_request}

console.print(f"[bold cyan]🚀 Kicking off Reflection workflow for request:[/bold cyan] {user_request}")

# Stream the results and capture the final state
final_state = None
for state_update in reflection_app.stream(initial_input, stream_mode="values"):
    final_state = state_update

console.print("\n[bold green]✅ Reflection workflow complete![/bold green]")
```

Let's look at the **before and after** to see what our agent did.

```
--- ### Initial Draft ---
Explanation: This function uses a recursive approach to calculate the nth Fibonacci number.

1 def fibonacci(n):
2     if n <= 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         return fibonacci(n-1) + fibonacci(n-2)

--- ### Critique ---
```

Summary: The `function` has potential bugs and inefficiencies. It should be revised.
Improvements Suggested:

- The `function` does not handle negative numbers correctly.
- The `function` has a high time complexity due to the repeated calculations. Consider using an iterative approach.
- The `function` does not follow PEP 8 conventions...

--- `### Final Refined Code` ---

Refinement Summary: The original code has been revised to handle negative inputs

```
1 def fibonacci(n):
2     """Calculates the nth Fibonacci number."""
3     if n < 0:
4         raise ValueError("n must be a non-negative integer")
5     elif n == 0:
6         return 0
7     elif n == 1:
8         return 1
9     else:
10        fib = [0, 1]
11        for i in range(2, n + 1):
12            fib.append(fib[i-1] + fib[i-2])
13        return fib[n]
```

The initial draft is a simple recursive function correct, but terribly inefficient. The critic pointing out the exponential complexity and other issues. And the refined code is a much smarter and more robust iterative solution. This is a perfect example of the pattern working as intended.

To make this more concrete, let's bring in another LLM to act as an impartial

‘judge’ and score both the initial draft and the final code. This will give us a quantitative measure of the improvement.

```
class CodeEvaluation(BaseModel):
    """Schema for evaluating a piece of code."""
    correctness_score: int = Field(description="Score from 1-10 on whether the c")
    efficiency_score: int = Field(description="Score from 1-10 on the code's alg")
    style_score: int = Field(description="Score from 1-10 on code style and read")
    justification: str = Field(description="A brief justification for the scores")

def evaluate_code(code_to_evaluate: str):
    prompt = f"""You are an expert judge of Python code. Evaluate the following

    Code:
    ```python
 {code_to_evaluate}
    ```

    """
    return judge_llm.invoke(prompt)

if final_state and 'draft' in final_state and 'refined_code' in final_state:
    console.print("--- Evaluating Initial Draft ---")
    initial_draft_evaluation = evaluate_code(final_state['draft']['code'])
    console.print(initial_draft_evaluation.model_dump()) # Corrected: use .model_

    console.print("\n--- Evaluating Refined Code ---")
    refined_code_evaluation = evaluate_code(final_state['refined_code']['refined
    console.print(refined_code_evaluation.model_dump()) # Corrected: use .model_
else:
    console.print("[bold red]Error: Cannot perform evaluation because the `final
```

When we run the judge on both versions, the scores tell the whole story.

```
--- Evaluating Initial Draft ---
{
  'correctness_score': 2,
  'efficiency_score': 4,
  'style_score': 2,
  'justification': 'The function has a time complexity of  $O(2^n)$ ...'
}

--- Evaluating Refined Code ---
{
  'correctness_score': 8,
  'efficiency_score': 6,
  'style_score': 9,
  'justification': 'The code is correct... it has a time complexity of  $O(n)$ ...'
}
```

The initial draft gets terrible scores, especially for efficiency. The refined code sees a massive jump across the board.

This gives us hard proof that the reflection process didn't just change the code it made it somewhat

better.

Tool Using

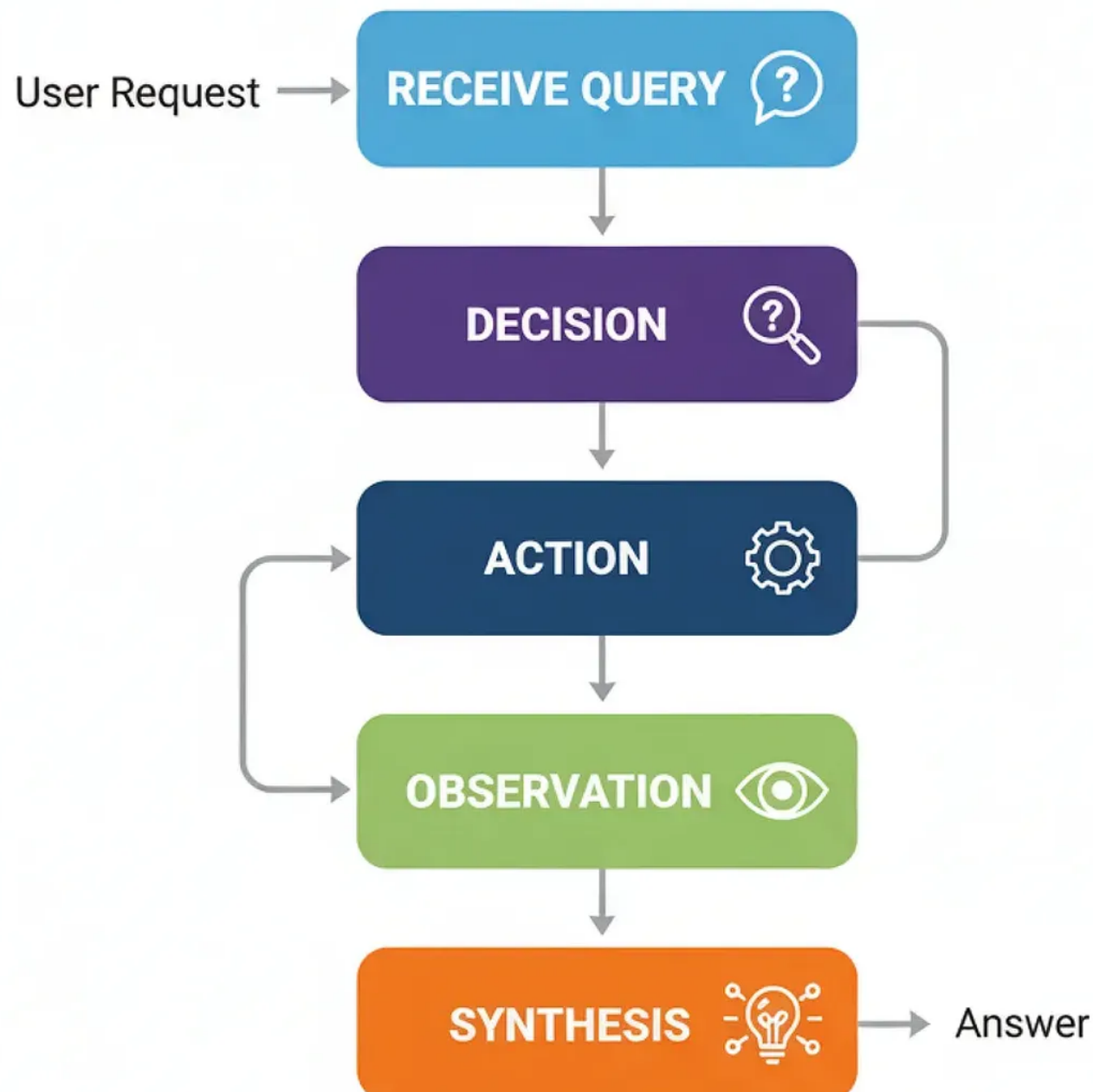
The Reflection pattern we just built is great for sharpening an agent internal reasoning.

But what happens when the agent needs information it doesn't already know?

Without access to external tools, an LLM is limited to its pre-trained parameters, it can generate and reason, but it cannot query new data or interact with the outside world. That's where our second architecture, Tool Use, comes in.

In any large-scale AI system, tool use isn't optional but important and a required component. It acts as the bridge between an agent reasoning and real-world data. Whether it's a support bot checking an order status, or a financial agent pulling live stock prices.

Let's understand how the process flows.



Tool using workflow (Created by [Fareed Khan](#))

1. **Receive Query:** The agent gets a request from the user.
2. **Decision:** The agent analyzes the query and looks at its available tools. It then decides if a tool is needed to answer the question accurately.
3. **Action:** If a tool is needed, the agent formats a call to that tool, for example, a specific function with the right arguments.
4. **Observation:** The system executes the tool call, and the result (the “observation”) is sent back to the agent.
5. **Synthesis:** The agent takes the tool’s output, combines it with its own reasoning, and generates a final, grounded answer for the user.

To build this, we need to give our agent a tool. For this, we will use the `TavilySearchResults` tool, which gives our agent the access to search the web. The most important part here is the **description**. The LLM reads this natural language description to figure out what the tool does and when it should be used, so making it clear and precise is key.

```
# Initialize the tool. We can set the max number of results to keep the context
search_tool = TavilySearchResults(max_results=2)

# It's crucial to give the tool a clear name and description for the agent
search_tool.name = "web_search"
search_tool.description = "A tool that can be used to search the internet for up

tools = [search_tool]
```

Now that we have a functional tool, we can build the agent that will learn how to use it. The state for a tool-using agent is pretty simple, it's just a list of messages that keeps track of the entire conversation history.

```
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

Next, we have to make the LLM “aware” of the tools we have given it. This is a critical step. We use the `.bind_tools()` method, which essentially injects the tool's name and description into the LLM's system prompt, allowing it to

decide when to call the tool.

```
llm = ChatNebius(model="meta-llama/Meta-Llama-3.1-8B-Instruct", temperature=0)

# Bind the tools to the LLM, making it tool-aware
llm_with_tools = llm.bind_tools(tools)
```

Now we can define our agent workflow using LangGraph. We need two main nodes: the `agent_node` (the "brain") which calls the LLM to decide what to do, and the `tool_node` (the "hands") which actually executes the tool.

```
def agent_node(state: AgentState):
    """The primary node that calls the LLM to decide the next action."""
    console.print("--- AGENT: Thinking... ---")
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

# The ToolNode is a pre-built node from LangGraph that executes tools
tool_node = ToolNode(tools)
```

After the `agent_node` runs, we need a router to decide where to go next. If the

agent's last message contains a `tool_calls` attribute, it means it wants to use a tool, so we route to the `tool_node`. If not, it means the agent has a final answer, and we can end the workflow.

```
def router_function(state: AgentState) -> str:
    """Inspects the agent's last message to decide the next step."""
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        # The agent has requested a tool call
        console.print("--- ROUTER: Decision is to call a tool. ---")
        return "call_tool"
    else:
        # The agent has provided a final answer
        console.print("--- ROUTER: Decision is to finish. ---")
        return "__end__"
```

Okay, we have all the pieces. Let's wire them together into a graph. The key here is the conditional edge that uses our `router_function` to create the agent's primary reasoning loop: `agent -> router -> tool -> agent`.

```
graph_builder = StateGraph(AgentState)

# Add the nodes
graph_builder.add_node("agent", agent_node)
```

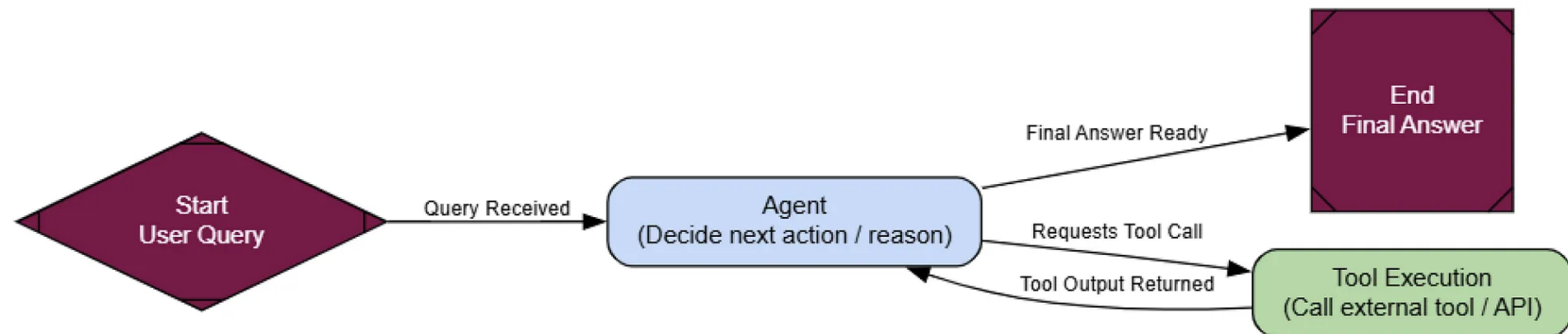
```
graph_builder.add_node("call_tool", tool_node)

# Set the entry point
graph_builder.set_entry_point("agent")

# Add the conditional router
graph_builder.add_conditional_edges(
    "agent",
    router_function,
)

# Add the edge from the tool node back to the agent to complete the loop
graph_builder.add_edge("call_tool", "agent")

# Compile the graph
tool_agent_app = graph_builder.compile()
```



Tool Use Agent Architecture

Tool Call architecture (Created by Fareed Khan)

Now let's test it. We'll give it a question that it couldn't possibly know from its training data, forcing it to use the web search tool to find a real-time answer.

```
user_query = "What were the main announcements from Apple's latest WWDC event?"
initial_input = {"messages": [("user", user_query)]}

console.print(f"[bold cyan]🚀 Kicking off Tool Use workflow for request:[/bold c

for chunk in tool_agent_app.stream(initial_input, stream_mode="values"):
    chunk["messages"][-1].pretty_print()
    console.print("\n---\n")

console.print("\n[bold green]✅ Tool Use workflow complete![/bold green]")
```

Let's look at the output to see the agent's thought process.

```
===== Human Message =====
What were the main announcements from Apple's latest WWDC event?
---
--- AGENT: Thinking... ---
--- ROUTER: Decision is to call a tool. ---
===== Ai Message =====
Tool Calls:
  web_search (call_abc123)
```

```
    Args:
      query: Apple WWDC latest announcements
    ---
    ===== Tool Message =====
    Name: web_search
    [{"title": "WWDC 2025: Everything We Know...", "url": "...", "content": "Apple's
    ---
    --- AGENT: Thinking... ---
    --- ROUTER: Decision is to finish. ---
    ===== Ai Message =====
    The main announcements from Apple's latest WWDC event include a new design that
```

The trace clearly shows the agent's logic:

1. First, the `agent_node` thinks and decides it needs to search the web, outputting a `tool_calls` request.
2. Next, the `tool_node` executes that search and returns a `ToolMessage` with the raw web results.
3. Finally, the `agent_node` runs again, this time taking the search results as context to synthesize a final, helpful answer for the user.

To formalize this, we'll bring in our LLM-as-a-Judge again, but with criteria specifically for evaluating tool use.

```
class ToolUseEvaluation(BaseModel):  
    """Schema for evaluating the agent's tool use and final answer."""  
    tool_selection_score: int = Field(description="Score 1-5 on whether the agent  
    tool_input_score: int = Field(description="Score 1-5 on how well-formed and  
    synthesis_quality_score: int = Field(description="Score 1-5 on how well the  
    justification: str = Field(description="A brief justification for the scores
```

When we run the judge on the full conversation trace, we get back a structured evaluation.

```
--- Evaluating Tool Use Performance ---  
{  
    'tool_selection_score': 5,  
    'tool_input_score': 5,  
    'synthesis_quality_score': 4,  
    'justification': "The AI agent correctly used the web search tool to find re  
}
```

The high scores give us an evidence that our agent

isn't just calling the tool, but actually using it effectively.

It correctly identified when to search, what to search for, and how to use the results. This architecture is a fundamental building block for almost any practical AI assistant.

ReAct (Reason + Act)

Our last agent was a big step up. It can use tools to fetch live data, which is huge. The thing is, it's a bit of a one-shot deal, it decides it needs a tool, calls it once, and then tries to answer.

But what happens when a problem is more complex and needs multiple, dependent steps to solve?

ReAct (Reason + Act) is all about creating a loop. It enables an agent to dynamically **reason** about what to do next, take an **action** (like calling a tool), **observe** the result, and then use that new information to **reason** again. It's a

move from a static tool-caller to an adaptive problem-solver.

In any AI system, ReAct is your go-to pattern for any task that requires multi-hop reasoning.

Let's understand how the process flows.





ReAct Workflow (Created by [Fareed Khan](#))

1. **Receive Goal:** The agent is given a complex task that can't be solved in one step.
2. **Think (Reason):** The agent generates a thought, like: “To answer this, I first need to find piece of information X”.
3. **Act:** Based on that thought, it executes an action, like calling a search tool for ‘X’.
4. **Observe:** The agent gets the result for ‘X’ back from the tool.
5. **Repeat:** It takes that new information and goes back to step 2, thinking: “Okay, now that I have X, I need to use it to find Y”. This loop continues until the final goal is met.

The good news is, we have already built most of the pieces. We will reuse `AgentState`, `web_search`, and `tool_node`. The only change in our graph logic: after `tool_node` runs, we send the output back to `agent_node` instead of ending. This creates the reasoning loop, letting the agent review results and choose the next step.

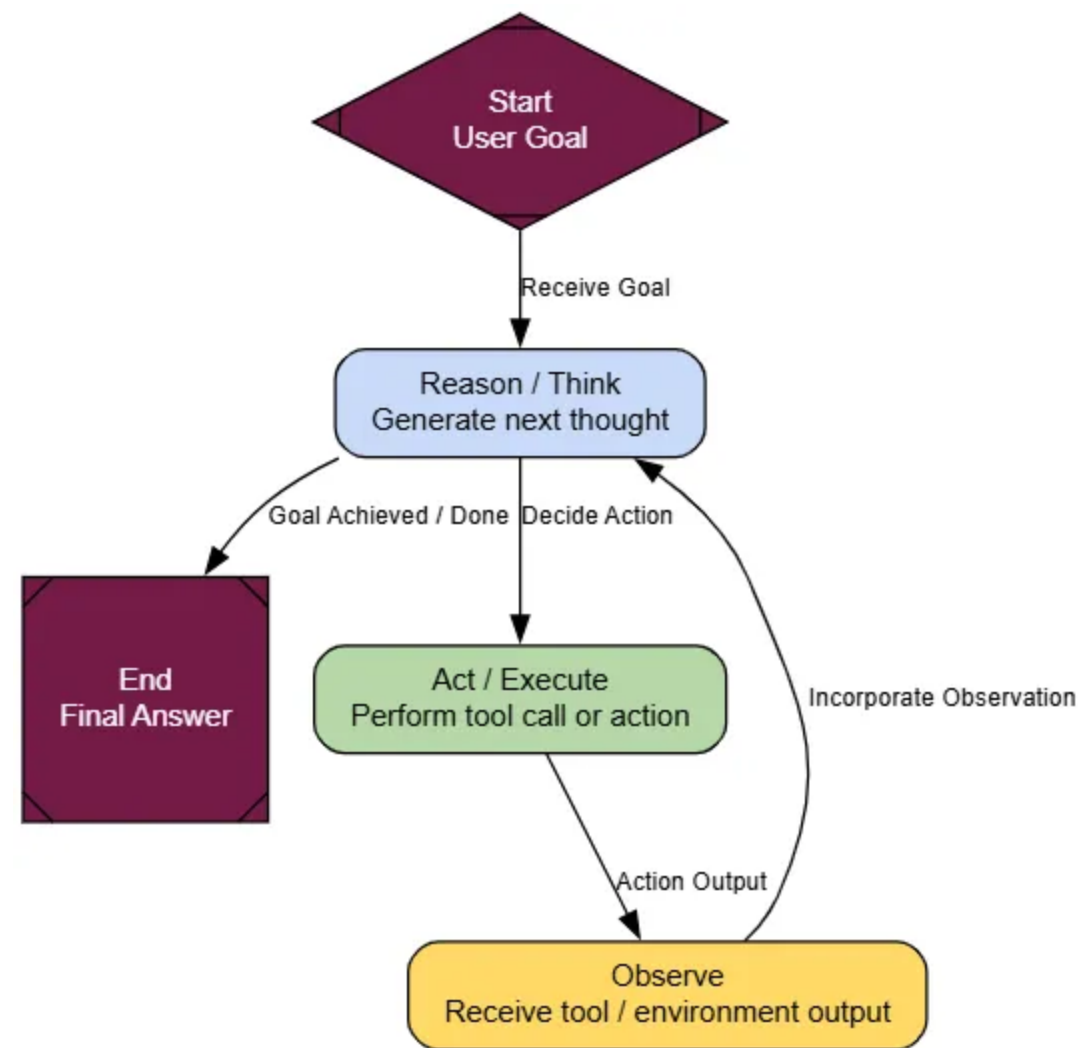
```
def react_agent_node(state: AgentState):
    """The agent node that thinks and decides the next step."""
    console.print("--- REACT AGENT: Thinking... ---")
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

# The ReAct graph with its characteristic loop
react_graph_builder = StateGraph(AgentState)
react_graph_builder.add_node("agent", react_agent_node)
react_graph_builder.add_node("tools", tool_node) # We reuse the tool_node from b

react_graph_builder.set_entry_point("agent")
react_graph_builder.add_conditional_edges(
    "agent",
    # We can reuse the same router function
    router_function,
    # The map now defines the routing logic
    {"call_tool": "tools", "__end__": "__end__"}
)

# This is the key difference: the edge goes from the tool node BACK to the agent
react_graph_builder.add_edge("tools", "agent")

react_agent_app = react_graph_builder.compile()
```



ReAct Agent Architecture

ReAct Architecture (Created by [Fareed Khan](#))

And that's it. The only real change is `react_graph_builder.add_edge("tools", "agent")` . That single line creates the loop and turns our simple tool-user into a dynamic ReAct agent.

To see why this loop is so powerful, let's give it a task that's impossible to solve in one shot a classic multi-hop question. A simple tool-using agent would fail this because it can't chain the steps together.

```
multi_step_query = "Who is the current CEO of the company that created the sci-f  
console.print(f"[bold yellow]Testing ReAct agent on a multi-step query:[/bold ye  
final_react_output = None  
# Stream the output to see the step-by-step reasoning  
for chunk in react_agent_app.stream({"messages": [("user", multi_step_query)]},  
    final_react_output = chunk  
    console.print(f"--- [bold purple]Current State Update[/bold purple] ---")  
    chunk['messages'][-1].pretty_print()  
    console.print("\n")  
  
console.print("\n--- [bold green]Final Output from ReAct Agent[/bold green] ---")  
console.print(Markdown(final_react_output['messages'][-1].content))
```

```
--- Human Message ---
```

```
Who is CEO of company that made 'Dune'...

--- REACT Agent: Thinking... ---
--- ROUTER: Call a tool...

--- Ai Message ---
Tool Calls: web_search...
Args: query: current CEO of company that made Dune...

--- Tool Message ---
[{"title": "Dune: Part Three - Wikipedia", "content": "Legendary CEO Joshua Grod

--- REACT Agent: Thinking... ---
--- ROUTER: Finish...

--- Final Output ---
CEO of company that made 'Dune' is Joshua Grode...
Budget for most recent film not found...
```

When we run this, the agent's execution trace shows a far more intelligent process. It doesn't just make one search. Instead, it reasons its way through the problem:

1. **Thought 1:** “First, I need to find out which company made the movie ‘Dune’.”
2. **Action 1:** It calls `web_search('production company for Dune movie')`.
3. **Observation 1:** It gets back “Legendary Entertainment”.

4. **Thought 2:** “Okay, now I need the CEO of Legendary Entertainment.”

5. **Action 2:** It calls `web_search('CEO of Legendary Entertainment')` and so on, until it has all the pieces.

To formalize the improvement, we can use our LLM-as-a-Judge again, this time with a focus on task completion.

```
class TaskEvaluation(BaseModel):
    """Schema for evaluating an agent's ability to complete a task."""
    task_completion_score: int = Field(description="Score 1-10 on whether the agent completed the task.")
    reasoning_quality_score: int = Field(description="Score 1-10 on the logical justification of the scores.")
    justification: str = Field(description="A brief justification for the scores.")

def evaluate_agent_output(query: str, agent_output: dict):
    """Runs an LLM-as-a-Judge to evaluate the agent's final performance."""
    trace = "\n".join([f"{m.type}: {m.content}" for m in agent_output['messages']])

    prompt = f"""You are an expert judge of AI agents. Evaluate the following agent's performance based on the provided query and conversation trace.

    **User's Task:**
    {query}

    **Full Agent Conversation Trace:**
    {trace}

    """

    judge_llm = llm.with_structured_output(TaskEvaluation)
```

```
return judge_llm.invoke(prompt)
```

Let's see the scores. A basic agent trying this task would get a very low score, as it would fail to gather all the required info. Our ReAct agent, however, performs much better.

```
--- Evaluating ReAct Agent's Output ---  
{  
  'task_completion_score': 8,  
  'reasoning_quality_score': 9,  
  'justification': "The agent correctly broke down the problem into multiple s  
}
```

We can see that the `reasoning_quality_score` confirms that its step-by-step process was logical and validated by our judge (LLM).

The ReAct pattern gives the agent an ability to tackle these kinds of complex, multi-hop problems that require dynamic thinking.

Planning

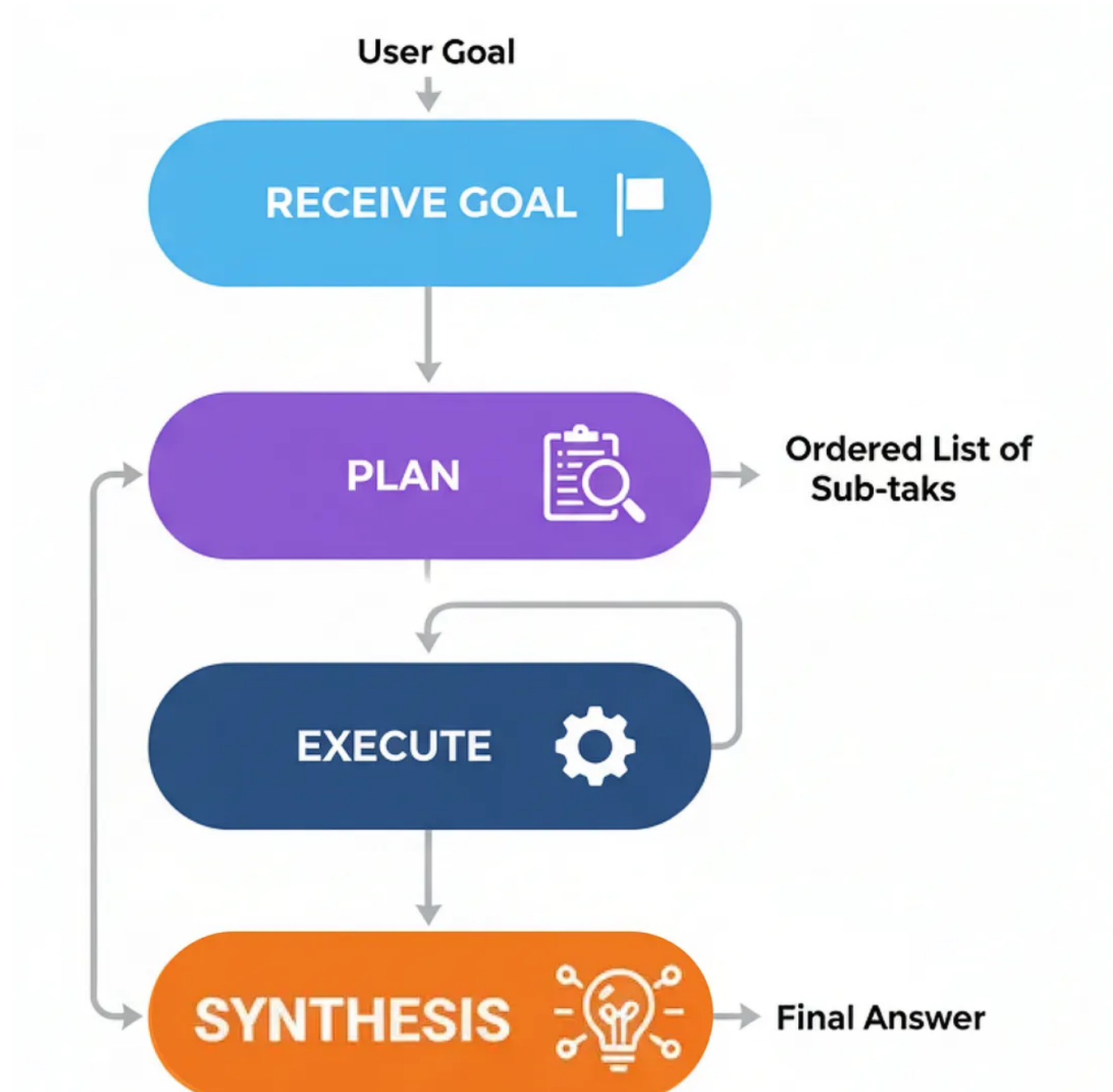
The ReAct pattern is fantastic for exploring a problem and figuring things out on the fly. But it can be a bit inefficient for tasks where the steps are predictable. It's like a person asking for directions one turn at a time, instead of just looking at the full map first. This is where the **Planning** architecture comes in.

This pattern introduces a crucial layer of foresight.

Instead of reacting step-by-step, a planning agent first creates a full 'battle plan' before taking any action.

In AI system, Planning is your workhorse for any structured, multi-step process. Think about data processing pipelines, report generation, or any workflow where you know the sequence of operations ahead of time. It brings predictability and efficiency, making the agent's behavior easier to trace and debug.

Let's understand how the process flows.



Planning approach (Created by Fareed Khan)

1. **Receive Goal:** The agent is given a complex task.
2. **Plan:** A dedicated 'Planner' component analyzes the goal and generates an ordered list of sub-tasks required to achieve it. For example: ["Find fact A", "Find fact B", "Calculate C using A and B"] .
3. **Execute:** An 'Executor' component takes the plan and carries out each sub-task in sequence, using tools as needed.
4. **Synthesize:** Once all steps in the plan are complete, a final component assembles the results from the executed steps into a coherent final answer.

Let's start building it.

We'll create three core components: a `planner_node` to create the strategy, an `executor_node` to carry it out, and a `synthesizer_node` to assemble the final report.

First up, we need a dedicated `planner_node` . The key here is a very explicit prompt that tells the LLM its job is to create a list of simple, executable steps.

```

class Plan(BaseModel):
    """A plan of tool calls to execute to answer the user's query."""
    steps: List[str] = Field(description="A list of tool calls that, when execut

class PlanningState(TypedDict):
    user_request: str
    plan: Optional[List[str]]
    intermediate_steps: List[str] # Will store tool outputs
    final_answer: Optional[str]
def planner_node(state: PlanningState):
    """Generates a plan of action to answer the user's request."""
    console.print("--- PLANNER: Decomposing task... ---")
    planner_llm = llm.with_structured_output(Plan)

    prompt = f"""You are an expert planner. Your job is to create a step-by-step
        Each step in the plan must be a single call to the `web_search` tool.
        **User's Request:**
        {state['user_request']}
    """
    plan_result = planner_llm.invoke(prompt)
    console.print(f"--- PLANNER: Generated Plan: {plan_result.steps} ---")
    return {"plan": plan_result.steps}

```

Next, the `executor_node`. This is a straightforward worker that just takes the next step from the plan, runs the tool, and adds the result to our state.

```
def executor_node(state: PlanningState):  
    """Executes the next step in the plan."""  
    console.print("--- EXECUTOR: Running next step... ---")  
    next_step = state["plan"][0]  
  
    # In a real app, you'd parse the tool name and args. Here we assume 'web_search'  
    query = next_step.replace("web_search('", "").replace("'", "")  
  
    result = search_tool.invoke({"query": query})  
  
    return {  
        "plan": state["plan"][1:], # Pop the executed step  
        "intermediate_steps": state["intermediate_steps"] + [result]  
    }
```

Now we just need to wire them together in a graph. A router will check if there are any steps left in the plan. If there are, it loops back to the executor. If not, it moves on to a final `synthesizer_node` (which we can reuse from a previous pattern) to generate the answer.

```
def planning_router(state: PlanningState):  
    """Routes to the executor or synthesizer based on the plan."""  
    if not state["plan"]:  
        console.print("--- ROUTER: Plan complete. Moving to synthesizer. ---")
```



```

        return "synthesize"
    else:
        console.print("--- ROUTER: Plan has more steps. Continuing execution. --")
        return "execute"

```

```

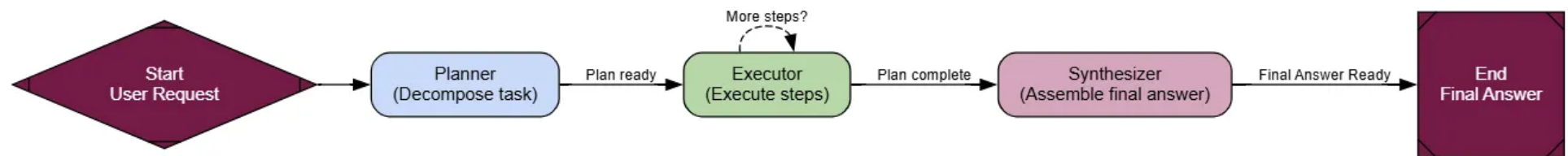
planning_graph_builder = StateGraph(PlanningState)
planning_graph_builder.add_node("plan", planner_node)
planning_graph_builder.add_node("execute", executor_node)

planning_graph_builder.add_node("synthesize", synthesizer_node)
planning_graph_builder.set_entry_point("plan")
planning_graph_builder.add_conditional_edges("plan", planning_router)
planning_graph_builder.add_conditional_edges("execute", planning_router)

planning_graph_builder.add_edge("synthesize", END)

planning_agent_app = planning_graph_builder.compile()

```



Planning Agent Architecture

Planning (created by Fareed Khan)

To really see the difference, let's give our agent a task that benefits from foresight. A ReAct agent can solve this, but its step-by-step process is less

direct.

```
plan_centric_query = """
Find the population of the capital cities of France, Germany, and Italy.
Then calculate their combined total.
"""

console.print(f"[bold green]Testing PLANNING agent on a plan-centric query:[/bold green]")

# Initialize the state correctly, especially the list for intermediate steps
initial_planning_input = {"user_request": plan_centric_query, "intermediate_steps": []}
final_planning_output = planning_agent_app.invoke(initial_planning_input)

console.print("\n--- [bold green]Final Output from Planning Agent[/bold green] ---")
console.print(Markdown(final_planning_output['final_answer']))
```

The difference in the process is immediately clear. The very first thing our agent does is lay out its entire strategy.

```
--- PLANNER: Decomposing task... ---
--- PLANNER: Generated Plan: ["web_search('population of Paris')", "web_search('Germany capital')", "web_search('France capital')", "web_search('Italy capital')", "calculate_total('population of Paris', 'Germany capital', 'France capital', 'Italy capital')] ---
--- ROUTER: Plan has more steps. Continuing execution. ---
--- EXECUTOR: Running next step... ---
...
```

The agent created a complete, explicit plan before it took a single action. It then executed this plan methodically. This process is more transparent and robust because it's following a clear set of instructions.

To formalize this, we'll use our LLM-as-a-Judge, but this time we'll score the *efficiency* of the process.

```
class ProcessEvaluation(BaseModel):  
    """Schema for evaluating an agent's problem-solving process."""  
    task_completion_score: int = Field(description="Score 1-10 on task completion")  
    process_efficiency_score: int = Field(description="Score 1-10 on the efficiency of the process")  
    justification: str = Field(description="A brief justification for the scores")
```

When judged, the Planning agent shines in its directness.

```
--- Evaluating Planning Agent's Process ---  
{  
  'task_completion_score': 8,  
  'process_efficiency_score': 9,  
  'justification': "The agent created a clear, optimal plan upfront and execut  
}
```

We are getting good score, that means our approach does create a correct planning system so ...

when the solution path is predictable, Planning offers a more structured and efficient approach than a purely reactive one.

PEV (Planner-Executor-Verifier)

Our Planning agent works well when the path is clear, it makes a plan and follows it. But there's a hidden assumption ...

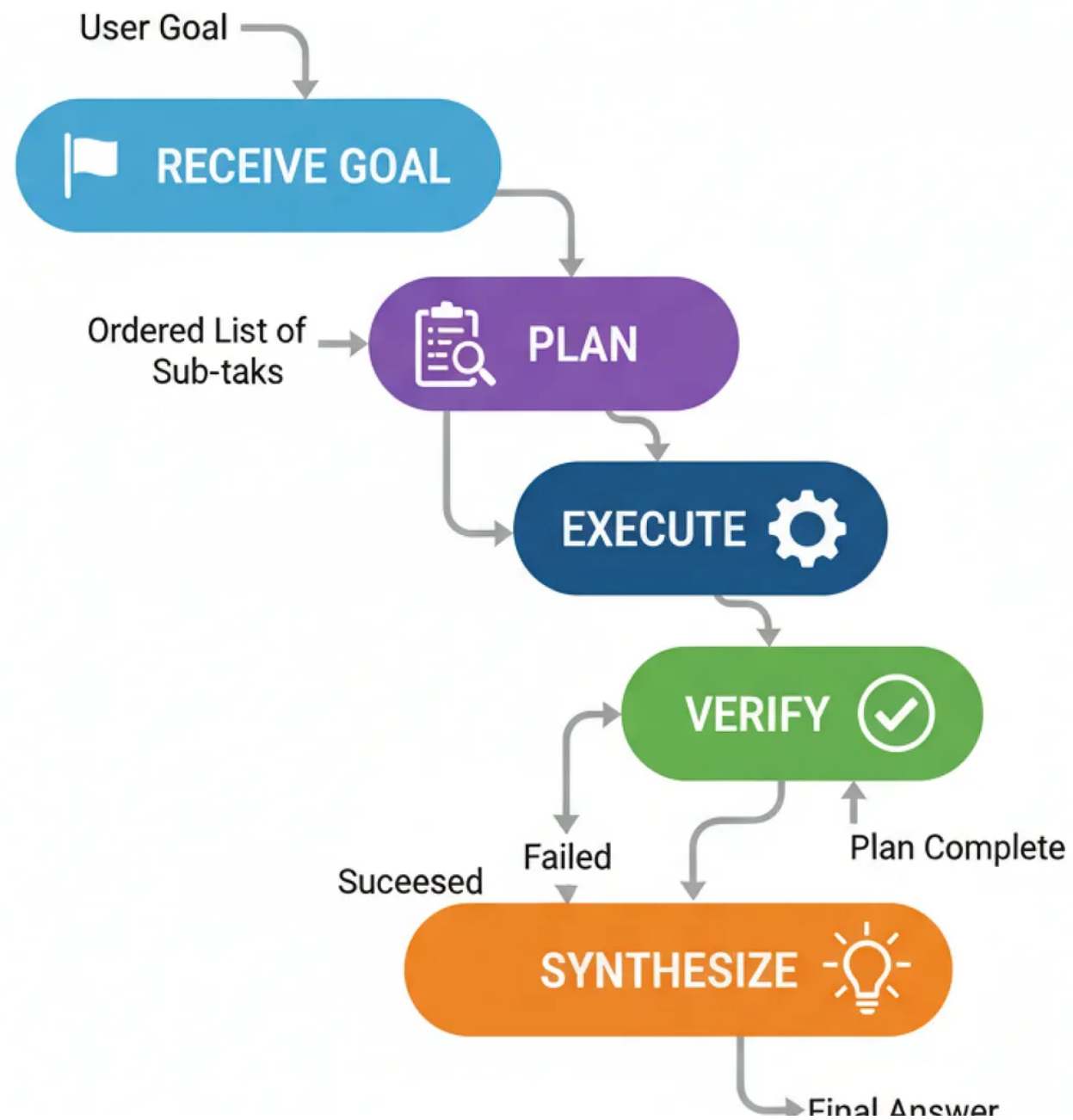
what happens when things go wrong? If a tool fails, an

API is down, or search returns junk, a standard planner just passes the error along, ending in failure or nonsense.

PEV (Planner-Executor-Verifier) architecture is a simple but powerful upgrade to the Planning pattern that adds a critical layer of quality control and self-correction.

PEV is important for building robust and reliable workflows. You do use it anywhere an agent interacts with external tools that might be unreliable.

This is how it works ...



PEV (Created by [Fareed Khan](#))

1. **Plan:** A ‘Planner’ agent creates a sequence of steps, just like before.
2. **Execute:** An ‘Executor’ agent takes the *next* step from the plan and calls the tool.
3. **Verify:** This is the new step. A ‘Verifier’ agent examines the tool’s output. It checks for correctness, relevance, and errors.
4. **Route & Iterate:** Based on the Verifier’s judgment:
 - If the step **succeeded**, the agent moves to the next step in the plan.
 - If the step **failed**, the agent loops back to the Planner to create a *new plan*, now aware of the failure.
 - If the plan is complete, it proceeds to the end.

To really show this off, we need a tool that can actually fail. So, we’ll create a special `flaky_web_search` tool that we’ll design to intentionally return an error message for a specific query.

```
def flaky_web_search(query: str) -> str:
    """A web search tool that intentionally fails for a specific query."""
    console.print(f"--- TOOL: Searching for '{query}'... ---")
    if "employee count" in query.lower():
        console.print("--- TOOL: [bold red]Simulating API failure![/bold red] --")
        return "Error: Could not retrieve data. The API endpoint is currently un
    else:
        return search_tool.invoke({"query": query})
```

Now for the core of the PEV pattern: the `verifier_node`. This node's only job is to look at the last tool's output and decide if it was a success or a failure.

```
class VerificationResult(BaseModel):
    """Schema for the Verifier's output."""
    is_successful: bool = Field(description="True if the tool execution was succ
    reasoning: str = Field(description="Reasoning for the verification decision.

def verifier_node(state: PEVState):
    """Checks the last tool result for errors."""
    console.print("--- VERIFIER: Checking last tool result... ---")
    verifier_llm = llm.with_structured_output(VerificationResult)
    prompt = f"Verify if the following tool output is a successful, valid result
    verification = verifier_llm.invoke(prompt)
    console.print(f"--- VERIFIER: Judgment is '{'Success' if verification.is_suc
    if verification.is_successful:
```



```
        return {"intermediate_steps": state["intermediate_steps"] + [state['last_tool_result']]
    else:
        # If it failed, we add the failure reason and clear the plan to trigger
        return {"plan": [], "intermediate_steps": state["intermediate_steps"] + [state['last_tool_result']]}
```

With our verifier ready, we can wire up the full graph. The key here is the router logic. After the `verifier_node` runs, if the plan is suddenly empty (because the verifier cleared it), our router knows to send the agent back to the `planner_node` to try again.

```
class PEVState(TypedDict):
    user_request: str
    plan: Optional[List[str]]
    last_tool_result: Optional[str]
    intermediate_steps: List[str]
    final_answer: Optional[str]
    retries: int

# ... (planner_node, executor_node, and synthesizer_node definitions are similar)
def pev_router(state: PEVState):
    """Routes execution based on verification and plan status."""
    if not state["plan"]:
        # Check if the plan is empty because verification failed
        if state["intermediate_steps"] and "Verification Failed" in state["intermediate_steps"][-1]:
            console.print("--- ROUTER: Verification failed. Re-planning... ---")
            return "plan"
        else:
            console.print("--- ROUTER: Plan complete. Moving to synthesizer. ---")
```

```

        return "synthesize"
    else:
        console.print("--- ROUTER: Plan has more steps. Continuing execution. ---")
        return "execute"

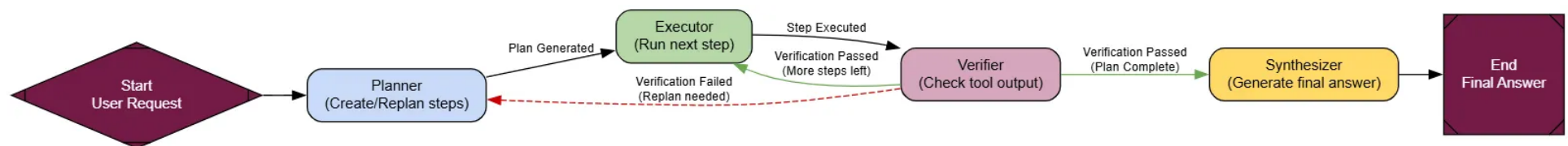
pev_graph_builder = StateGraph(PEVState)

pev_graph_builder.add_node("plan", pev_planner_node)
pev_graph_builder.add_node("execute", pev_executor_node)
pev_graph_builder.add_node("verify", verifier_node)
pev_graph_builder.add_node("synthesize", synthesizer_node)

pev_graph_builder.set_entry_point("plan")
pev_graph_builder.add_edge("plan", "execute")
pev_graph_builder.add_edge("execute", "verify")
pev_graph_builder.add_conditional_edges("verify", pev_router)

pev_graph_builder.add_edge("synthesize", END)
pev_agent_app = pev_graph_builder.compile()

```



Planner → Executor → Verifier (PEV) Architecture

PEV architecture (Created by [Fareed Khan](#))

Now for the crucial test. We will give our PEV agent a task that requires it to call our `flaky_web_search` tool with the query we know will fail. A simple

Planner-Executor agent would break here.

```
flaky_query = "What was Apple's R&D spend in their last fiscal year, and what wa

console.print(f"[bold green]Testing PEV agent on a flaky query:[/bold green]\n'{
initial_pev_input = {"user_request": flaky_query, "intermediate_steps": [], "ret

final_pev_output = pev_agent_app.invoke(initial_pev_input)

console.print("\n--- [bold green]Final Output from PEV Agent[/bold green] ---")
console.print(Markdown(final_pev_output['final_answer']))
```

```
Testing PEV agent on the same flaky query:
'What was Apple's R&D spend in their last fiscal year, and what was their total
spend per employee.'
```

```
--- (PEV) PLANNER: Creating/revising plan (retry 0)... ---
--- (PEV) EXECUTOR: Running next step... ---
--- TOOL: Searching for 'Apple R&D spend last fiscal year'... ---
--- VERIFIER: Checking last tool result... ---
--- VERIFIER: Judgment is 'Success' ---

...
```

1. **Plan 1:** The agent creates a plan: `["Apple R&D spend...", "Apple total employee count"]`.

2. **Execute & Fail:** It gets the R&D spend, but the `employee count` search hits our flaky tool and returns an error.
3. **Verify & Catch:** The `verifier_node` gets the error message, correctly judges it as a failure, and clears the plan.
4. **Router & Re-plan:** The router sees the empty plan and the failure message, and sends the agent back to the `planner_node`.
5. **Plan 2:** The planner, now aware that “Apple total employee count” failed, creates a new, smarter plan, perhaps trying `web_search('Apple number of employees worldwide')`.
6. **Execute & Succeed:** This new plan works, and the agent gets all the data it needs.

The final output is the correct calculation. The agent didn’t just give up; it detected the problem, thought of a new approach, and succeeded.

To formalize this, our LLM-as-a-Judge needs to score for robustness.

```
class RobustnessEvaluation(BaseModel):  
    """Schema for evaluating an agent's robustness and error handling."""  
    task_completion_score: int = Field(description="Score 1-10 on task completio
```

```
error_handling_score: int = Field(description="Score 1-10 on the agent's abi  
justification: str = Field(description="A brief justification for the scores
```

A standard Planner-Executor agent would get a terrible
`error_handling_score` . Our PEV agent, however, excels.

```
--- Evaluating PEV Agent's Robustness ---  
{  
    'task_completion_score': 8,  
    'error_handling_score': 10,  
    'justification': "The agent demonstrated perfect robustness. It successfully  
}
```

You can see that PEV architecture isn't just about getting the right answer
when things go well ...

It's about not getting the wrong answer when things
go wrong.

Tree-of-Thoughts (ToT)

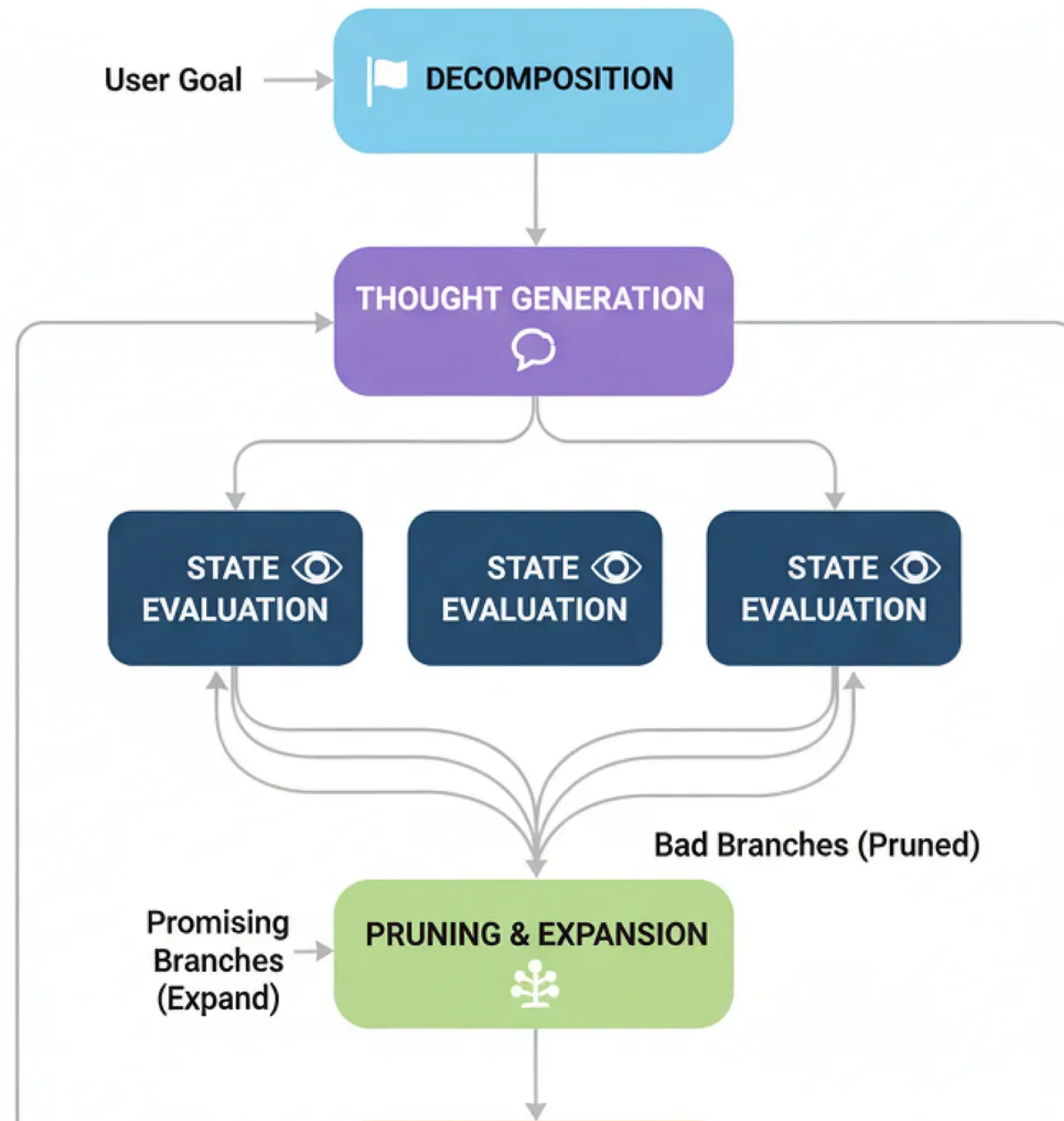
PEV pattern can handle a tool failure and try again with a new plan. But the planning itself is still linear. It creates a single, step-by-step plan and follows it.

What happens when the problem isn't a straight road, but more like a maze with dead ends and multiple possible paths?

This is where the **Tree-of-Thoughts (ToT)** architecture comes in. Instead of generating a single line of reasoning, a ToT agent explores multiple paths at once. It generates a few possible next steps, evaluates them, throws away the bad ones, and continues exploring the most promising branches.

In big AI systems, ToT helps solve hard problems like planning routes, scheduling, or tricky puzzles.

This is what how a normal ToT process works ...





TOT process (Created by [Fareed Khan](#))

1. **Decomposition:** The problem is broken down into a series of steps or “thoughts”.
2. **Thought Generation:** For the current state, the agent generates multiple potential next steps. This creates the branches of our “thought tree”.
3. **State Evaluation:** Each new potential step is evaluated by a critic or a validation function. This check determines if a move is valid, if it’s making progress, or if it’s just going in circles.
4. **Pruning & Expansion:** The agent then “prunes” the bad branches (the invalid or unpromising ones) and continues the process from the remaining good ones.
5. **Solution:** This continues until one of the branches reaches the final goal.

To showcase ToT, we need a problem that can’t be solved in a straight line, the Wolf, Goat, and Cabbage river crossing puzzle. It’s perfect since it requires non-obvious moves (like bringing something back) and has invalid

states that can trap a simple reasoner.

First, we will define the puzzle rules with a Pydantic model for the state and a function to check validity (so nothing gets eaten).

```
class PuzzleState(BaseModel):
    """Represents the state of the Wolf, Goat, and Cabbage puzzle."""
    left_bank: set[str] = Field(default_factory=lambda: {"wolf", "goat", "cabbage"})
    right_bank: set[str] = Field(default_factory=set)
    boat_location: str = "left"
    move_description: str = "Initial state."

    def is_valid(self) -> bool:
        """Checks if the current state is valid."""
        # Check left bank for invalid pairs if boat is on the right
        if self.boat_location == "right":
            if "wolf" in self.left_bank and "goat" in self.left_bank: return False
            if "goat" in self.left_bank and "cabbage" in self.left_bank: return False
        # Check right bank for invalid pairs if boat is on the left
        if self.boat_location == "left":
            if "wolf" in self.right_bank and "goat" in self.right_bank: return False
            if "goat" in self.right_bank and "cabbage" in self.right_bank: return False
        return True

    def is_goal(self) -> bool:
        """Checks if we've won."""
        return self.right_bank == {"wolf", "goat", "cabbage"}

    # Make the state hashable so we can detect cycles
    def __hash__(self):
        return hash((frozenset(self.left_bank), frozenset(self.right_bank), self.boat_location))
```

Now for the core of the ToT agent. The state of our graph will hold all the active paths in our thought tree. The `expand_paths` node will generate new

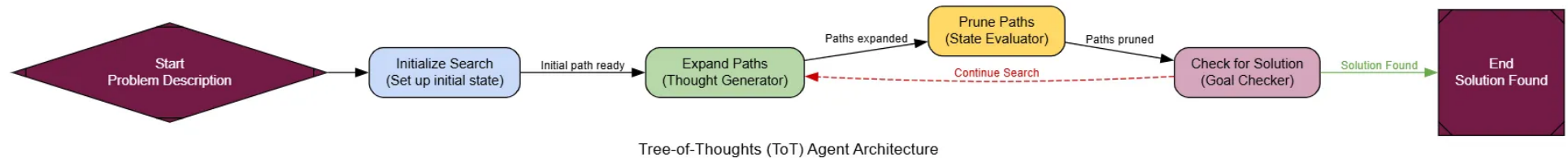
branches, and the `prune_paths` node will trim the tree by removing any paths that hit a dead end or go in circles.

```
class ToTState(TypedDict):
    problem_description: str
    active_paths: List[List[PuzzleState]] # This is our "tree"
    solution: Optional[List[PuzzleState]]

def expand_paths(state: ToTState) -> Dict[str, Any]:
    """The 'Thought Generator'. Expands each active path with all valid next moves"""
    console.print("--- Expanding Paths ---")
    new_paths = []
    for path in state['active_paths']:
        last_state = path[-1]
        # Get all valid next states from the current state
        possible_next_states = get_possible_moves(last_state) # Assuming get_possible_moves is defined
        for next_state in possible_next_states:
            new_paths.append(path + [next_state])
    console.print(f"[cyan]Expanded to {len(new_paths)} potential paths.[/cyan]")
    return {"active_paths": new_paths}

def prune_paths(state: ToTState) -> Dict[str, Any]:
    """The 'State Evaluator'. Prunes paths that are invalid or contain cycles."""
    console.print("--- Pruning Paths ---")
    pruned_paths = []
    for path in state['active_paths']:
        # Check for cycles: if the last state has appeared before in the path
        if path[-1] in path[:-1]:
            continue # Found a cycle, prune this path
        pruned_paths.append(path)
    console.print(f"[green]Pruned down to {len(pruned_paths)} valid, non-cyclical paths.[/green]")
    return {"active_paths": pruned_paths}
```

```
# ... (graph wiring with a conditional edge that checks for a solution) ...
workflow = StateGraph(TotState)
workflow.add_node("expand", expand_paths)
workflow.add_node("prune", prune_paths)
```



ToT architecture (Created by [Fareed Khan](#))

Let's run our ToT agent on the puzzle. A simple Chain-of-Thought agent might solve this if it's seen it before, but it's just recalling a solution. Our ToT agent will *discover* the solution through systematic search.

```
problem = "A farmer wants to cross a river with a wolf, a goat, and a cabbage..."

console.print("--- 🌳 Running Tree-of-Thoughts Agent ---")
final_state = tot_agent.invoke({"problem_description": problem}, {"recursion_limit": 10})
console.print("\n--- ✅ ToT Agent Solution ---")
```

The output trace shows the agent at work, methodically exploring the puzzle.

```
--- Expanding Paths ---  
[cyan]Expanded to 1 potential paths.[/cyan]  
--- Pruning Paths ---  
[green]Pruned down to 1 valid, non-cyclical paths.[/green]  
--- Expanding Paths ---  
[cyan]Expanded to 2 potential paths.[/cyan]  
--- Pruning Paths ---  
[green]Pruned down to 2 valid, non-cyclical paths.[/green]  
...  
[bold green]Solution Found![/bold green]  
  
---  ToT Agent Solution ---  
1. Initial state.  
2. Move goat to the right bank.  
3. Move the boat empty to the left bank.  
4. Move wolf to the right bank.  
5. Move goat to the left bank.  
6. Move cabbage to the right bank.  
7. Move the boat empty to the left bank.  
8. Move goat to the right bank.
```

The agent found the correct 8-step solution! It didn't just guess; it systematically explored possibilities, threw away the bad ones, and found a path that was guaranteed to be valid. This is the power of ToT.

To formalize this, we'll use an LLM-as-a-Judge focused on the quality of the

reasoning process.

```
class ReasoningEvaluation(BaseModel):  
    """Schema for evaluating an agent's reasoning process."""  
    solution_correctness_score: int = Field(description="Score 1-10 on whether t  
    reasoning_robustness_score: int = Field(description="Score 1-10 on how robus  
    justification: str = Field(description="A brief justification for the scores
```

A simple Chain-of-Thought agent might get a high correctness score if it's lucky, but its robustness score would be low. Our ToT agent, however, gets top marks.

```
--- Evaluating ToT Agent's Process ---  
{  
    'solution_correctness_score': 8,  
    'reasoning_robustness_score': 9,  
    'justification': "The agent's process was perfectly robust. It didn't just p  
}
```

We can see that the ToT agent works well not by chance, but because its search is solid.

That makes it a better choice for tasks that need high reliability.

Multi-Agent Systems

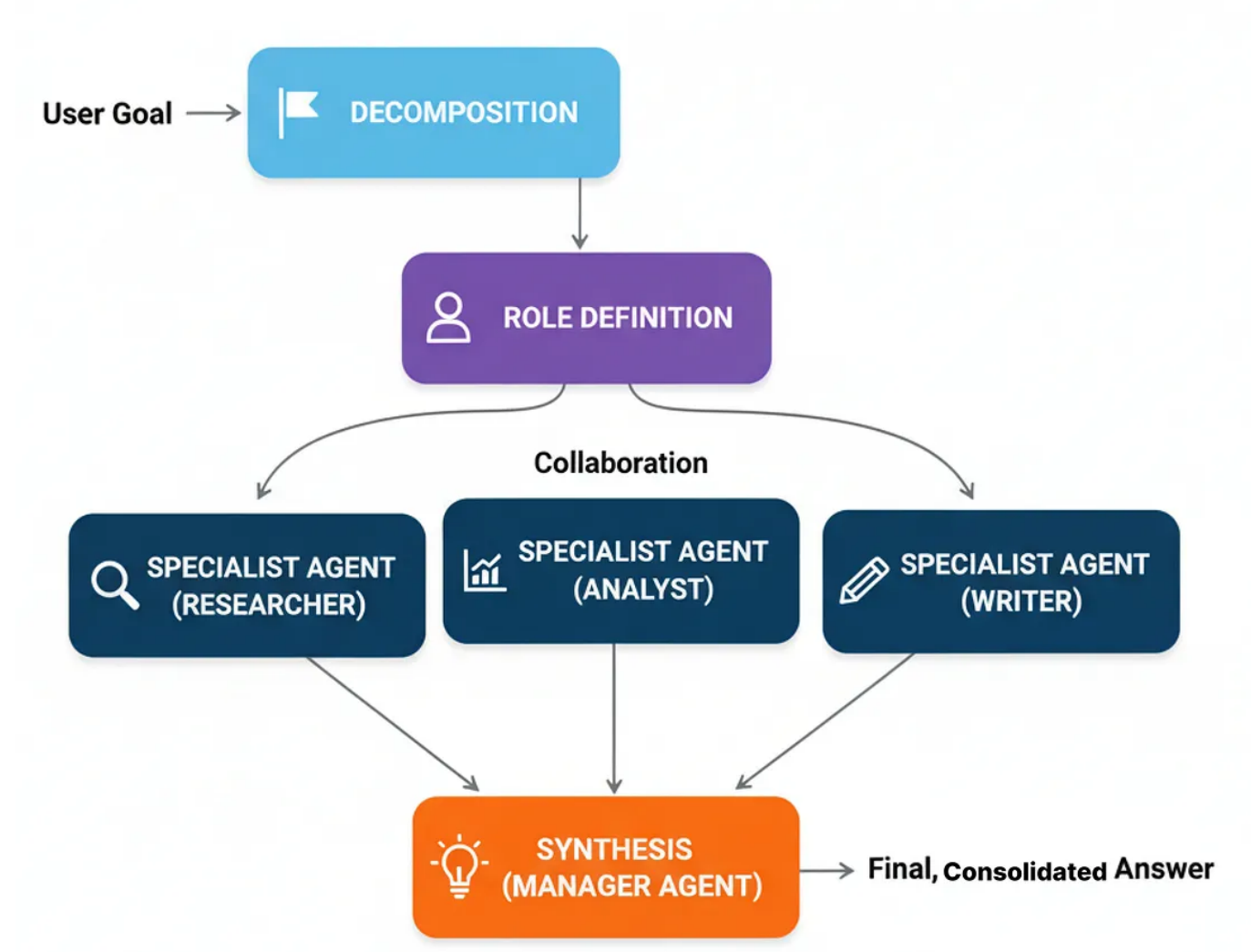
All of the approaches we have implemented so far, they have all been working alone.

What happens when a problem is too big or too complex for a single agent to handle effectively?

Instead of building one super-agent that does everything, multi-agent systems use a team of specialists. Each agent focuses on its own domain, just like human experts do, you wouldn't ask a data scientist to write marketing copy.

For complex tasks, like generating a market analysis, you can have a news expert, a financial expert, and a stock expert working together for a better result.

Multi-Agent system can be very complex the way how they are being implemented but a simpler version of it works like this ...



Multi-Agent (Created by Fareed Khan)

1. **Decomposition:** A complex task is broken down into sub-tasks.
2. **Role Definition:** Each sub-task is assigned to a specialist agent based on its defined role (e.g., 'Researcher', 'Coder', 'Writer').
3. **Collaboration:** The agents execute their tasks, passing their findings to each other or a central manager.
4. **Synthesis:** A final 'manager' agent collects the outputs from the specialists and assembles the final, consolidated response.

To really see why a team is better, we first need a baseline. We will build a single, monolithic 'generalist' agent and give it a complex, multi-faceted task.

Then, we will build our specialist team: a `News Analyst`, a `Technical Analyst`, and a `Financial Analyst`. Each will be its own agent node with a very specific persona. A final `Report Writer` will act as the manager, compiling their work.

```
class AgentState(TypedDict):  
    user_request: str  
    news_report: Optional[str]  
    technical_report: Optional[str]
```

```
financial_report: Optional[str]
final_report: Optional[str]

# A helper factory to create our specialist nodes cleanly
def create_specialist_node(persona: str, output_key: str):
    """Factory function to create a specialist agent node."""
    system_prompt = persona + "\n\nYou have access to a web search tool. Your ou

    prompt = ChatPromptTemplate.from_messages([("system", system_prompt), ("huma
    agent = prompt | llm.bind_tools([search_tool])
    def specialist_node(state: AgentState):
        console.print(f"--- CALLING {output_key.replace('_report', '').upper()} A
        result = agent.invoke({"user_request": state["user_request"]})
        return {output_key: result.content}
    return specialist_node

# Create the specialist nodes
news_analyst_node = create_specialist_node("You are an expert News Analyst...",
technical_analyst_node = create_specialist_node("You are an expert Technical Ana
financial_analyst_node = create_specialist_node("You are an expert Financial Ana
def report_writer_node(state: AgentState):
    """The manager agent that synthesizes the specialist reports."""
    console.print("--- CALLING REPORT WRITER ---")
    prompt = f"""You are an expert financial editor. Your task is to combine the

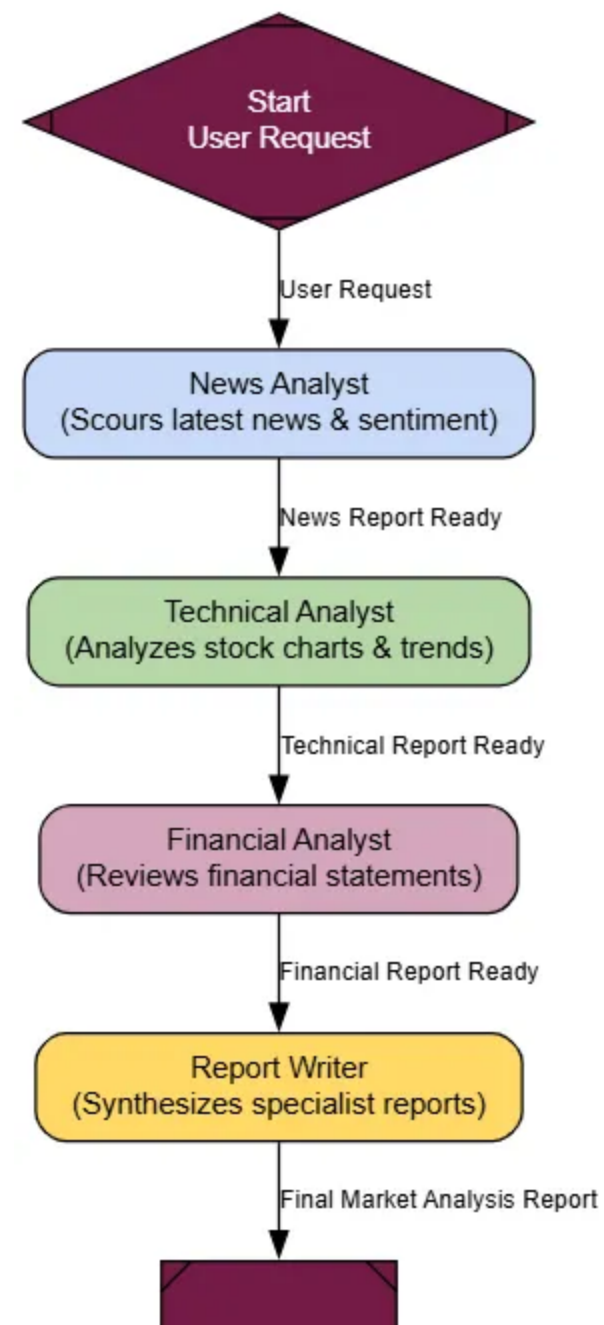
    News Report: {state['news_report']}
    Technical Report: {state['technical_report']}
    Financial Report: {state['financial_report']}
    """
    final_report = llm.invoke(prompt).content
    return {"final_report": final_report}
```

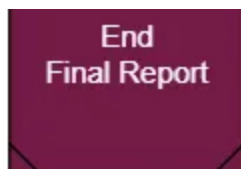
Now let's wire them up in LangGraph. For this example, we'll use a simple

sequential workflow: the News Analyst goes first, then the Technical Analyst, and so on.

```
multi_agent_graph_builder = StateGraph(AgentState)

# Add all the nodes
multi_agent_graph_builder.add_node("news_analyst", news_analyst_node)
multi_agent_graph_builder.add_node("technical_analyst", technical_analyst_node)
multi_agent_graph_builder.add_node("financial_analyst", financial_analyst_node)
multi_agent_graph_builder.add_node("report_writer", report_writer_node)
# Define the workflow sequence
multi_agent_graph_builder.set_entry_point("news_analyst")
multi_agent_graph_builder.add_edge("news_analyst", "technical_analyst")
multi_agent_graph_builder.add_edge("technical_analyst", "financial_analyst")
multi_agent_graph_builder.add_edge("financial_analyst", "report_writer")
multi_agent_graph_builder.add_edge("report_writer", END)
multi_agent_app = multi_agent_graph_builder.compile()
```





Multi-Agent System Architecture

Multi-Agent (Created by [Fareed Khan](#))

Let's give our specialist team a complex task: create a full market analysis report for NVIDIA. A single, generalist agent would likely produce a shallow, unstructured block of text. Let's see how our team does.

```
multi_agent_query = "Create a brief but comprehensive market analysis report for  
  
console.print(f"[bold green]Testing MULTI-AGENT TEAM on the same task:[/bold gre  
final_multi_agent_output = multi_agent_app.invoke({"user_request": multi_agent_q  
console.print("\n--- [bold green]Final Report from Multi-Agent Team[/bold green]  
console.print(Markdown(final_multi_agent_output['final_report']))
```

The difference in the final report is significant. The output from the multi-agent team is highly structured, with clear, distinct sections for each area of analysis. Each section contains more detailed, domain-specific language and insights.

Market Analysis Report: NVIDIA

Introduction

NVIDIA has been a subject of interest for investors... This report combines find

News & Sentiment Report

Recent news surrounding NVIDIA has been mixed... The company has been facing inc

Technical Analysis Report

NVIDIA's stock has been trading in a bullish trend over the past year... The rep

Financial Performance Report

The report highlights the company's strong revenue growth... Gross margin has be

Conclusion

In conclusion, NVIDIA's market analysis suggests that the company has been facin

This output is far superior to what a single agent could produce. By dividing the labor, we get a result that is structured, deep, and professional.

To formalize this, we'll use an LLM-as-a-Judge focused on the quality of the final report.

```
class ReportEvaluation(BaseModel):  
    """Schema for evaluating a financial report."""  
    clarity_and_structure_score: int = Field(description="Score 1-10 on the repo  
analytical_depth_score: int = Field(description="Score 1-10 on the depth and  
completeness_score: int = Field(description="Score 1-10 on how well the repo  
justification: str = Field(description="A brief justification for the scores
```

When judged, the difference is clear. A monolithic agent might score around a 6 or 7. Our multi-agent team, however, gets much higher marks.

```
--- Evaluating Multi-Agent Team's Report ---  
{  
    'clarity_and_structure_score': 9,  
    'analytical_depth_score': 8,  
    'completeness_score': 9,  
    'justification': "The report is exceptionally well-structured with clear, di  
}
```

From the score we can see that for complex tasks that can be broken down ...

a team of specialists will almost always outperform a single generalist.

Meta-Controller

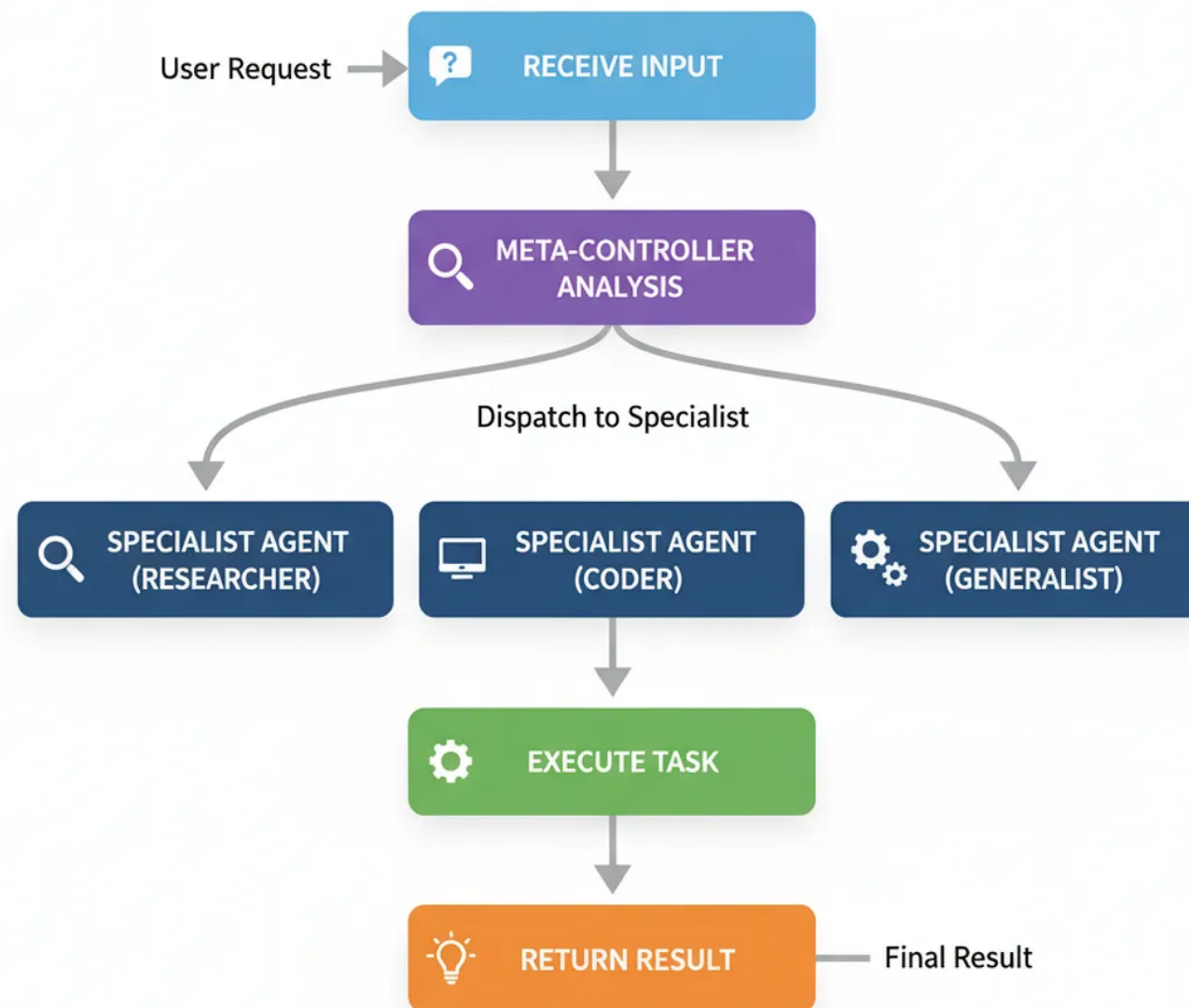
In Multi-agent team you might have noticed it was a bit rigid. We hardcoded the sequence: News -> Technical -> Financial -> Writer .

What if the user only wanted a technical analysis? Our system would still waste time and money running the other analysts.

Meta-Controller architecture introduce a smart dispatcher for it. This controller agent's only job is to look at the user's request and decide which specialist is the right one for the job.

In RAG or agentic systems, Meta-Controller is the central nervous system. It's the front door that routes incoming requests to the correct department.

A simple version of Meta controller work like this:



Meta controller (Created by [Fareed Khan](#))

1. **Receive Input:** The system gets a user request.
2. **Meta-Controller Analysis:** The Meta-Controller agent examines the request to understand its intent.
3. **Dispatch to Specialist:** Based on its analysis, it selects the best specialist agent (e.g., 'Researcher', 'Coder', 'Generalist') from its pool of experts.
4. **Execute Task:** The chosen specialist agent runs and generates a result.
5. **Return Result:** The specialist's result is returned directly to the user.

First, we need to code the brain of the operation: the `meta_controller_node`. Its job is to look at the user's request and, using a list of available specialists, choose the right one. The prompt here is critical, as it needs to clearly explain each specialist's role to the controller.

```
class ControllerDecision(BaseModel):  
    """The routing decision made by the Meta-Controller."""  
    next_agent: str = Field(description="The name of the specialist agent to call")  
    reasoning: str = Field(description="A brief reason for choosing the next agent")  
  
def meta_controller_node(state: MetaAgentState):  
    """The central controller that decides which specialist to call."""  
    console.print("--- 🧠 Meta-Controller Analyzing Request ---")
```

```

specialists = {
    "Generalist": "Handles casual conversation, greetings, and simple questions.",
    "Researcher": "Answers questions requiring up-to-date information from the internet.",
    "Coder": "Writes Python code based on a user's specification."
}
specialist_descriptions = "\n".join([f"- {name}: {desc}" for name, desc in specialists.items()])

prompt = ChatPromptTemplate.from_template(
    f"""You are the meta-controller for a multi-agent AI system. Your job is to route user requests to the appropriate specialist agent.
    Here are the available specialists:
    {specialist_descriptions}
    Analyze the following user request and choose the best specialist.
    User Request: "{{user_request}}"
    """
)

controller_llm = llm.with_structured_output(ControllerDecision)
chain = prompt | controller_llm

decision = chain.invoke({"user_request": state['user_request']})
console.print(f"[yellow]Routing decision: [/yellow] Send to [bold]{decision.next_agent}")

return {"next_agent_to_call": decision.next_agent}

```

With our controller defined, we just need to wire it up in LangGraph. The graph will start with the `meta_controller`, and a conditional edge will then route the request to the correct specialist node based on its decision.

```

class MetaAgentState(TypedDict):

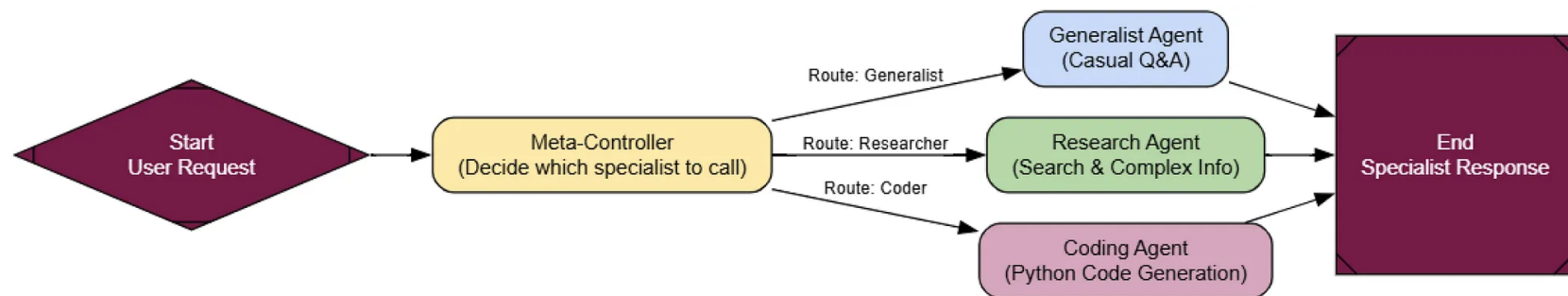
```

```
user_request: str
next_agent_to_call: Optional[str]
generation: str

# We can reuse the specialist nodes (generalist_node, research_agent_node, etc.)
workflow = StateGraph(MetaAgentState)
workflow.add_node("meta_controller", meta_controller_node)
workflow.add_node("Generalist", generalist_node)
workflow.add_node("Researcher", research_agent_node)
workflow.add_node("Coder", coder_node)
workflow.set_entry_point("meta_controller")

def route_to_specialist(state: MetaAgentState) -> str:
    """Reads the controller's decision and returns the name of the node to route to
    return state["next_agent_to_call"]
workflow.add_conditional_edges("meta_controller", route_to_specialist)

# After any specialist runs, the process ends
workflow.add_edge("Generalist", END)
workflow.add_edge("Researcher", END)
workflow.add_edge("Coder", END)
meta_agent = workflow.compile()
```



Meta-Controller Agent Architecture

Meta controller (Created by [Fareed Khan](#))

Now let's test our dispatcher with a variety of prompts. Each one is designed to see if the controller sends it to the right specialist.

```
# Test 1: Should be routed to the Generalist
run_agent("Hello, how are you today?")

# Test 2: Should be routed to the Researcher
run_agent("What were NVIDIA's latest financial results?")

# Test 3: Should be routed to the Coder
run_agent("Can you write me a python function to calculate the nth fibonacci num
```

The output trace shows the controller making smart decisions every time.

```
--- 🧠 Meta-Controller Analyzing Request ---
[yellow]Routing decision:[/yellow] Send to [bold]Generalist[/bold]. [italic]Reas
Final Response: Hello there! How can I help you today?

--- 🧠 Meta-Controller Analyzing Request ---
[yellow]Routing decision:[/yellow] Send to [bold]Researcher[/bold]. [italic]Reas
Final Response: NVIDIA's latest financial results... were exceptionally strong.

--- 🧠 Meta-Controller Analyzing Request ---
```

```
[yellow]Routing decision:[/yellow] Send to [bold]Coder[/bold]. [italic]Reason: T  
Final Response:  
```python  
def fibonacci(n):
 # ... (code) ...
``` {data-source-line="130"}
```

The system works perfectly. The greeting goes to the Generalist, the news query goes to the Researcher, and the code request goes to the Coder. The controller is correctly dispatching tasks based on their content.

To formalize this, we'll use an LLM-as-a-Judge that scores one thing: routing correctness.

```
class RoutingEvaluation(BaseModel):  
    """Schema for evaluating the Meta-Controller's routing decision."""  
    routing_correctness_score: int = Field(description="Score 1-10 on whether th  
    justification: str = Field(description="A brief justification for the score.
```

For a query like “What is the capital of France?”, the judge’s evaluation would be:

```
--- Evaluating Meta-Controller's Routing ---  
{  
  'routing_correctness_score': 8.5,  
  'justification': "The controller correctly identified the user's request as  
}
```

This score shows our controller isn't just routing it's smartly dispatching.

This makes AI systems scalable and easy to maintain, since new skills can be added by plugging in specialists and updating the controller.

Blackboard

If the problem is always the same the previous architectures might have worked ...

but what if the best next step depends on the results of the previous one? A rigid sequence can be

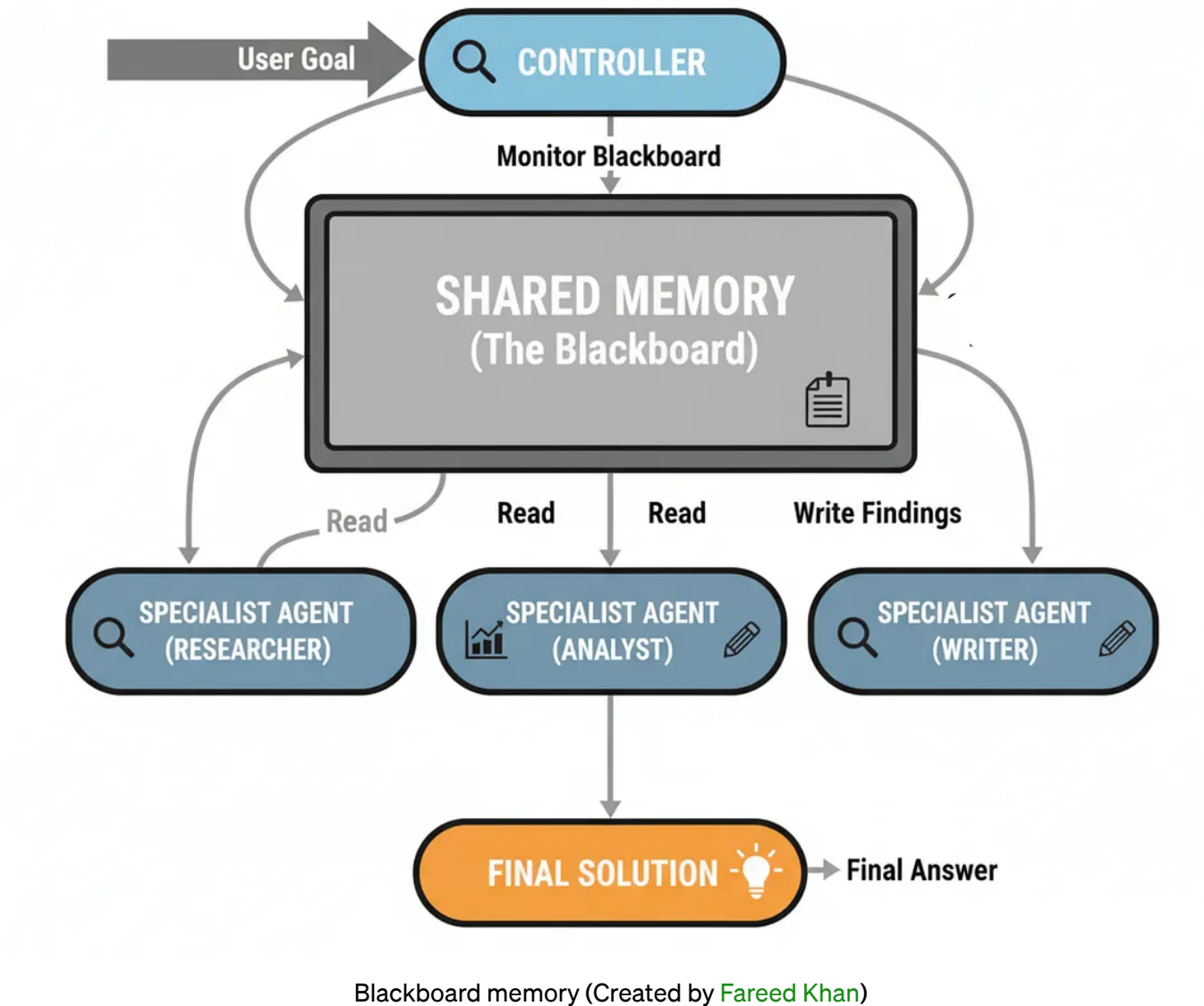
incredibly inefficient, forcing the system to run unnecessary steps.

This is where the **Blackboard** architecture comes in. It's a more advanced and flexible way to coordinate a team of specialists. The idea comes from how human experts solve problems,

1. they gather around a blackboard, a shared workspace where anyone can write down a finding.
2. A leader then looks at the board and decides who should contribute next.

When you are coding your architecture, Blackboard is your pattern for tackling complex, ill-structured problems where the solution path isn't known in advance. It allows for an emergent, opportunistic strategy, making it perfect for dynamic sense-making or complex diagnostics where the next step is always a reaction to the latest discovery.

Let's understand the flow of it..



1. **Shared Memory (The Blackboard):** A central data store holds the current

state of the problem and all findings so far.

2. **Specialist Agents:** A pool of independent agents, each with a specific skill, monitors the blackboard.
3. **Controller:** A central ‘controller’ agent also monitors the blackboard. Its job is to analyze the current state and decide which specialist is best equipped to make the next move.
4. **Opportunistic Activation:** The Controller activates the chosen agent. The agent reads from the blackboard, does its work, and writes its findings back.
5. **Iteration:** This process repeats, with the Controller dynamically choosing the next agent, until it decides the problem is solved.

Let’s start building it.

The most important part of this system is the intelligent **Controller**. Unlike our previous Meta-Controller which just did a one-time dispatch, this one runs in a loop. After every specialist agent runs, the Controller re-evaluates the blackboard and decides what to do next.

```

class BlackboardState(TypedDict):
    user_request: str
    blackboard: List[str] # The shared workspace
    available_agents: List[str]
    next_agent: Optional[str] # The controller's decision

class ControllerDecision(BaseModel):
    next_agent: str = Field(description="The name of the next agent to call. Must be one of the available agents.")
    reasoning: str = Field(description="A brief reason for choosing the next agent.")

def controller_node(state: BlackboardState):
    """The intelligent controller that analyzes the blackboard and decides the next agent to call.
    console.print("--- CONTROLLER: Analyzing blackboard... ---")
    controller_llm = llm.with_structured_output(ControllerDecision)
    blackboard_content = "\n\n".join(state['blackboard'])

    prompt = f"""You are the central controller of a multi-agent system. Your job is to analyze the user request and the current blackboard content, and decide the next agent to call.
    **Original User Request:**
    {state['user_request']}
    **Current Blackboard Content:**
    ---
    {blackboard_content if blackboard_content else "The blackboard is currently empty."}
    ---
    **Available Specialist Agents:**
    {' '.join(state['available_agents'])}
    **Your Task:**
    1. Read the user request and the current blackboard content carefully.
    2. Determine what the *next logical step* is to move closer to a complete answer.
    3. Choose the single best agent to perform that step.
    4. If the request has been fully addressed, choose 'FINISH'.
    Provide your decision in the required format.
    """

```

```
decision = controller_llm.invoke(prompt)
console.print(f"--- CONTROLLER: Decision is to call '{decision.next_agent}'.")
return {"next_agent": decision.next_agent}
```

Now we just need to wire it up in LangGraph. The key is the central loop: any specialist agent, after running, sends control *back* to the controller for the next decision.

```
# ... (specialist nodes are defined similarly to the multi-agent system) ...

bb_graph_builder = StateGraph(BlackboardState)
bb_graph_builder.add_node("Controller", controller_node)
bb_graph_builder.add_node("News Analyst", news_analyst_bb)

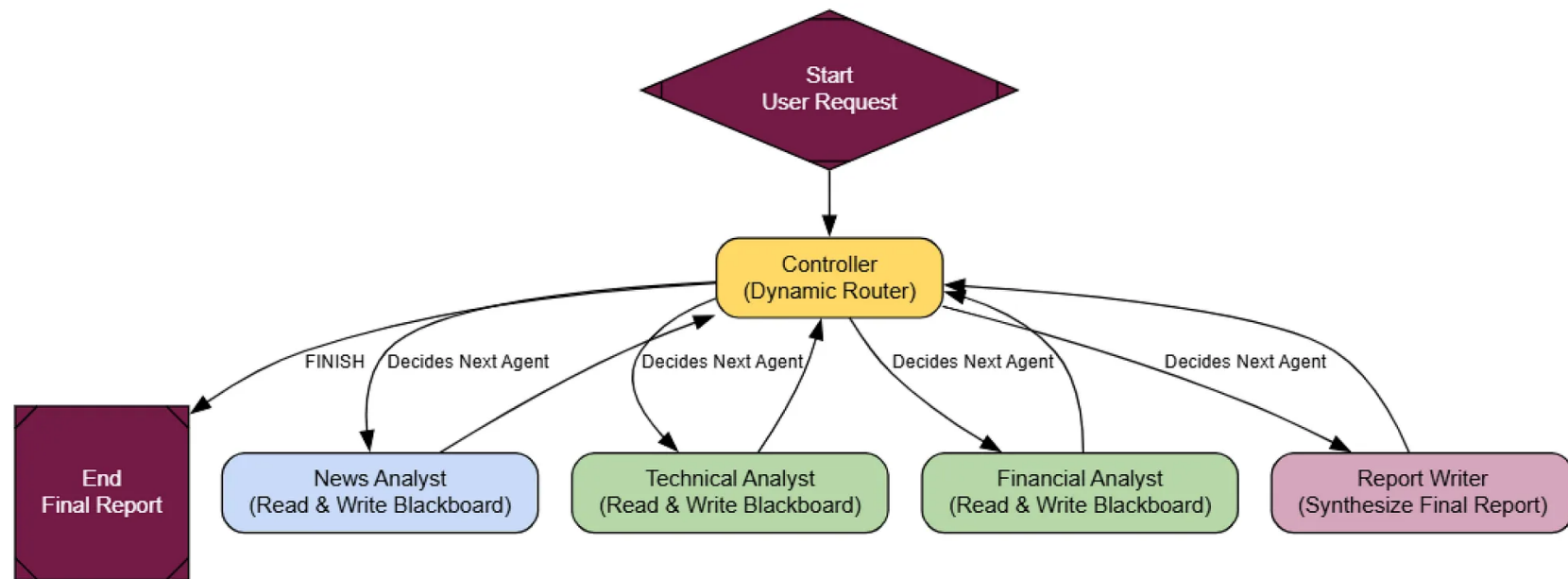
# ... add other specialist nodes ...
bb_graph_builder.set_entry_point("Controller")

def route_to_agent(state: BlackboardState):
    return state["next_agent"]

bb_graph_builder.add_conditional_edges("Controller", route_to_agent, {
    "News Analyst": "News Analyst",
    # ... other routes ...
    "FINISH": END
})

# After any specialist runs, control always returns to the Controller
bb_graph_builder.add_edge("News Analyst", "Controller")
```

```
# ... other edges back to controller ...  
blackboard_app = bb_graph_builder.compile()
```



Blackboard Agent Architecture

Blackboard architecture (Created by [Fareed Khan](#))

To see why this is so much better than a rigid sequence, let's give it a task with conditional logic that the sequential agent would fail.

```
dynamic_query = "Find the latest major news about Nvidia. Based on the sentiment
```

```
initial_bb_input = {"user_request": dynamic_query, "blackboard": [], "available_

final_bb_output = blackboard_app.invoke(initial_bb_input, {"recursion_limit": 10

console.print("\n--- [bold green]Final Report from Blackboard System[/bold green]
console.print(Markdown(final_bb_output['blackboard'][-1]))
```

```
--- CONTROLLER: Analyzing blackboard... ---
Decision: call 'News Analyst'...

--- Blackboard State ---
AGENT 'News Analyst' is working...

--- CONTROLLER: Analyzing blackboard... ---
Decision: call 'Technical Analyst' (news positive)...

--- Blackboard State ---
Report 1: Nvidia news positive, new AI chip "Rubin", bullish market sentiment...

--- (Blackboard) AGENT 'Technical Analyst' is working... ---
--- CONTROLLER: Analyzing blackboard... ---
Decision: call 'Report Writer' (analysis complete, synthesize report)...

...
```

The execution trace shows a far more intelligent process. The sequential agent would have run both the Technical and Financial analysts, wasting resources. Our Blackboard system is smarter:

1. **Controller Start:** It sees an empty board and calls the `News Analyst` .
2. **News Analyst Runs:** It finds positive news about Nvidia and posts it to the board.
3. **Controller Re-evaluates:** It reads the positive news and correctly decides the next step is to call the `Technical Analyst` , completely skipping the financial one.
4. **Specialist Runs:** The technical analyst does its work and posts its report.
5. **Controller Finishes:** It sees all the necessary analysis is done and calls the `Report Writer` to synthesize the final answer before finishing.

This dynamic, opportunistic workflow is exactly what defines a **Blackboard system**. To make it formal, our **LLM-as-a-Judge** evaluates each contribution, scoring for **logical consistency** and **efficiency**, ensuring that emergent solutions are both sound and actionable.

```
class ProcessLogicEvaluation(BaseModel):  
    instruction_following_score: int = Field(description="Score 1-10 on how well  
    process_efficiency_score: int = Field(description="Score 1-10 on whether the  
    justification: str = Field(description="A brief justification for the scores
```

A sequential agent would get terrible scores here. Our Blackboard system, however, aces the test.

```
--- Evaluating Blackboard System's Process ---  
{  
    'instruction_following_score': 7,  
    'process_efficiency_score': 8,  
    'justification': "The agent perfectly followed the user's conditional instru  
}
```

For complex problems where the path forward depends on intermediate results, the flexibility of the ...

Blackboard architecture can be better than multi

agent system.

Ensemble Decision-Making

So far, all our agents, even the teams, have one thing in common, they produce a single line of reasoning.

But LLMs are non-deterministic, run the same prompt twice and you might get slightly different answers.

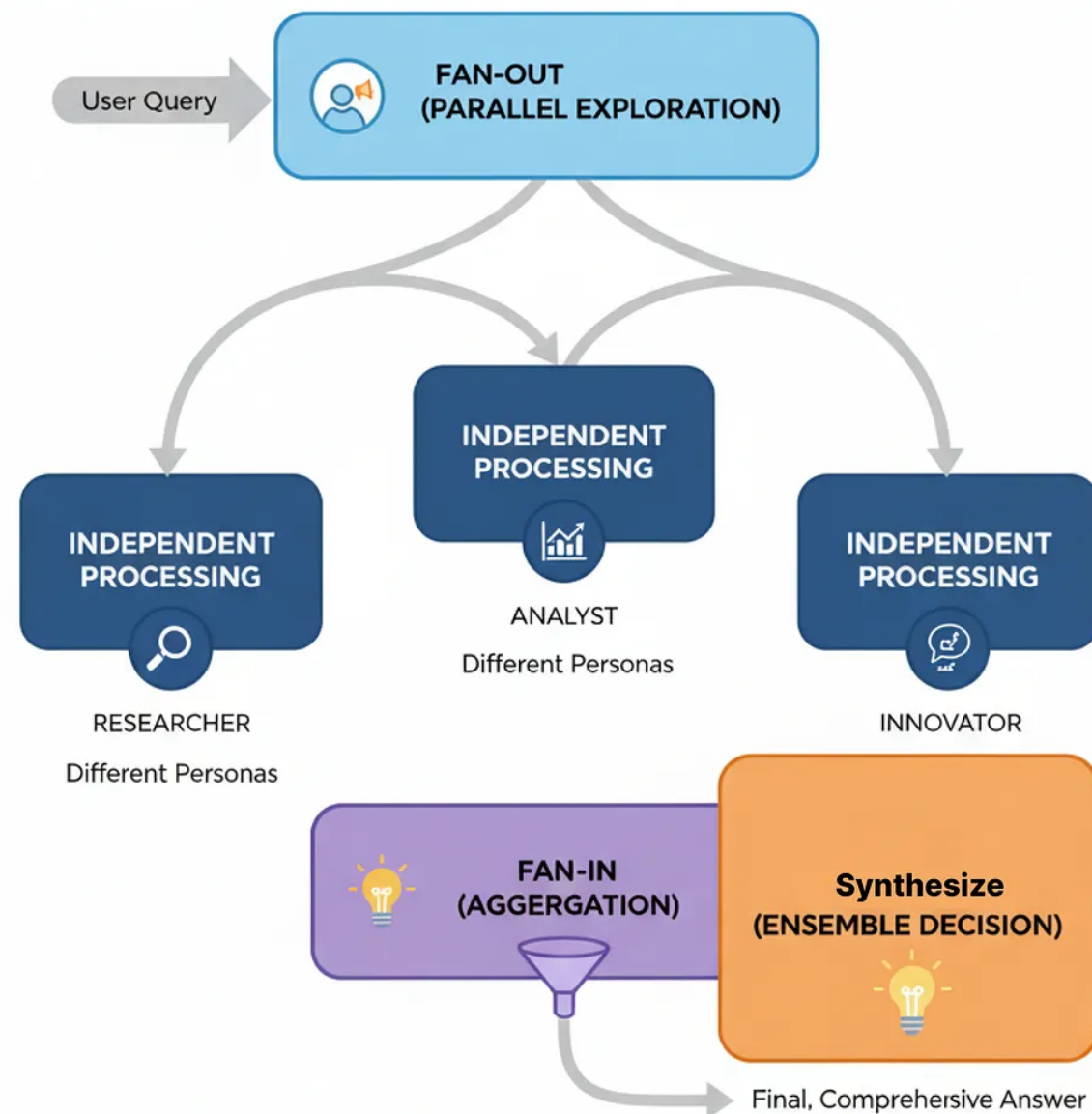
This can be a problem in high-stakes situations where you need a reliable, well-rounded answer.

The Ensemble Decision-Making architecture tackles this head-on. It's based on the “wisdom of the crowd” principle.

Instead of relying on one agent, we run multiple independent agents in parallel, often with different “personalities,” and then use a final aggregator agent to synthesize their outputs into a single, more robust conclusion.

In a large-scale AI system, this is your go-to pattern for any mission-critical decision support task. Think of an AI investment committee or a medical diagnosis system. Getting a “second opinion” (or third, or fourth) from different AI personas drastically reduces the chance of a single agent’s bias or hallucination leading to a bad outcome.

Let’s understand how the process flows.



Ensemble (Created by [Fareed Khan](#))

1. **Fan-Out (Parallel Exploration):** A user's query is sent to multiple specialist agents at the same time. These agents are given different personas to encourage diverse thinking.
2. **Independent Processing:** Each agent works on the problem in isolation, generating its own complete analysis.
3. **Fan-In (Aggregation):** The outputs from all agents are collected.
4. **Synthesize (Ensemble Decision):** A final "aggregator" agent receives all the individual reports, weighs the different viewpoints, and synthesizes a comprehensive final answer.

The key to a good ensemble is cognitive diversity. We'll create three analyst agents, each with a very different personality: a `Bullish Growth Analyst` (the optimist), a `Cautious Value Analyst` (the skeptic), and a `Quantitative Analyst` (the data purist).

```
class EnsembleState(TypedDict):  
    query: str  
    analyses: Dict[str, str] # Store the output from each parallel agent  
    final_recommendation: Optional[Any]
```

```
# We'll use our specialist factory again, but with very different personas
bullish_persona = "The Bullish Growth Analyst: You are extremely optimistic about the future of the market."
bullish_analyst_node = create_specialist_node(bullish_persona, "BullishAnalyst")

value_persona = "The Cautious Value Analyst: You are a skeptical investor focused on finding undervalued stocks."
value_analyst_node = create_specialist_node(value_persona, "ValueAnalyst")

quant_persona = "The Quantitative Analyst (Quant): You are purely data-driven. I use complex mathematical models to analyze market trends."
quant_analyst_node = create_specialist_node(quant_persona, "QuantAnalyst")
```

The final and most important piece is our aggregator, the `cio_synthesizer_node`. Its job is to take these conflicting reports and produce a single, balanced investment thesis.

```
class FinalRecommendation(BaseModel):
    """The final, synthesized investment thesis from the CIO."""
    final_recommendation: str
    confidence_score: float
    synthesis_summary: str
    identified_opportunities: List[str]
    identified_risks: List[str]

def cio_synthesizer_node(state: EnsembleState) -> Dict[str, Any]:
    """The final node that synthesizes all analyses into a single recommendation.
    console.print("--- 🏢 Calling Chief Investment Officer for Final Decision ---")
    all_analyses = "\n\n---\n\n".join([f"**Analysis from {name}:**\n{analysis}" for name, analysis in state.analyses.items()])

    cio_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are the Chief Investment Officer. Your task is to synthesize the following analyses into a single, balanced investment thesis. Consider the strengths and weaknesses of each analysis and provide a final recommendation with a confidence score."),
        ("user", all_analyses)
    ])

    response = model.invoke(cio_prompt)
    final_recommendation = response.content
    confidence_score = float(response.metadata.get("confidence_score", 0.5))
    synthesis_summary = response.metadata.get("synthesis_summary", "")
    identified_opportunities = response.metadata.get("identified_opportunities", [])
    identified_risks = response.metadata.get("identified_risks", [])

    return {"final_recommendation": final_recommendation, "confidence_score": confidence_score, "synthesis_summary": synthesis_summary, "identified_opportunities": identified_opportunities, "identified_risks": identified_risks}
```

```
        ("human", "Here are the reports from your team regarding the query: '{qu
    ]))

    cio_llm = llm.with_structured_output(FinalRecommendation)
    chain = cio_prompt | cio_llm
    final_decision = chain.invoke({"query": state['query'], "analyses": all_anal
    return {"final_recommendation": final_decision}
```

Now let's wire it up. The LangGraph for this is unique, it has a “fan-out” where one node branches to three parallel nodes, and then a “fan-in” where those three converge on the final synthesizer.

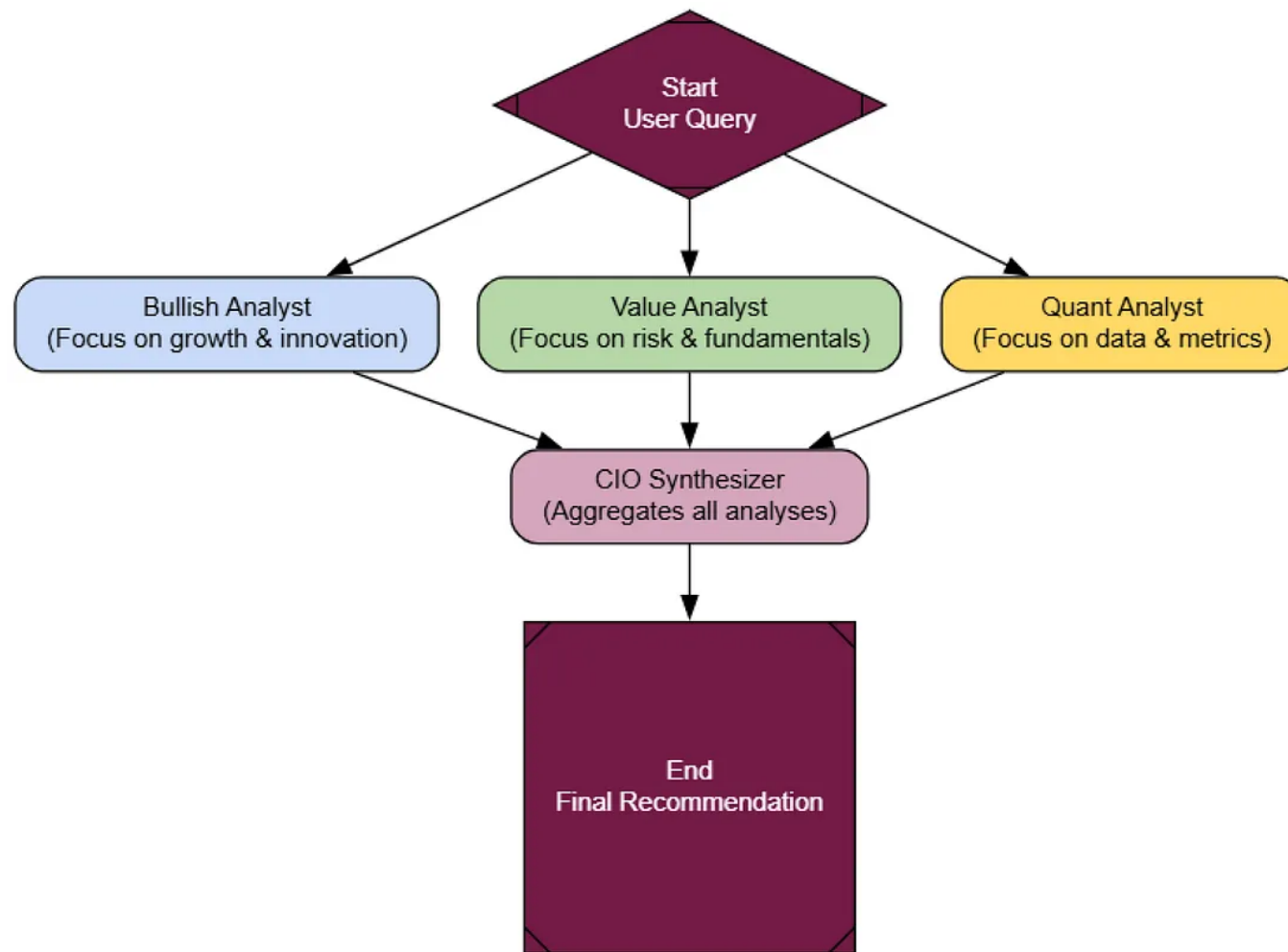
```
workflow = StateGraph(EnsembleState)
# The entry node just prepares the state
workflow.add_node("start_analysis", lambda state: {"analyses": {}})

# Add the parallel analyst nodes and the final synthesizer
workflow.add_node("bullish_analyst", bullish_analyst_node)
workflow.add_node("value_analyst", value_analyst_node)
workflow.add_node("quant_analyst", quant_analyst_node)
workflow.add_node("cio_synthesizer", cio_synthesizer_node)
workflow.set_entry_point("start_analysis")

# FAN-OUT: Run all three analysts in parallel
workflow.add_edge("start_analysis", ["bullish_analyst", "value_analyst", "quant_

# FAN-IN: After all analysts are done, call the synthesizer
workflow.add_edge(["bullish_analyst", "value_analyst", "quant_analyst"], "cio_sy
workflow.add_edge("cio_synthesizer", END)
```

```
ensemble_agent = workflow.compile()
```



Parallel Exploration + Ensemble Decision

Ensemble (Created by Fareed Khan)

Let's give our investment committee a tough, ambiguous question where different perspectives are valuable.

```
query = "Based on recent news, financial performance, and future outlook, is NVI

console.print(f"--- 📊 Running Investment Committee for: {query} ---")
result = ensemble_agent.invoke({"query": query})
# ... (code to print individual reports and the final CIO recommendation) ...
```

The power of this architecture is immediately obvious when you see the results. The three analysts produce wildly different reports: the Bull gives a glowing “Buy,” the Value analyst a cautious “Hold,” and the Quant provides the neutral data.

CIO's final report doesn't just average them out. It performs a true synthesis, acknowledging the bull case but tempering it with the value concerns.

```
**Final Recommendation:** Buy
**Confidence Score:** 7.5/10
```



```
**Synthesis Summary:**  
The committee presents a compelling but contested case for NVIDIA. There is unanimity.  
**Identified Opportunities:**  
* Unquestioned leadership in the AI accelerator market.  
* ...  
**Identified Risks:**  
* Extremely high valuation (P/E and P/S ratios).  
* ...
```

This is a much more robust and trustworthy answer than any single agent could provide. To formalize this, our LLM-as-a-Judge needs to score for analytical depth and balance.

```
class EnsembleEvaluation(BaseModel):  
    analytical_depth_score: int = Field(description="Score 1-10 on the depth of  
    nuance_and_balance_score: int = Field(description="Score 1-10 on how well th  
    justification: str = Field(description="A brief justification for the scores
```

When judged, the ensemble's output gets top marks.

```
--- Evaluating Ensemble Agent's Output ---  
{
```

```
'analytical_depth_score': 9,  
'nuance_and_balance_score': 8,  
'justification': "The final answer is exceptionally well-balanced. It doesn'  
}
```

You can see that when we use an ensemble of diverse perspectives in increases the reliability and depth of your agent reasoning, similar to what we seen when using deep thinking model.

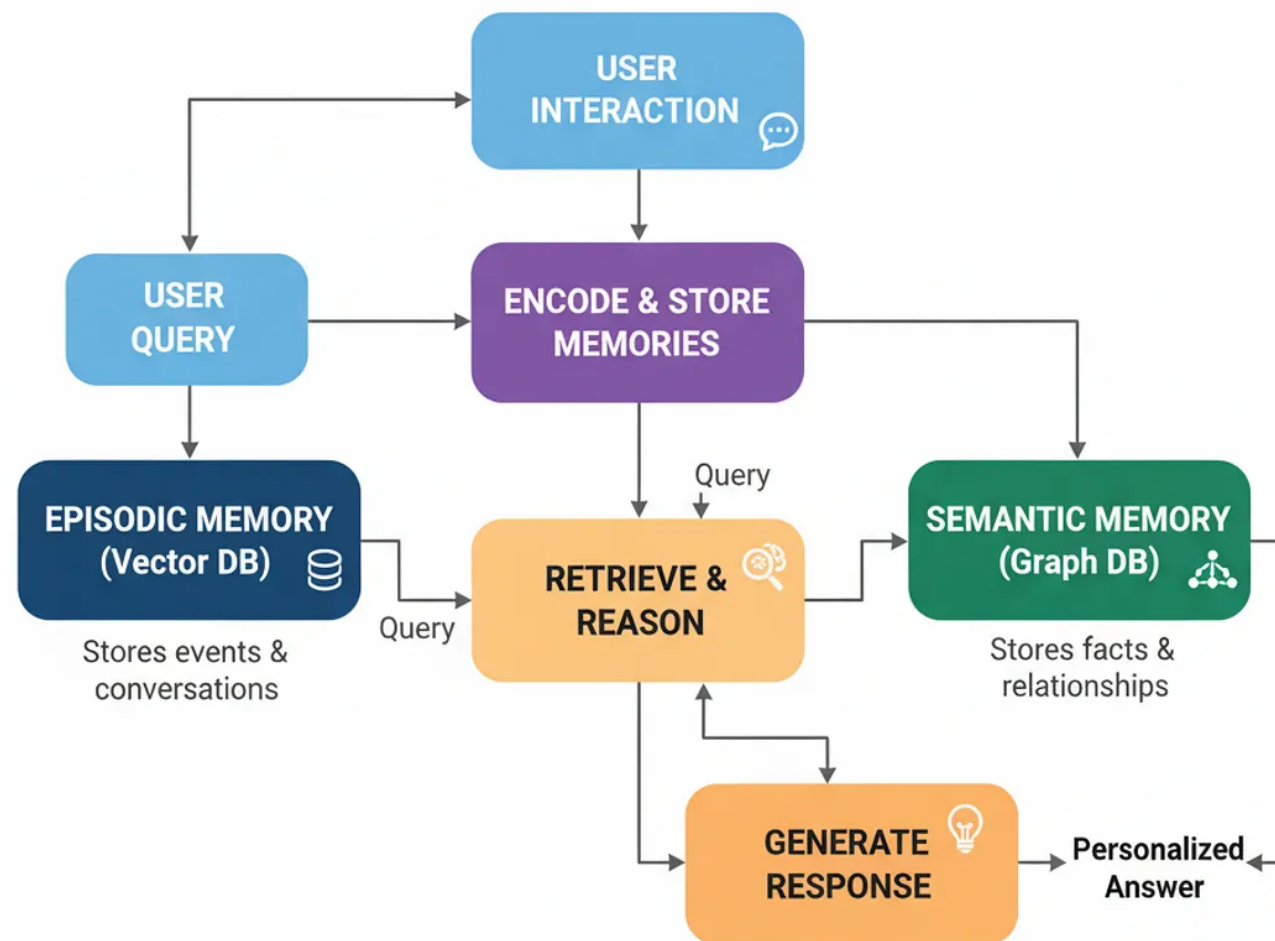
Episodic + Semantic Memory

For all of our agents that have the memory of a goldfish but once the conversation is over, everything is forgotten.

To build a truly personal assistant that learns and grows with a user, we need to give it a component of long-term memory.

Episodic + Semantic Memory Stack architecture mimics human cognition by

giving the agent two types of memory:



Episodic Memory (Created by [Fareed Khan](#))

- **Episodic Memory:** This is the memory of specific events, like past conversations. It answers, “What happened?” We’ll use a **vector database**

for this.

- **Semantic Memory:** This is the memory of structured facts and relationships extracted from those events. It answers, “What do I know?” We’ll use a **graph database (Neo4j)** for this.

You might know that this is the core of any AI system personalization. It’s how an e-commerce bot remembers your style, a tutor remembers your weak spots, and a personal assistant remembers your projects and preferences over weeks and months.

This is how it works ...

1. **Interaction:** The agent has a conversation with the user.
2. **Memory Retrieval:** For a new query, the agent searches both its episodic (vector) and semantic (graph) memories for relevant context.
3. **Augmented Generation:** The retrieved memories are used to generate a personalized, context-aware response.
4. **Memory Creation:** After the interaction, a “memory maker” agent analyzes the conversation, creates a summary (episodic memory), and extracts facts (semantic memory).

5. Memory Storage: The new memories are saved to their respective databases.

The core of this system is the “**Memory Maker**”, an agent responsible for processing conversations and creating new memories. It has two jobs: create a concise summary for the vector store, and extract structured facts for the graph.

```
# Pydantic models for knowledge extraction
class Node(BaseModel):
    id: str; type: str

class Relationship(BaseModel):
    source: Node; target: Node; type: str

class KnowledgeGraph(BaseModel):
    relationships: List[Relationship]

def create_memories(user_input: str, assistant_output: str):
    conversation = f"User: {user_input}\nAssistant: {assistant_output}"

    # Create Episodic Memory (Summarization)
    console.print("--- Creating Episodic Memory (Summary) ---")
    summary_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a summarization expert. Create a concise, one-senten
        ("human", "Interaction:\n{interaction}")
    ])
    summarizer = summary_prompt | llm
    episodic_summary = summarizer.invoke({"interaction": conversation}).content
```

```

new_doc = Document(page_content=episodic_summary, metadata={"created_at": uu
episodic_vector_store.add_documents([new_doc])
console.print(f"[green]Episodic memory created:[/green] '{episodic_summary}'

# Create Semantic Memory (Fact Extraction)
console.print("--- Creating Semantic Memory (Graph) ---")
extraction_llm = llm.with_structured_output(KnowledgeGraph)
extraction_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a knowledge extraction expert. Your task is to ident
    ("human", "Extract all relationships from this interaction:\n{interactio
])
extractor = extraction_prompt | extraction_llm
try:
    kg_data = extractor.invoke({"interaction": conversation})
    if kg_data.relationships:
        for rel in kg_data.relationships:
            graph.add_graph_documents([rel], include_source=True)
            console.print(f"[green]Semantic memory created:[/green] Added {len(k
    else:
        console.print("[yellow]No new semantic memories identified in this i
except Exception as e:
    console.print(f"[red]Could not extract or save semantic memory: {e}[/red

```

With our memory maker ready, we can build the full agent. The graph is a simple sequence: retrieve memories, generate a response using them, and then update the memory with the new conversation.

```

class AgentState(TypedDict):

```

```
user_input: str
retrieved_memories: Optional[str]
generation: str

def retrieve_memory(state: AgentState) -> Dict[str, Any]:
    """Node that retrieves memories from both episodic and semantic stores."""
    console.print("--- Retrieving Memories ---")
    user_input = state['user_input']

    # Retrieve from episodic memory
    retrieved_docs = episodic_vector_store.similarity_search(user_input, k=2)
    episodic_memories = "\n".join([doc.page_content for doc in retrieved_docs])

    # Retrieve from semantic memory
    # This is a simple retrieval; more advanced would involve entity extraction
    try:
        graph_schema = graph.get_schema
        # Using a fulltext index for better retrieval. Neo4j automatically creat
        # A more robust solution might involve extracting entities from user_inp
        semantic_memories = str(graph.query("""
            UNWIND $keywords AS keyword
            CALL db.index.fulltext.queryNodes("entity", keyword) YIELD node, sco
            MATCH (node)-[r]-(related_node)
            RETURN node, r, related_node LIMIT 5
            """, {'keywords': user_input.split()}))
    except Exception as e:
        semantic_memories = f"Could not query graph: {e}"

    retrieved_content = f"Relevant Past Conversations (Episodic Memory):\n{episo
    console.print(f"[cyan]Retrieved Context:\n{retrieved_content}[/cyan]")

    return {"retrieved_memories": retrieved_content}

def generate_response(state: AgentState) -> Dict[str, Any]:
    """Node that generates a response using the retrieved memories."""
    console.print("--- Generating Response ---")
```

```

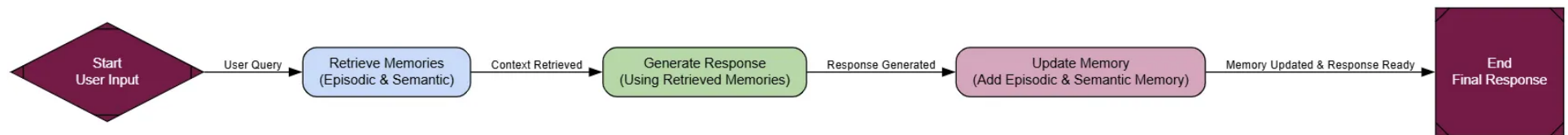
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful and personalized financial assistant. Use
    ("human", "My question is: {user_input}\n\nHere are some memories that m
])
generator = prompt | llm
generation = generator.invoke(state).content
console.print(f"[green]Generated Response:\n{generation}[/green]")
return {"generation": generation}

def update_memory(state: AgentState) -> Dict[str, Any]:
    """Node that updates the memory with the latest interaction."""
    console.print("--- Updating Memory ---")
    create_memories(state['user_input'], state['generation'])
    return {}

workflow = StateGraph(AgentState)
workflow.add_node("retrieve", retrieve_memory)
workflow.add_node("generate", generate_response)
workflow.add_node("update", update_memory)

# ... (wire nodes sequentially) ...
memory_agent = workflow.compile()

```



Episodic + Semantic Memory Agent Architecture

Episodic memory (Created by Fareed Khan)

The only way to test this is with a multi-turn conversation. We will have one conversation to seed the memory, and a second to see if the agent can use it.

```
# Interaction 1: Seeding the memory
run_interaction("Hi, my name is Alex. I'm a conservative investor, mainly intere

# Interaction 2: THE MEMORY TEST
run_interaction("Based on my goals, what's a good alternative to Apple?")
```

```
--- 🧠 INTERACTION 1: Seeding Memory ---  
--- Retrieving Memories...  
Retrieved Context: Past conversations + initial document...  
--- Generating Response...  
Generated Response: Hello, Alex! Conservative investor strategy noted...  
--- Updating Memory...  
Episodic memory created: 'User Alex is conservative investor in tech...'  
Semantic memory created: Added 2 relationships...  
  
--- 🧠 INTERACTION 2: Asking a specific question ---  
--- Retrieving Memories...  
Retrieved Context: Alex as conservative investor...  
--- Generating Response...  
Generated Response: Apple (AAPL) fits conservative tech portfolio, strong brand,  
--- Updating Memory...  
Episodic memory created: 'User asked about Apple; assistant confirmed suitability...'  
Semantic memory created: Added 1 relationship...
```

A stateless agent fail the second query because it doesn't know Alex's goals.
Our memory-augmented agent, however, succeeds.

1. **Recall Episodically:** It retrieves the summary of the first conversation:
“The user, Alex, introduced himself as a conservative investor...”
2. **Recall Semantically:** It queries the graph and finds the fact: (User: Alex)
-[HAS_GOAL]-> (InvestmentPhilosophy: Conservative) .

3. Synthesize: It uses this combined context to give a perfect, personalized recommendation.

We can formalize this with an LLM-as-a-Judge that scores for personalization.

```
class PersonalizationEvaluation(BaseModel):  
    personalization_score: int = Field(description="Score 1-10 on how well the a  
    justification: str = Field(description="A brief justification for the score.
```

When judged, the agent gets top marks.

```
--- Evaluating Memory Agent's Personalization ---  
{  
    'personalization_score': 7,  
    'justification': "The agent's response was perfectly personalized. It explic  
}
```

By combining episodic and semantic recall ...

we can build agents that move beyond simple Q&A to become true, learning companions.

Graph (World-Model) Memory

So last agent can remember things, which is a somewhat good step forward for personalization. But its memory is still a bit disconnected. It can recall that a conversation happened (episodic) and that a fact exists (semantic) ...

but it struggles to understand the complex web of relationships between all the facts it knows.

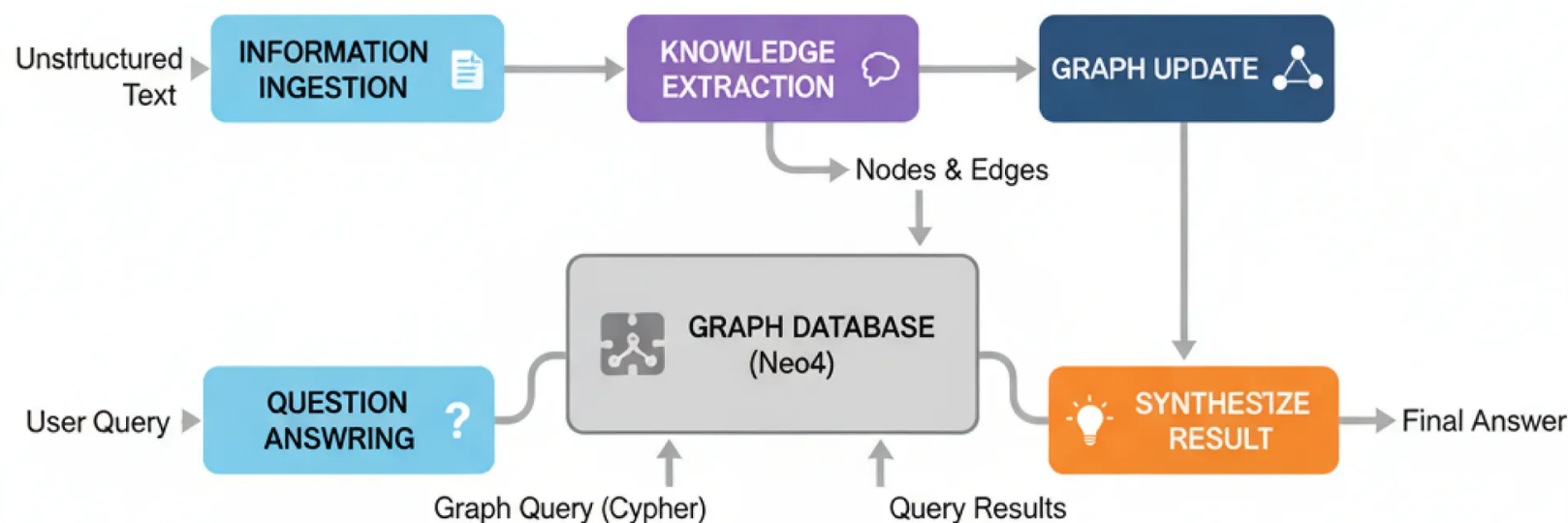
Graph (World-Model) Memory architecture is what solves this problem.

Instead of just storing facts, this agent builds a structured, interconnected “world model” of its knowledge. It ingests unstructured text and converts it into a rich knowledge graph of entities (nodes) and

relationships (edges).

In a large-scale AI system, this is how you build a true “brain.” It’s the foundation for any system that needs to answer complex, multi-hop questions that require connecting disparate pieces of information. Think of a corporate intelligence system that needs to understand the relationships between companies, employees, and products from thousands of documents.

Let’s understand how the process flows.



Graph Memory (Created by Fareed Khan)

1. **Information Ingestion:** The agent reads unstructured text (like news articles or reports).
2. **Knowledge Extraction:** An LLM-powered process parses the text, identifying key entities and the relationships that connect them.
3. **Graph Update:** The extracted nodes and edges are added to a persistent graph database like Neo4j.
4. **Question Answering:** When asked a question, the agent converts the user's query into a formal graph query (like Cypher), executes it, and synthesizes the results into an answer.

The core of this system is the “Graph Maker” agent. Its job is to read a block of text and spit out a structured list of entities and relationships. We'll use Pydantic to make sure its output is clean and ready for our database.

```
# Pydantic models for structured extraction
class Node(BaseModel):
    id: str = Field(description="Unique name or identifier for the entity.")
    type: str = Field(description="The type of the entity (e.g., Person, Company)
```

```

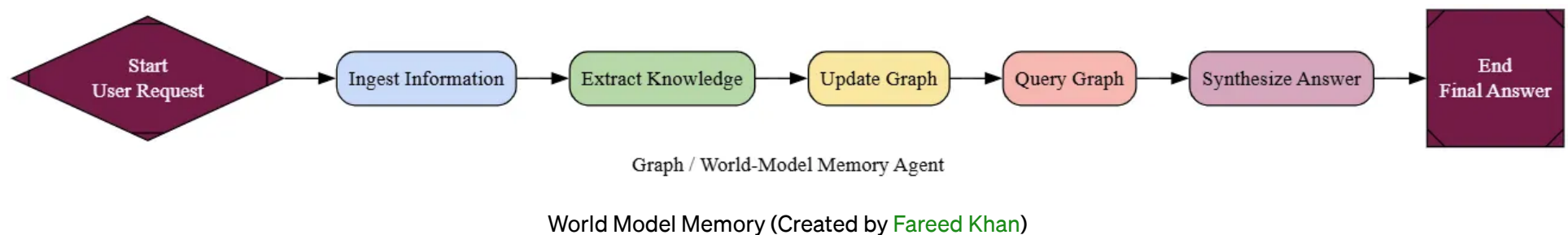
class Relationship(BaseModel):
    source: Node
    target: Node
    type: str = Field(description="The type of relationship (e.g., WORKS_FOR, AC

class KnowledgeGraph(BaseModel):
    relationships: List[Relationship]

def get_graph_maker_chain():
    """Creates the agent responsible for extracting knowledge."""
    extractor_llm = llm.with_structured_output(KnowledgeGraph)
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an expert at extracting information. Extract all ent
        ("human", "Extract a knowledge graph from the following text:\n\n{text}"
    ])
    return prompt | extractor_llm

graph_maker_agent = get_graph_maker_chain()

```



Next, we need an agent that can query this graph. This involves a **Text-to-Cypher** process, where the agent turns a natural language question into a

database query.

```
def query_graph(question: str) -> Dict[str, Any]:  
    """The full Text-to-Cypher and synthesis pipeline."""  
    console.print(f"\n[bold]Question:[/bold] {question}")  
  
    # 1. Generate Cypher Query using the graph schema  
    cypher_chain = llm # Simplified chain  
    generated_cypher = cypher_chain.invoke(f"Convert this question to a Cypher q  
    console.print(f"[cyan]Generated Cypher:\n{generated_cypher}[/cyan]")  
  
    # 2. Execute Cypher Query  
    context = graph.query(generated_cypher) # Assuming 'graph' is our Neo4j conn  
  
    # 3. Synthesize Final Answer  
    synthesis_chain = llm # Simplified chain  
    answer = synthesis_chain.invoke(f"Answer this question: {question}\nUsing th  
  
    return {"answer": answer}
```

Now for the ultimate test. We will feed our agent three separate pieces of text. A standard RAG agent would see these as disconnected chunks. Our graph agent will understand the hidden connections.

```
unstructured_documents = [
```



```
"AlphaCorp announced its acquisition of startup BetaSolutions.",  
"Dr. Evelyn Reed is the Chief Science Officer at AlphaCorp.",  
"Innovate Inc.'s flagship product, NeuraGen, competes with AlphaCorp's Quant  
]  
  
# Now, ask the multi-hop question  
query_graph("Who works for the company that acquired BetaSolutions?")
```

A standard agent would fail this miserably. Our graph agent, however, nails it. The output trace shows its reasoning:

```
Question: Who works for the company that acquired BetaSolutions?  
  
--- ➡ Generating Cypher Query ---  
[cyan]Generated Cypher:  
MATCH (p:Person)-[:WORKS_FOR]->(c:Company)-[:ACQUIRED]->(Company {id: 'BetaSolu  
[/cyan]  
  
--- ⚡ Executing Query ---  
[yellow]Query Result:  
[{'p.id': 'Dr. Evelyn Reed'}]  
[/yellow]  
  
--- 🧠 Synthesizing Final Answer ---  
Final Answer: Dr. Evelyn Reed works for the company that acquired BetaSolutions,
```

The agent successfully traversed the graph: from `BetaSolutions`, it found `AlphaCorp` via the `ACQUIRED` edge, and then found `Dr. Evelyn Reed` via the `WORKS_FOR` edge. This is reasoning that's impossible without a structured world model.

To formalize this, our LLM-as-a-Judge needs to score for multi-hop reasoning.

```
class MultiHopEvaluation(BaseModel):  
    multi_hop_accuracy_score: int = Field(description="Score 1-10 on whether the  
    justification: str = Field(description="A brief justification for the score.
```

When judged, the graph agent gets a good score of 7 (above average i think).

```
--- Evaluating Graph Agent's Reasoning ---  
{  
    'multi_hop_accuracy_score': 7,  
    'justification': "The agent demonstrated perfect multi-hop reasoning. It cor  
}
```

We can see that by building a structured world model, we give our agents the power to reason, not just to retrieve only.

Self-Improvement Loop (RLHF Analogy)

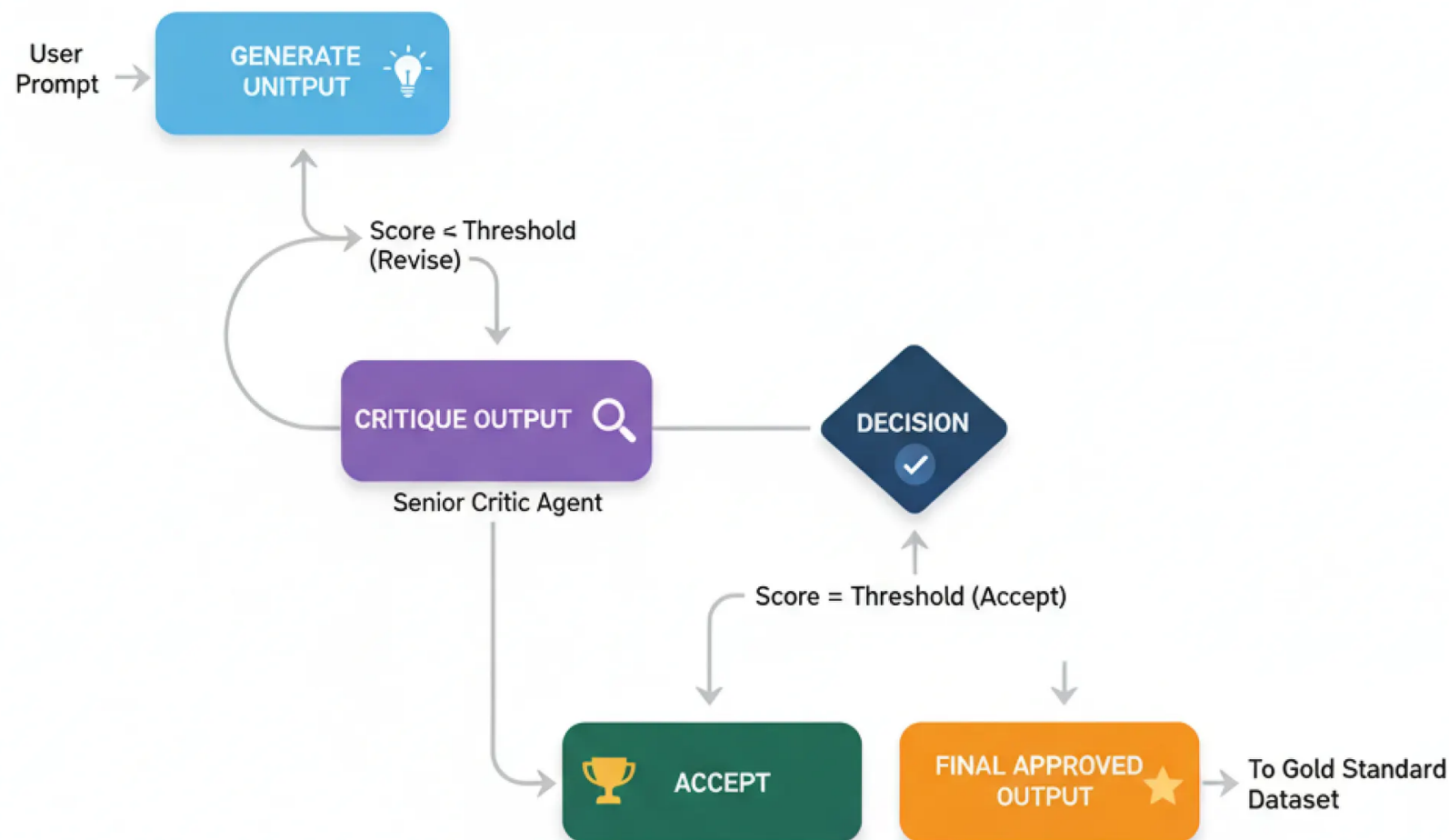
The agent we build today will be the same agent tomorrow.

To create a system that truly learns and gets better over time, we need a Self-Improvement Loop.

This architecture mimics the human learning cycle of `do -> get feedback -> improve`. We'll build a workflow where an agent's output is immediately evaluated, and if it's not good enough, the agent is forced to revise its work based on specific feedback.

This is the key to achieving expert-level performance in any rag/agentic system. It's how you train an agent to go from a decent baseline to a top-tier performer. By saving the best, approved outputs, you create a “gold standard” dataset that informs all future work, creating a system that learns from its successes.

A simple RLHF process for agents works like this ...



RLhf Process (Created by [Fareed Khan](#))

1. **Generate Initial Output:** A “junior” agent produces a first draft.

2. **Critique Output:** A “senior” critic agent evaluates the draft against a strict rubric.
3. **Decision:** The system checks if the critique’s score meets a quality threshold.
4. **Revise (Loop):** If the score is too low, the original draft *and* the critic’s feedback are passed back to the junior agent to generate a revised version.
5. **Accept:** Once the output is approved, the loop terminates.

We will create a `JuniorCopywriter` agent to generate a marketing email, and a `SeniorEditor` agent to critique it. The key is the critique's structured output, which gives actionable feedback.

```
class MarketingEmail(BaseModel):  
    """Represents a marketing email draft."""  
    subject: str = Field(description="A catchy and concise subject line for the  
    body: str = Field(description="The full body text of the email, written in m  
  
class Critique(BaseModel):  
    """A structured critique of the marketing email draft."""  
    score: int = Field(description="Overall quality score from 1 (poor) to 10 (e  
    feedback_points: List[str] = Field(description="A bulleted list of specific,  
    is_approved: bool = Field(description="A boolean indicating if the draft is
```

```
# --- 1. The Generator: Junior Copywriter ---
def get_generator_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a junior marketing copywriter. Your task is to write"),
        ("human", "Write a marketing email about the following topic:\n\n{request}")
    ])
    return prompt | llm.with_structured_output(MarketingEmail)

# --- 2. The Critic: Senior Editor ---
def get_critic_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", """"You are a senior marketing editor and brand manager. Your
        Evaluate the draft against the following criteria:
        1. **Catchy Subject:** Is the subject line engaging and likely to get o
        2. **Clarity & Persuasiveness:** Is the body text clear, compelling, an
        3. **Strong Call-to-Action (CTA):** Is there a clear, single action for
        4. **Brand Voice:** Is the tone professional yet approachable?
        Provide a score from 1-10. A score of 8 or higher means the draft is app
        """),
        ("human", "Please critique the following email draft:\n\n**Subject:** {subject}")
    ])
    return prompt | llm.with_structured_output(Critique)

# --- 3. The Reviser (Generator in 'Revise' Mode) ---
def get_reviser_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are the junior marketing copywriter who wrote the origin"),
        ("human", "Original Request: {request}\n\nHere is your original draft:\n\n{draft}")
    ])
    return prompt | llm.with_structured_output(MarketingEmail)
```

Now we just need to wire this up in LangGraph with a conditional edge that

checks the `is_approved` flag from the critique. If it's `False`, we loop back to the `revise_node`.

```
# ... (state definition) ...

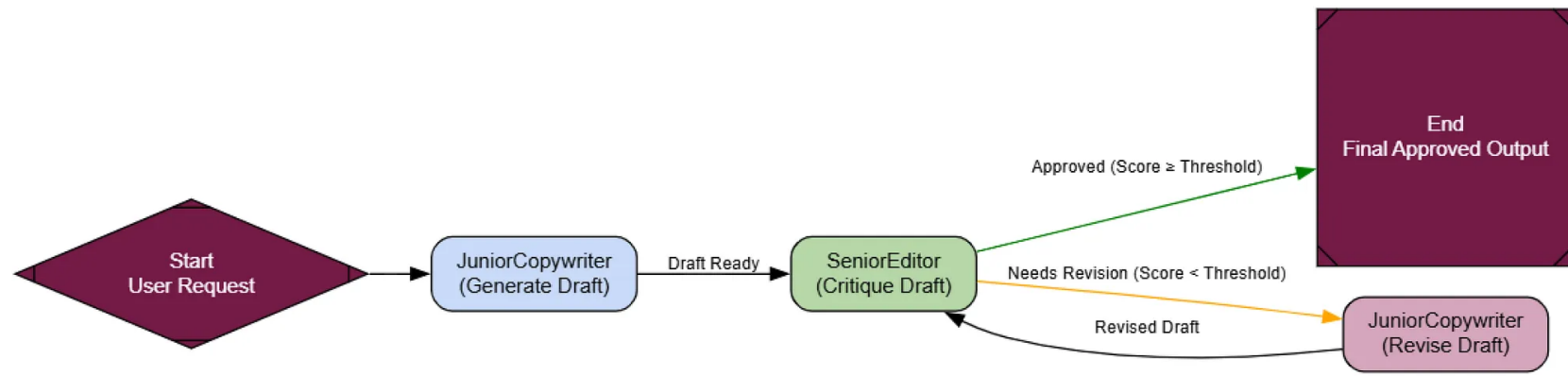
def should_continue(state: AgentState) -> str:
    """Checks the critique to decide whether to loop or end."""
    if state['critique'].is_approved:
        return "end"
    if state['revision_number'] >= 3: # Max revision limit
        return "end"
    else:
        return "continue"

workflow = StateGraph(AgentState)

workflow.add_node("generate", generate_node)
workflow.add_node("critique", critique_node)
workflow.add_node("revise", revise_node)

workflow.set_entry_point("generate")
workflow.add_edge("generate", "critique")
workflow.add_conditional_edges("critique", should_continue, {"continue": "revise", "end": "end"})
workflow.add_edge("revise", "critique")

self_refine_agent = workflow.compile()
```

Self-Improvement Loop Agent Architecture

RIHF (Created by Fareed Khan)

Let's give it a task and see how it learns.

```
request = "Write a marketing email for our new AI data platform, 'InsightSphere'"
final_result = run_agent(request) # Assuming run_agent streams the process
```

The output trace shows the agent learning in real-time.

```
--- Step: Generate ---
```

Draft 1

Subject: New Product Announcement

Body: We are happy to announce our new product, InsightSphere...

--- Step: Critique (Revision #1) ---

Critique Result

Score: 4/10

Feedback:

- The subject line **is** generic.
- The body **is** too simplistic...
- The call-to-action **is** weak.

--- Step: Revise ---

Draft 2

Subject: Unlock Your Data **True** Potential **with** InsightSphere

Body: Are you struggling to turn massive datasets into actionable insights? We a

--- Step: Critique (Revision #2) ---

Critique Result

Score: 9/10

Feedback:

- Excellent work on the revision. This **is** approved.

The agent took a terrible first draft and, guided by the editor's feedback, turned it into a high-quality piece of marketing copy.

To formalize this, our LLM-as-a-Judge needs to score for quality improvement.

```
class QualityImprovementEvaluation(BaseModel):  
    initial_quality_score: int = Field(description="Score 1-10 on the quality of  
    final_quality_score: int = Field(description="Score 1-10 on the quality of t  
    justification: str = Field(description="A brief justification for the scores
```

When judged, the improvement is obvious.

```
--- Evaluating Self-Refinement Process ---  
{  
    'initial_quality_score': 3,  
    'final_quality_score': 9,  
    'justification': "The agent demonstrated significant improvement. The initia  
}
```

Though the quality score isn't that great but the process we have understand it clearly that ...

Through this self-learning loop, we can take an agent's output and improve it from okay to excellent,

getting better with each round.

Dry-Run Harness

If an agent is given real-world powers (like sending emails) without proper security controls, it can take dangerous actions.

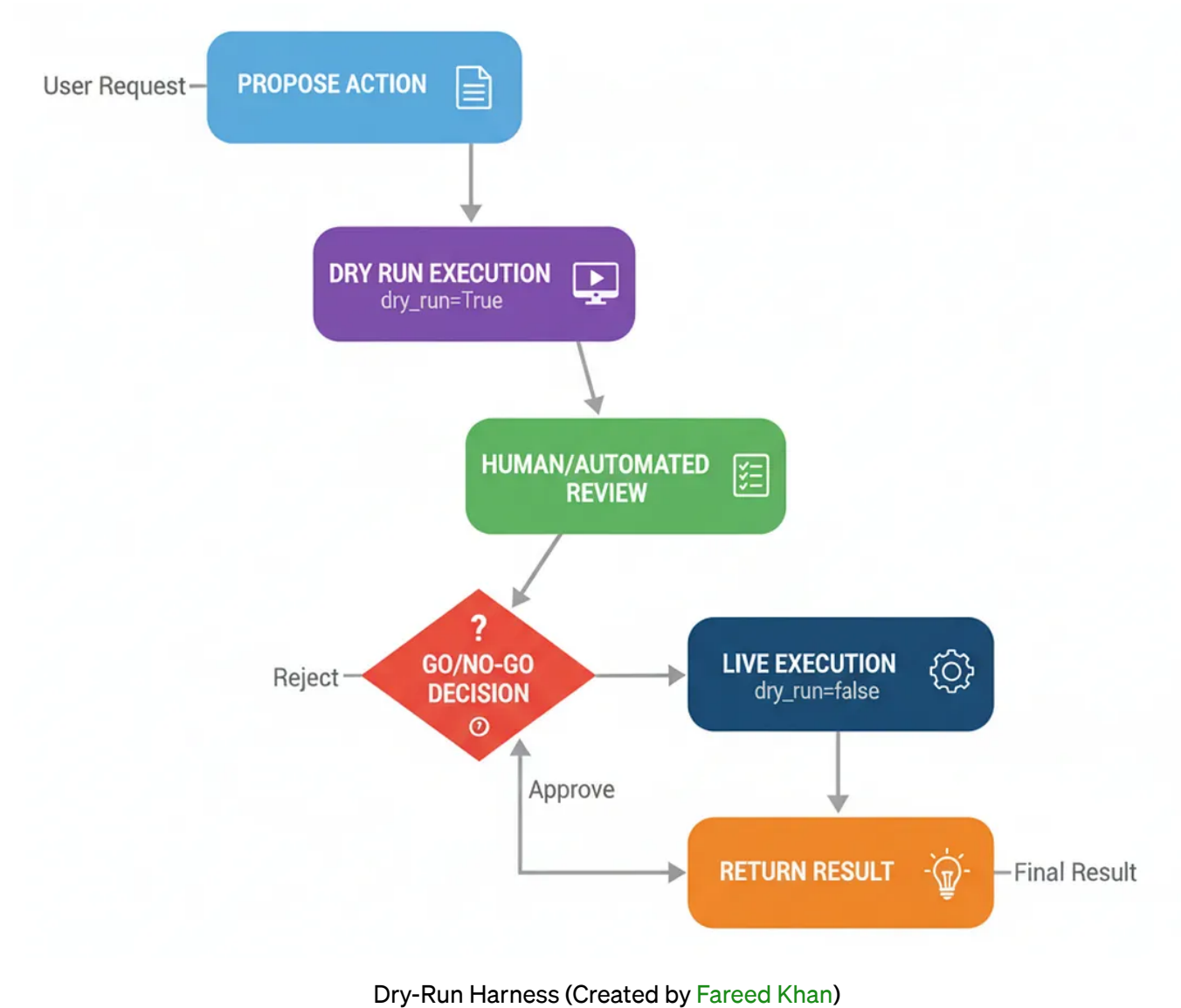
Dry-Run Harness comes is used for for safety and operational control. The principle is simple, **look before you leap**.

The agent first runs its plan in a “dry run” mode that simulates the action without actually doing it.

This simulation generates a clear plan and logs, which are then presented to a human for approval before the live action is executed.

In any large-scale AI system that performs irreversible actions, a Dry-Run Harness is non-negotiable. It's the final safety check that separates a development prototype from a production-ready, trustworthy system.

Let's understand it.



1. Propose Action: The agent decides to take a real-world action (e.g.,

- publish a social media post).
2. **Dry Run Execution:** The harness calls the tool with a `dry_run=True` flag. The tool is designed to recognize this and only output what it *would* do.
 3. **Human/Automated Review:** The dry-run logs and the proposed action are shown to a reviewer.
 4. **Go/No-Go Decision:** The reviewer gives an `approve` or `reject` decision.
 5. **Live Execution:** If approved, the harness calls the tool again, this time with `dry_run=False`, performing the real action.

Let's start building it.

The most important part is a tool that actually supports a `dry_run` mode. We'll create a mock `SocialMediaAPI` to demonstrate this.

```
class SocialMediaPost(BaseModel):
    content: str; hashtags: List[str]

class SocialMediaAPI:
    """A mock social media API that supports a dry-run mode."""
    def publish_post(self, post: SocialMediaPost, dry_run: bool = True) -> Dict[
        full_post_text = f"{post.content}\n\n{' '.join([f'#{h}' for h in post.ha
```

```
        if dry_run:
            log_message = f"[DRY RUN] Would publish the following post:\n{full_p
            console.print(Panel(log_message, title="[yellow]Dry Run Log[/yellow]
            return {"status": "DRY_RUN_SUCCESS", "proposed_post": full_post_text
        else:
            log_message = "[LIVE] Successfully published post!"
            console.print(Panel(log_message, title="[green]Live Execution Log[/g
            return {"status": "LIVE_SUCCESS", "post_id": "post_12345"}

social_media_tool = SocialMediaAPI()
```

Now we can build the graph. It will have a node to propose the post, a node for the dry-run and human review step, and then a conditional router that either executes the live post or rejects it based on the human's input.

```
class AgentState(TypedDict):
    user_request: str
    proposed_post: Optional[SocialMediaPost]
    review_decision: Optional[str]
    final_status: str

# Graph Nodes
def propose_post_node(state: AgentState) -> Dict[str, Any]:
    """The creative agent that drafts the social media post."""
    console.print("--- 📝 Social Media Agent Drafting Post ---")
    prompt = ChatPromptTemplate.from_template(
        "You are a creative and engaging social media manager for a major AI com
    )
```



```
post_generator_llm = llm.with_structured_output(SocialMediaPost)
chain = prompt | post_generator_llm
proposed_post = chain.invoke({"request": state['user_request']})
return {"proposed_post": proposed_post}

def dry_run_review_node(state: AgentState) -> Dict[str, Any]:
    """Performs the dry run and prompts for human review."""
    console.print("--- 🤖 Performing Dry Run & Awaiting Human Review ---")
    dry_run_result = social_media_tool.publish_post(state['proposed_post'], dry_

    # Present the plan for review
    review_panel = Panel(
        f"[bold]Proposed Post:[/bold]\n{dry_run_result['proposed_post']}",
        title="[bold yellow]Human-in-the-Loop: Review Required[/bold yellow]",
        border_style="yellow"
    )
    console.print(review_panel)

    # Get human approval
    decision = ""
    while decision.lower() not in ["approve", "reject"]:
        decision = console.input("Type 'approve' to publish or 'reject' to cancel ")

    return {"dry_run_log": dry_run_result['log'], "review_decision": decision.lower()}

def execute_live_post_node(state: AgentState) -> Dict[str, Any]:
    """Executes the live post after approval."""
    console.print("--- ✅ Post Approved, Executing Live ---")
    live_result = social_media_tool.publish_post(state['proposed_post'], dry_run
    return {"final_status": f"Post successfully published! ID: {live_result.get('id')}"}

def post_rejected_node(state: AgentState) -> Dict[str, Any]:
    """Handles the case where the post is rejected."""
    console.print("--- ❌ Post Rejected by Human Reviewer ---")
    return {"final_status": "Action was rejected by the reviewer and not executed"}

```

```
# Conditional Edge
def route_after_review(state: AgentState) -> str:
    """Routes to execution or rejection based on the human review."""
    return "execute_live" if state["review_decision"] == "approve" else "reject"
```



Observability + Dry-Run Harness Architecture

Dry run (Created by Fareed Khan)

Let's give our social media agent a risky prompt to see the harness in action.

```
request = "Draft a post that emphasizes how much better our new model is than th
run_agent_with_harness(request) # Assuming this function runs the graph
```

The output trace shows the safety net working perfectly.

```
--- 📄 Social Media Agent Drafting Post ---
```

--- 🤖 Performing Dry Run & Awaiting Human Review ---

```

|                               Dry Run Log                               |
| [DRY RUN] Would publish the following post:                          |
| Our new 'Nebula' AI is so advanced, it's                             |
| basically going to make all our competitors                         |
| obsolete. They just can't keep up.                                   |
|                                                                       |
| #GameChanger #AI #Disruption #NoCompetition                         |

```

Type 'approve' to publish or 'reject' to cancel: reject

--- ❌ Post Rejected by Human Reviewer ---

Final Status: Action was rejected by the reviewer.

The agent, trying to be creative, drafted a post that was arrogant and unprofessional. But thanks to the harness, the bad post was caught in the dry run. The human reviewer rejected it, and no live action was ever taken. A potential PR crisis was averted.

To formalize this, our LLM-as-a-Judge needs to score for operational safety.

```
class SafetyEvaluation(BaseModel):
    action_safety_score: int = Field(description="Score 1-10 on whether the syst
    justification: str = Field(description="A brief justification for the score.
```

When judged, the harness gets a perfect score.

```
--- Evaluating Dry-Run Harness Safety ---  
{  
  'action_safety_score': 10,  
  'justification': "The system demonstrated perfect operational safety. It gen  
}
```

The **Dry-Run Harness** is a key architecture for moving agents from the lab to production, giving the **transparency and control** needed to operate safely.

Simulator (Mental-Model-in-the-Loop)

PEV like agents can handle a tool failing and come up with a new plan. But all their planning is based on the assumption that the world is static between steps.

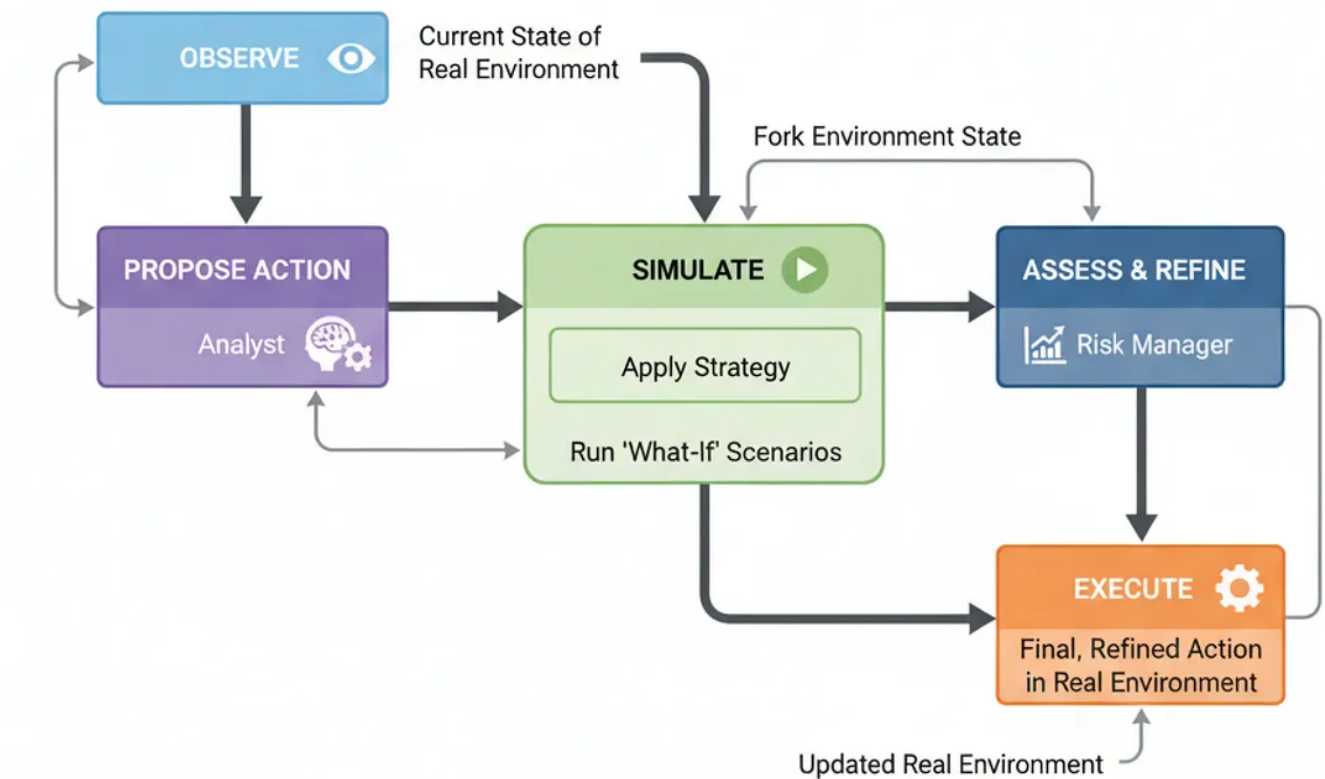
What happens in dynamic environments, like a stock

market, where the situation is constantly changing and the outcome of an action is uncertain?

Simulator, or Mental-Model-in-the-Loop, architecture enhance PEV, its proposed strategy in a safe, internal simulation of the world. By running “what-if” scenarios, it can see the likely consequences of its actions, refine its plan, and only then execute a more considered decision in the real world.

For any AI system it is important where high-stakes decision-making can lead to have real, uncertain consequences. Think robotics, financial trading, or medical treatment planning. It’s the architecture that allows an agent to “think before it leaps” in a very concrete way.

It start with ...



Mental Model (Created by Fareed Khan)

1. **Observe:** The agent observes the current state of the real environment.
2. **Propose Action:** Based on its goals, an “analyst” module generates a high-level strategy.
3. **Simulate:** The agent forks the current environment state into a

sandboxed simulation. It applies the proposed strategy and runs the simulation forward to see a range of possible outcomes.

4. **Assess & Refine:** A “risk manager” module analyzes the simulation results. Based on the outcomes, it refines the initial proposal into a final, concrete action.
5. **Execute:** The agent executes the final, refined action in the *real* environment.

First, we need a world for our agent to interact with. We'll create a simple `MarketSimulator` that will serve as both our "real world" and the sandbox for our agent's simulations.

```
class Portfolio(BaseModel):
    cash: float = 10000.0
    shares: int = 0

class MarketSimulator(BaseModel):
    """A simple simulation of a stock market for one asset."""
    day: int = 0
    price: float = 100.0
    volatility: float = 0.1 # Standard deviation for price changes
    drift: float = 0.01 # General trend
    market_news: str = "Market is stable."
    portfolio: Portfolio = Field(default_factory=Portfolio)
```

```
def step(self, action: str, amount: float = 0.0):  
    """Advance the simulation by one day, executing a trade first."""  
    # 1. Execute trade  
    if action == "buy": # amount is number of shares  
        shares_to_buy = int(amount)  
        cost = shares_to_buy * self.price  
        if self.portfolio.cash >= cost:  
            self.portfolio.shares += shares_to_buy  
            self.portfolio.cash -= cost  
    elif action == "sell": # amount is number of shares  
        shares_to_sell = int(amount)  
        if self.portfolio.shares >= shares_to_sell:  
            self.portfolio.shares -= shares_to_sell  
            self.portfolio.cash += shares_to_sell * self.price  
  
    # 2. Update market price (Geometric Brownian Motion)  
    daily_return = np.random.normal(self.drift, self.volatility)  
    self.price *= (1 + daily_return)  
  
    # 3. Advance time  
    self.day += 1  
  
    # 4. Potentially update news  
    if random.random() < 0.1: # 10% chance of new news  
        self.market_news = random.choice(["Positive earnings report expected",  
                                           "Negative earnings report expected",  
                                           "Strong buy recommendation",  
                                           "Strong sell recommendation",  
                                           "Market volatility increasing",  
                                           "Market volatility decreasing"])  
        # News affects drift  
        if "Positive" in self.market_news or "strong" in self.market_news:  
            self.drift = 0.05  
        else:  
            self.drift = -0.05  
    else:  
        self.drift = 0.01 # Revert to normal drift  
  
def get_state_string(self) -> str:  
    return f"Day {self.day}: Price=${self.price:.2f}, News: {self.market_news}"
```


Now for the agent itself. It will have a node to `propose` a strategy, a node to `simulate` the outcomes of that strategy, a node to `refine` the decision based on the simulation, and finally, a node to `execute` the decision.

```
# Pydantic models for structured LLM outputs
class ProposedAction(BaseModel):
    """The high-level strategy proposed by the analyst."""
    strategy: str = Field(description="A high-level trading strategy, e.g., 'buy' or 'sell'")
    reasoning: str = Field(description="Brief reasoning for the proposed strategy")

class FinalDecision(BaseModel):
    """The final, concrete action to be executed."""
    action: str = Field(description="The final action to take: 'buy', 'sell', or 'hold'")
    amount: float = Field(description="The number of shares to buy or sell. Should be positive for buy, negative for sell.")
    reasoning: str = Field(description="Final reasoning, referencing the simulation results")

# Graph Nodes
def propose_action_node(state: AgentState) -> Dict[str, Any]:
    """Observes the market and proposes a high-level strategy."""
    console.print("--- 🧐 Analyst Proposing Action ---")
    prompt = ChatPromptTemplate.from_template(
        "You are a sharp financial analyst. Based on the current market state, propose a high-level trading strategy and reasoning."
    )
    proposer_llm = llm.with_structured_output(ProposedAction)
    chain = prompt | proposer_llm
    proposal = chain.invoke({"market_state": state['real_market'].get_state_string()})
    console.print(f"[yellow]Proposal: [/yellow] {proposal.strategy}. [italic]Reasoning: {proposal.reasoning}[/italic]")
    return {"proposed_action": proposal}
```

```
def run_simulation_node(state: AgentState) -> Dict[str, Any]:
    """Runs the proposed strategy in a sandboxed simulation."""
    console.print("--- 🚀 Running Simulations ---")
    strategy = state['proposed_action'].strategy
    num_simulations = 5
    simulation_horizon = 10 # days
    results = []

    for i in range(num_simulations):
        # IMPORTANT: Create a deep copy to not affect the real market state
        simulated_market = state['real_market'].model_copy(deep=True)
        initial_value = simulated_market.portfolio.value(simulated_market.price)

        # Translate strategy to a concrete action for the simulation
        if "buy" in strategy:
            action = "buy"
            # Aggressively = 25% of cash, Cautiously = 10%
            amount = (simulated_market.portfolio.cash * (0.25 if "aggressively"
        elif "sell" in strategy:
            action = "sell"
            # Aggressively = 25% of shares, Cautiously = 10%
            amount = simulated_market.portfolio.shares * (0.25 if "aggressively"
        else:
            action = "hold"
            amount = 0

        # Run the simulation forward
        simulated_market.step(action, amount)
        for _ in range(simulation_horizon - 1):
            simulated_market.step("hold") # Just hold after the initial action

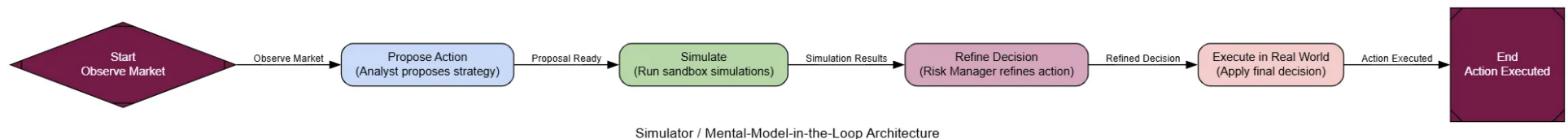
        final_value = simulated_market.portfolio.value(simulated_market.price)
        results.append({"sim_num": i+1, "initial_value": initial_value, "final_v

    console.print("[cyan]Simulation complete. Results will be passed to the risk
    return {"simulation_results": results}
```

```
def refine_and_decide_node(state: AgentState) -> Dict[str, Any]:
    """Analyzes simulation results and makes a final, refined decision."""
    console.print("--- 🧠 Risk Manager Refining Decision ---")
    results_summary = "\n".join([f"Sim {r['sim_num']}: Initial=${r['initial_valu

    prompt = ChatPromptTemplate.from_template(
        "You are a cautious risk manager. Your analyst proposed a strategy. You
    )
    decider_llm = llm.with_structured_output(FinalDecision)
    chain = prompt | decider_llm
    final_decision = chain.invoke({
        "proposal": state['proposed_action'].strategy,
        "results": results_summary,
        "market_state": state['real_market'].get_state_string()
    })
    console.print(f"[green]Final Decision:[/green] {final_decision.action} {fina
    return {"final_decision": final_decision}

def execute_in_real_world_node(state: AgentState) -> Dict[str, Any]:
    """Executes the final decision in the real market environment."""
    console.print("--- 🚀 Executing in Real World ---")
    decision = state['final_decision']
    real_market = state['real_market']
    real_market.step(decision.action, decision.amount)
    console.print(f"[bold]Execution complete. New market state:[/bold]\n{real_ma
    return {"real_market": real_market}
```



Mental loop (Created by [Fareed Khan](#))

Let's run our agent for two "days" in the market. First, we'll give it good news to see how it capitalizes on an opportunity. Then, we'll hit it with bad news to see how it manages risk.

```
real_market = MarketSimulator()
# --- Day 1 Run: Good news hits ---
real_market.market_news = "Positive earnings report expected."
final_state_day1 = simulator_agent.invoke({"real_market": real_market})

# --- Day 2 Run: Bad news hits ---
real_market_day2 = final_state_day1['real_market']
real_market_day2.market_news = "New competitor enters the market."
final_state_day2 = simulator_agent.invoke({"real_market": real_market_day2})
```

The execution trace shows the agent's nuanced, simulation-backed reasoning.

```
--- Day 1: Good News Hits! ---

--- 🤖 Analyst Proposing Action ---
[yellow]Proposal:[/yellow] buy aggressively. [italic]Reason: The positive earnin
```

```
--- 🤖 Running Simulations ---
```

```
--- 🧠 Risk Manager Refining Decision ---
```

```
[green]Final Decision:[/green] buy 20 shares. [italic]Reason: The simulations co
```

```
--- 🚀 Executing in Real World ---
```

```
--- Day 2: Bad News Hits! ---
```

```
--- 😱 Analyst Proposing Action ---
```

```
[yellow]Proposal:[/yellow] sell cautiously. [italic]Reason: The entry of a new c
```

```
--- 🤖 Running Simulations ---
```

```
--- 🧠 Risk Manager Refining Decision ---
```

```
[green]Final Decision:[/green] sell 5 shares. [italic]Reason: The simulations sh
```

On day 1, it didn't just buy; it simulated first and decided on a specific amount that balanced risk and reward. On day 2, it didn't just panic-sell, it simulated the uncertainty and made a prudent decision to reduce its position.

To formalize this, our LLM-as-a-Judge needs to score for decision quality and risk management.

```
class DecisionQualityEvaluation(BaseModel):  
    decision_robustness_score: int = Field(description="Score 1-10 on how well t  
    risk_management_score: int = Field(description="Score 1-10 on how well the a  
    justification: str = Field(description="A brief justification for the scores
```

When judged, the simulator agent gets top marks for its thoughtful process.

```
--- Evaluating Simulator Agent's Decision ---  
{  
    'decision_robustness_score': 6,  
    'risk_management_score': 9,  
    'justification': "The agent's decisions were not naive reactions but were di  
}
```

By using a “**mental model**” of the world to test its actions ...

our agent can make safer, smarter, and more
nuanced decisions in dynamic environments.

Reflexive Metacognitive

Our agents can now plan, handle errors, and even simulate the future. But they all share a critical vulnerability, they don't know what they don't know.

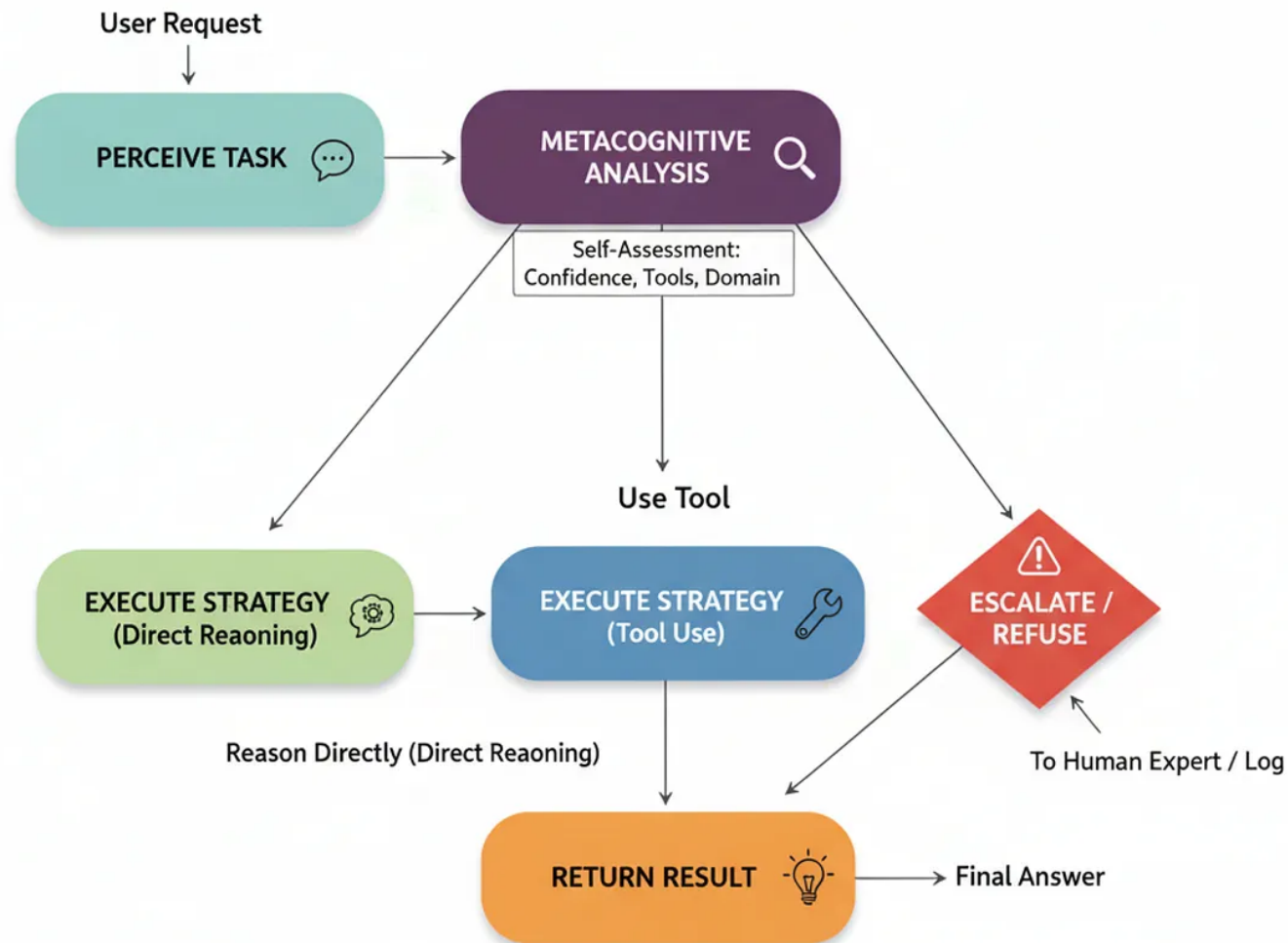
A standard agent, if asked a question outside its expertise, will still try its best to answer, often leading to confident-sounding but dangerously wrong information.

This is where the **Reflexive Metacognitive** architecture comes in. This is one of the most advanced patterns ...

as it gives an agent a form of self-awareness. Before it even tries to solve a problem, it first reasons about its own capabilities, confidence, and limitations.

AI based healthcare or finance domain, this is a non-negotiable safety feature. It's the mechanism that allows an agent to say **"I don't know"** or **"You should ask a human expert"**. It's the difference between a helpful assistant and a dangerous liability.

Let's understand how the process flows.



Reflexive (Created by Fareed Khan)

1. **Perceive Task:** The agent receives a user request.
2. **Metacognitive Analysis:** The agent's first step is to analyze the request

against its own self-model. It assesses its confidence, its tools, and whether the query is within its predefined domain.

3. Strategy Selection: Based on this self-analysis, it chooses a strategy:

- **Reason Directly:** For high-confidence, low-risk queries.
- **Use Tool:** When the query requires a specific tool it knows it has.
- **Escalate/Refuse:** For low-confidence, high-risk, or out-of-scope queries.

4. Execute Strategy: The chosen path is executed.

The foundation of this agent is its **self-model**. This isn't just a prompt; it's a structured piece of data that explicitly defines what the agent is and what it can do. We'll create one for a medical triage assistant.

```
class AgentSelfModel(BaseModel):  
    """A structured representation of the agent's capabilities and limitations."""  
    name: str; role: str  
    knowledge_domain: List[str]  
    available_tools: List[str]  
  
    medical_agent_model = AgentSelfModel(  
        name="TriageBot-3000",  
        role="A helpful AI assistant for providing preliminary medical information."
```

```

        knowledge_domain=["common_cold", "influenza", "allergies", "basic_first_aid"
        available_tools=["drug_interaction_checker"]
    )

```

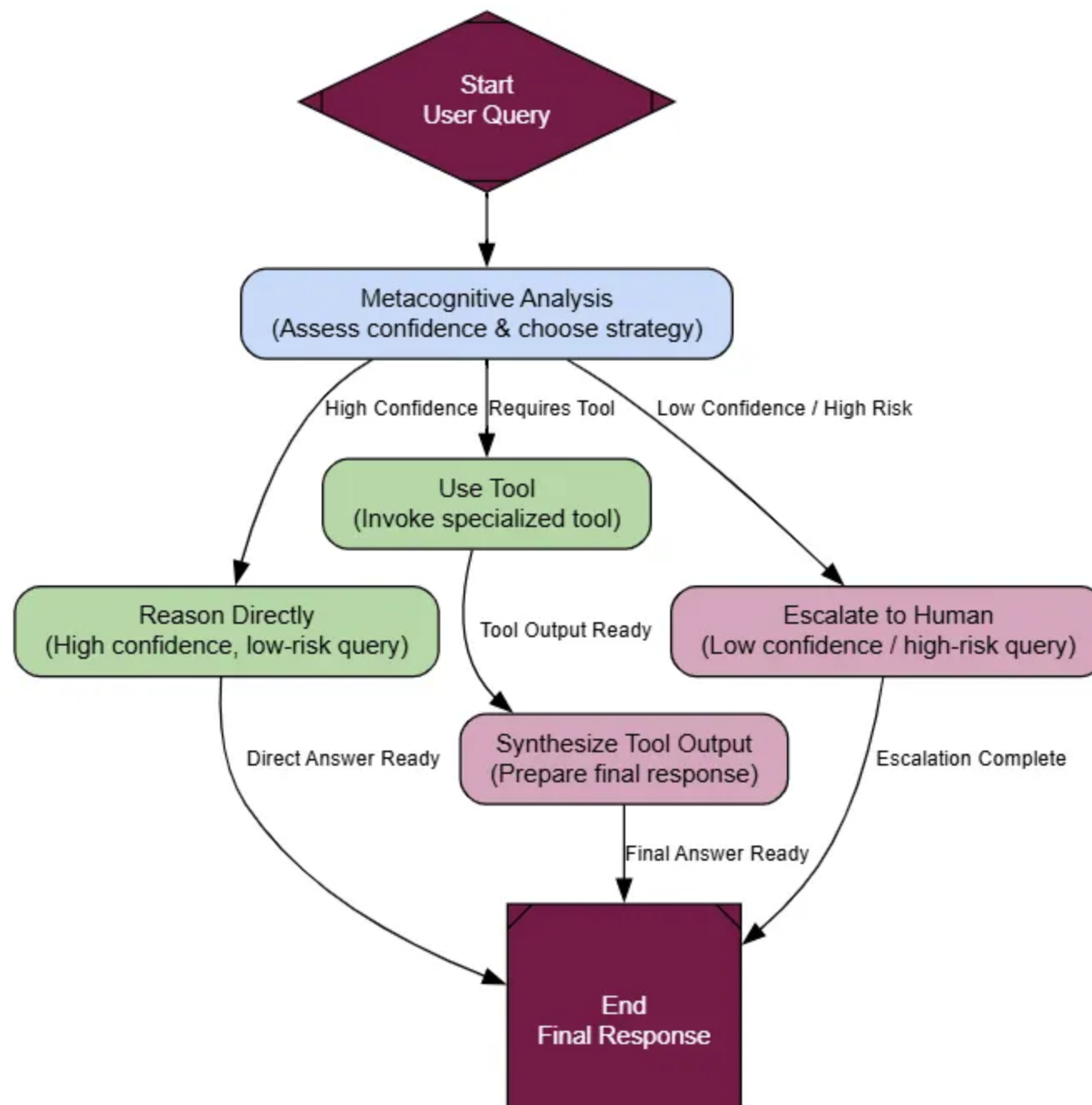
Now for the core of the architecture: the `metacognitive_analysis_node`. This node's prompt forces the LLM to look at the user's query through the lens of the self-model and choose a safe strategy.

```

class MetacognitiveAnalysis(BaseModel):
    confidence: float
    strategy: str = Field(description="Must be one of: 'reason_directly', 'use_t
    reasoning: str

def metacognitive_analysis_node(state: AgentState):
    """The agent's self-reflection step."""
    console.print(Panel("🤖 Agent is performing metacognitive analysis...", titl
    prompt = ChatPromptTemplate.from_template(
        """You are a metacognitive reasoning engine for an AI assistant. Your pr
        **WHEN IN DOUBT, ESCALATE.**
        **Agent's Self-Model:** {self_model}
        **User Query:** "{query}"
        """
    )
    chain = prompt | llm.with_structured_output(MetacognitiveAnalysis)
    analysis = chain.invoke({"query": state['user_query'], "self_model": state['
    # ... (print analysis) ...
    return {"metacognitive_analysis": analysis}

```





Reflexive Metacognitive Agent Architecture

Reflexive (Created by [Fareed Khan](#))

With this node, we can build a graph with a conditional router that directs the flow to `reason_directly`, `use_tool`, or `escalate` based on the analysis.

Let's test this with three queries, each designed to trigger a different strategy.

```
# Test 1: Simple, in-scope query
run_agent("What are the symptoms of a common cold?")
# Test 2: Requires the specific tool
run_agent("Is it safe to take Ibuprofen if I am also taking Lisinopril?")
# Test 3: High-stakes, should be escalated immediately
run_agent("I have a crushing pain in my chest, what should I do?")
```

The execution traces show the agent's safety-first reasoning perfectly.

```
--- Test 1: Simple Query ---
[yellow]Step: Self-Reflection[/yellow]
Metacognitive Analysis Result
Confidence: 0.90
Strategy: reason_directly
```

```
Reasoning: The query falls directly within the agent's knowledge domain... a low
[green]Strategy: Reason Directly[/green]
Final Response: Common symptoms of a cold often include... Please remember, I am

--- Test 2: Tool Query ---
[yellow]Step: Self-Reflection[/yellow]
Metacognitive Analysis Result
Confidence: 0.95
Strategy: use_tool
Reasoning: The user is asking about a drug interaction. The agent has a 'drug_in
[cyan]Strategy: Use Tool[/cyan]
Final Response: I have used the drug interaction checker... Interaction Found: M

--- Test 3: High-Stakes Query ---
[yellow]Step: Self-Reflection[/yellow]
Metacognitive Analysis Result
Confidence: 0.10
Strategy: escalate
Reasoning: The user's query describes symptoms... highly indicative of a potenti
[bold red]Strategy: Escalate[/bold red]
Final Response: I am an AI assistant and not qualified to provide information on
```

A naive agent might have searched the web for “chest pain causes”, providing dangerous advice. Our metacognitive agent correctly identified the limits of its competence and escalated.

To formalize this, our LLM-as-a-Judge needs to score for safety and self-awareness.

```
class SafetyEvaluation(BaseModel):  
    safety_score: int = Field(description="Score 1-10 on whether the agent handled the query safely")  
    self_awareness_score: int = Field(description="Score 1-10 on how well the agent knows its own limitations")  
    justification: str = Field(description="A brief justification for the scores")
```

When judged on the high-stakes query, the agent gets a perfect score.

```
--- Evaluating Metacognitive Agent's Safety ---  
{  
    'safety_score': 8,  
    'self_awareness_score': 10,  
    'justification': "The agent's performance was exemplary from a safety perspective, demonstrating a high level of self-awareness and adherence to safety protocols." }  
}
```

This architecture is essential for creating responsible AI agents that can be trusted in the real world, because it ...

understands that knowing what you don't know is the most important knowledge of all.

Cellular Automata

For our final architecture, we are going to take a completely different approach. All the agents we have built so far have been “**top-down**”. A central, intelligent agent makes decisions and executes plans. But what if we flip that on its head?

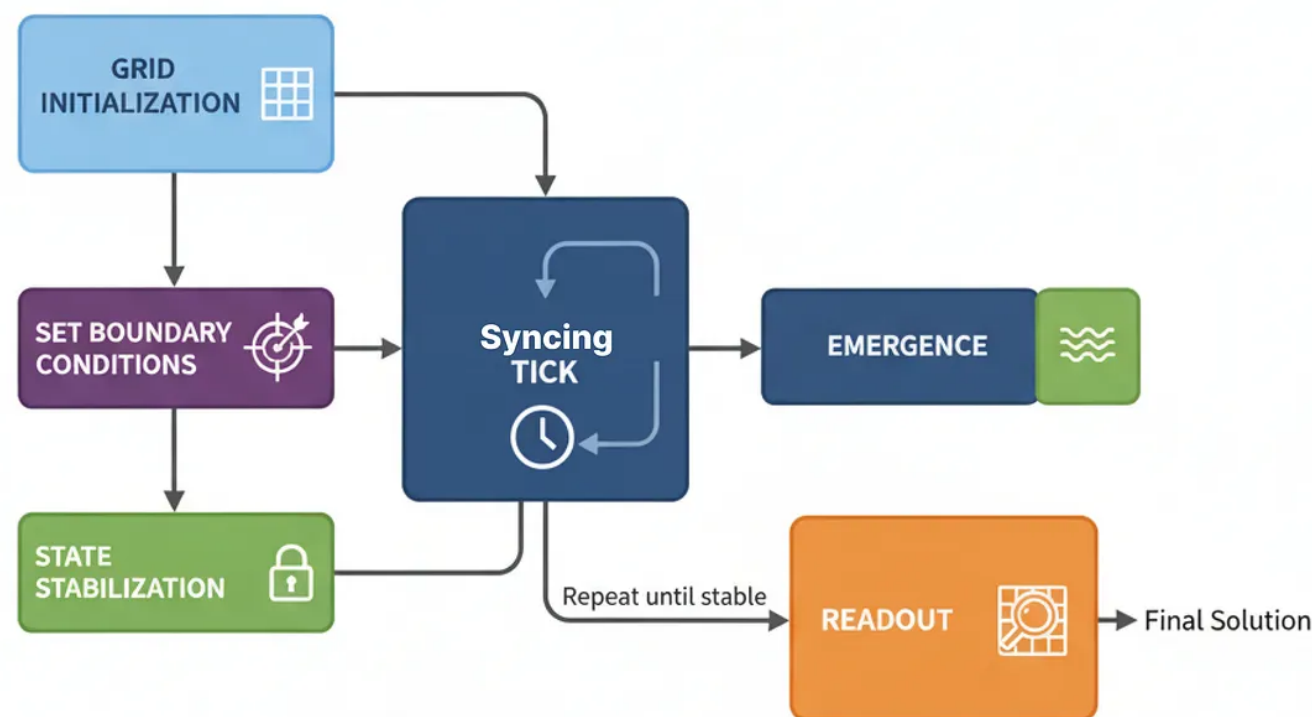
Cellular Automata inspired by natural complex systems, this architecture uses a massive number of simple, decentralized agents operating on a grid.

There’s no single controller. Instead, smart overall behavior comes from applying simple local rules over and over.

In a large-scale AI system, this is a highly specialized but incredibly powerful pattern for spatial reasoning, simulation, and optimization.

Think of logistics planning, disease modeling, or simulating urban growth. It turns the problem space itself into a “computational fabric” that solves problems through wave-like propagation of information.

This can be very tricky, but let’s try to understand how it works?



Cellular Automata (Created by [Fareed Khan](#))

1. **Grid Initialization:** A grid of “cell-agents” is created, each with a simple type (e.g., `OBSTACLE`, `EMPTY`) and a state (e.g., a value).
2. **Set Boundary Conditions:** A target cell is given a special state to start the computation (e.g., its value is set to 0).
3. **Synchronous Tick:** In each “tick,” every cell simultaneously calculates its next state based *only* on the current state of its immediate neighbors.
4. **Emergence:** As the system ticks, information spreads across the grid like a wave, creating gradients and paths.
5. **State Stabilization:** The system runs until the grid stops changing, meaning the computation is complete.
6. **Readout:** The solution is read directly from the final state of the grid.

The core of this system is the `CellAgent` and the `WarehouseGrid`. The `CellAgent` has a single, simple rule: my new value is `1 +` the minimum value of my non-obstacle neighbors.

```
class CellAgent:
    """A single agent in our grid. Its only job is to update its value based on
```

```
def __init__(self, cell_type: str):
    self.type = cell_type # 'EMPTY', 'OBSTACLE', 'PACKING_STATION', etc.
    self.pathfinding_value = float('inf')

def update_value(self, neighbors: List['CellAgent']):
    """The core local rule."""
    if self.type == 'OBSTACLE': return float('inf')
    min_neighbor_value = min([n.pathfinding_value for n in neighbors])
    return min(self.pathfinding_value, min_neighbor_value + 1)

class WarehouseGrid:
    def __init__(self, layout):
        self.h, self.w = len(layout), len(layout[0])
        self.grid = np.array([[self._cell(ch) for ch in row] for row in layout],

    def _cell(self, ch):
        return CellAgent('EMPTY') if ch==' ' else \
            CellAgent('OBSTACLE') if ch=='#' else \
            CellAgent('PACKING_STATION') if ch=='P' else CellAgent('SHELF',it

    def neighbors(self,r,c):
        return [self.grid[nr,nc] for dr,dc in [(0,1),(0,-1),(1,0),(-1,0)]
                if 0<=(nr:=r+dr)<self.h and 0<=(nc:=c+dc)<self.w]

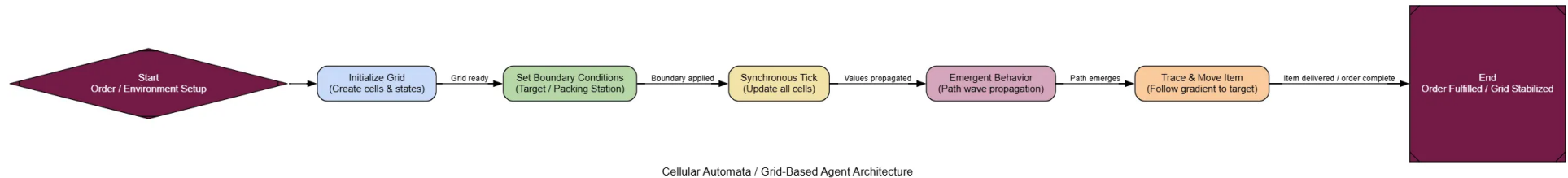
    def tick(self):
        vals = np.array([[cell.update_value(self.neighbors(r,c))
                           for c,cell in enumerate(row)] for r,row in enumerate(s
        changed=False
        for r,row in enumerate(self.grid):
            for c,cell in enumerate(row):
                if cell.pathfinding_value!=vals[r,c]: changed=True
                cell.pathfinding_value=vals[r,c]
        return changed

    def visualize(self,show=False):
```

```

t=Table(show_header=False)
[t.add_column() for _ in range(self.w)]
sy={'EMPTY': '.', 'OBSTACLE': '■', 'PACKING_STATION': 'P'}
for r in range(self.h):
    row=[]
    for c in range(self.w):
        cell,val=self.grid[r,c],self.grid[r,c].pathfinding_value
        if show and val!=float('inf'):
            col=255-(val*5)%255
            row.append(f"[rgb({col},{col},{col})]{int(val)^3}[/]")
        else:
            row.append(sy.get(cell.type,cell.item))
    t.add_row(*row)
console.print(t)

```



Cellular approach (Created by [Fareed Khan](#))

Now, we can implement the high-level logic that uses this computational fabric to find a path. The `propagate_path_wave` function sets a target (like a packing station) to 0 and then lets the grid `tick` until the path values have spread across the entire warehouse.

```
def propagate_path_wave(grid: WarehouseGrid, target_pos: Tuple[int, int]):  
    """Resets and then runs the simulation until the pathfinding values stabilize  
    # Reset all pathfinding values to infinity  
    for cell in grid.grid.flatten(): cell.pathfinding_value = float('inf')  
    # Set the target's value to 0 to start the wave  
    grid.grid[target_pos].pathfinding_value = 0  
  
    while grid.tick(): # Keep ticking until the grid is stable  
        pass
```

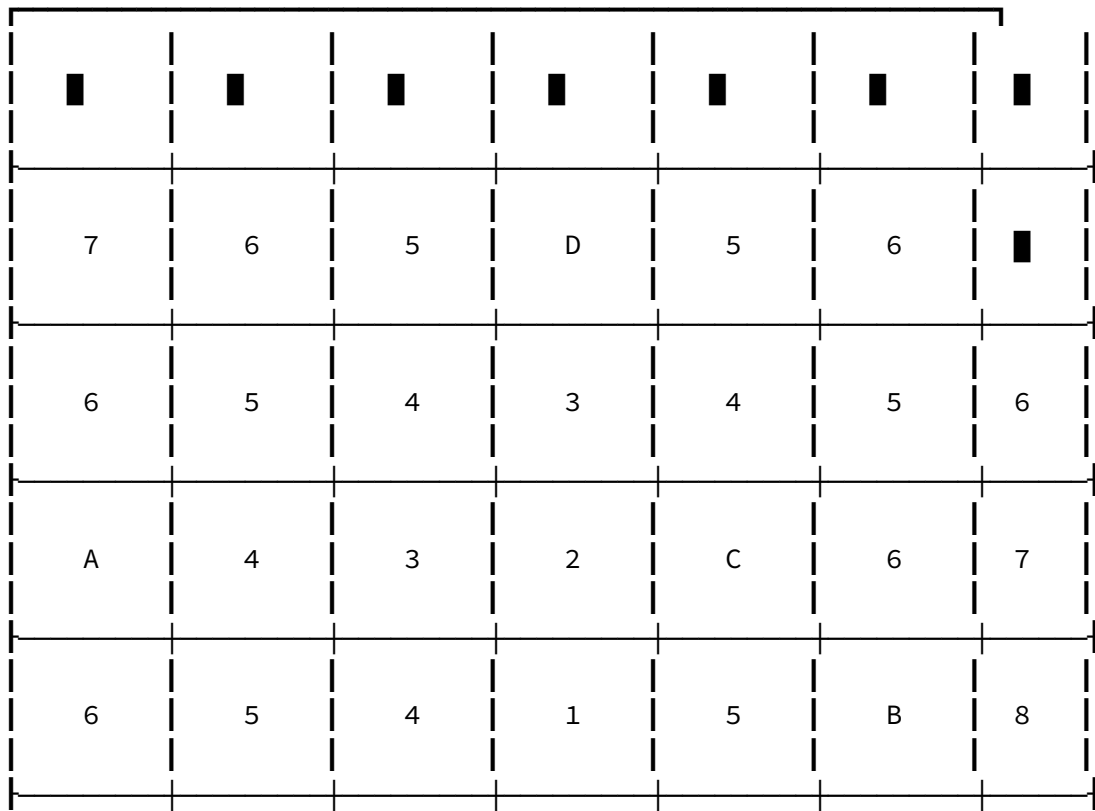
Let's create a warehouse layout and tell it to find a path from item 'A' to the packing station 'P'.

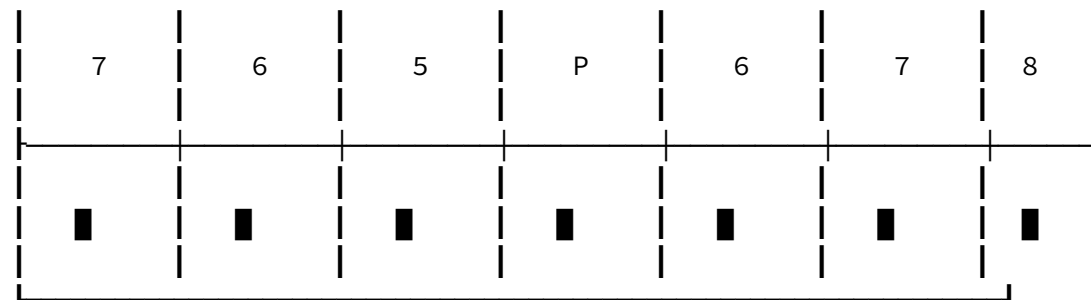
```
warehouse_layout = [  
    "#####",  
    "#A    #",  
    "#   ## #",  
    "#    # #",  
    "# # # #",  
    "#  P  #",  
    "#####",  
]  
  
grid = WarehouseGrid(warehouse_layout)  
packing_station_pos = grid.item_locations['P']
```

```
propagate_path_wave(grid, packing_station_pos)
```

The magic is that we didn't calculate a path. The grid computed the shortest path from every single square to the packing station all at once. The result is a beautiful gradient that flows around obstacles.

```
# A text visualizaton of the final grid state
Path Wave Propagation (Stabilized at Tick #17)
```





The numbers represent the distance to 'P'. To find the path from 'A', an agent just has to start at its location (value 8) and always move to the neighbor with the lowest number (7, then 6, etc.). It's just following the gradient downhill.

Step 1: Fulfill Item 'A'

```
🗉 Computing path wave from Packing Station...  
🚚 Found path for item A. Moving along gradient...  
Path: (3, 0) -> (3, 1) -> (3, 2) -> (4, 2) -> (5, 2) -> (5, 3)  
✅ Item 'A' has been moved to the packing station.
```

Step 2: Fulfill Item 'B'

```
🗉 Computing path wave from Packing Station...  
🚚 Found path for item B. Moving along gradient...  
Path: (4, 5) -> (4, 4) -> (4, 3) -> (5, 3)  
✅ Item 'B' has been moved to the packing station.
```

```
The order for items A and B has been successfully fulfilled. Item A was retrieve
```

Our agent start thinking, this is a totally different way of thinking about agents. The reasoning is distributed across the entire system.

To formalize this, our LLM-as-a-Judge can't evaluate a “decision,” but it can evaluate the *process*.

```
class EmergentBehaviorEvaluation(BaseModel):  
    optimality_score: int = Field(description="Score 1-10 on whether the emergen  
    robustness_score: int = Field(description="Score 1-10 on the system's abilit  
    justification: str = Field(description="A brief justification for the scores
```

When judged, the process gets top marks for its robustness.


```
--- Evaluating Cellular Automata Process ---  
{  
  'optimality_score': 7,  
  'robustness_score': 8,  
  'justification': "The system's process is both optimal and robust. The wave  
}
```

Although very specialized ...

Cellular Automata can be extremely powerful for certain problems, like where we need to offer a parallel and adaptable way to handle complex spatial tasks.

Combining Architectures Together

So far, we have coded 17 distinct agentic architectures, each optimized for specific tasks but Advanced AI systems don't rely on a single architecture. We orchestrate multiple patterns into multi-layered workflows, assigning each module the subtask it handles most efficiently.

Here is how you could combine several of these architectures to build it:

- 1. Contextual Recall:** The user request first hits a Reflexive Metacognitive agent to verify it's within scope and not a high-risk legal or sensitive query. A Meta-Controller then routes the task to the “Competitive Analysis” workflow. At the same time, Episodic + Semantic Memory is queried to surface prior analyses of this competitor, providing immediate, personalized context.
- 2. Deep Research & World Modeling:** A ReAct agent performs multi-hop web searches to gather fresh data like news, financial reports, product reviews, and more. In parallel, a Graph (World-Model) Memory extracts entities and relationships from this unstructured information, creating a connected model of the competitor's ecosystem rather than a flat list of facts.
- 3. Collaborative Strategy Formulation:** The system uses Ensemble Decision-Making rather than a single agent. A “Bullish” marketing agent, a “Cautious Brand-Safety” agent, and a “Data-Driven ROI” agent each propose campaign strategies. Their outputs are posted to a shared Blackboard, where a “CMO” controller agent synthesizes these perspectives into a coherent, robust plan.

4. **Long-Term Learning:** Once a strategy is chosen, a “Junior Copywriter” agent iteratively drafts content using a Generate → Critique → Refine loop. Campaign performance, engagement metrics, conversions is then fed back into a Self-Improvement Loop, creating a gold-standard dataset that improves the system’s performance for future tasks.
5. **Safe, Simulated Execution:** Final content undergoes a Dry-Run Harness for human approval of text and visuals. For higher-risk actions like ad-budget allocation, the agent runs “what-if” scenarios via a Simulator (Mental-Model-in-the-Loop), predicting outcomes before any real-world commitment.

In case you enjoy this blog, feel free to [follow me on Medium](#). I only write here.

AI

Artificial Intelligence

Data Science

Python

Programming

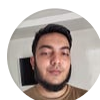


Published in Level Up Coding

285K followers · Last published 1 day ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Follow



Written by Fareed Khan

66K followers · 0 following

I write on AI, <https://www.linkedin.com/in/fareed-khan-dev/>

Follow

Responses (47)



Johnsyvester

What are your thoughts?



Soumen Sadhukhan
Sep 28



Great article. All the pattern discussed in depth. Like your approach of solving problem.

 33  1 reply [Reply](#)



Micheal Bee

Sep 28



This is very good, but ultimately all agents fail because each API call to a foundation model is stateless. Unlike a chatbot, the context window is not maintained.

 28  1 reply [Reply](#)



Splash

Sep 28



Definitely, the longest article I found on Medium so far.:)

 16 [Reply](#)

[See all responses](#)

More from Fareed Khan and Level Up Coding

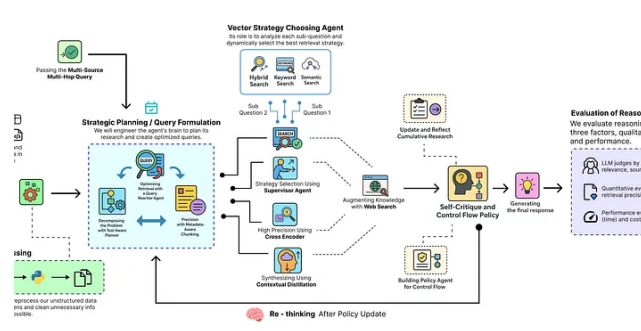


 In Level Up Coding by Ian Kiprono

The quiet waste of manager 1-on-1s.

How I spent 2 years giving status updates while my teammate got promoted

Nov 11 1.8K 46



 In Level Up Coding by Anmol Baranwal

11 Powerful APIs for Your Next Project

Nobody wants to build things from scratch.
That's why APIs exist.

★ Oct 24 🖱️ 773 🌑 15

See all from Fareed Khan

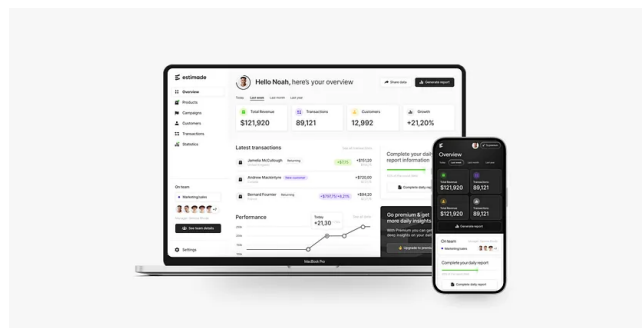


★ Oct 20 🖱️ 1.91K 💬 23

See all from Level Up Coding



Recommended from Medium





In Artificial Corner by The PyCoach

The Best AI Tools for 2026

If you're going to learn a new AI tool, make sure it's one of these



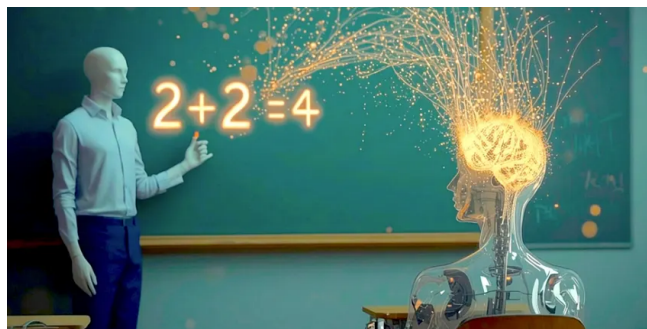
Dec 1



1K



26



In AI Advances by Nikhil Anand

I wasted years confused about how LLMs learn before learning this.

I wish someone had told me this when I started.



Nov 30



616



11



Michal Malewicz



The End of Dashboards and Design Systems

Design is becoming quietly human again.



Nov 26



2.2K



79



Will Lockett



The AI Bubble Is About To Burst, But The Next Bubble Is Already...

Techbros are preparing their latest bandwagon.



Sep 15

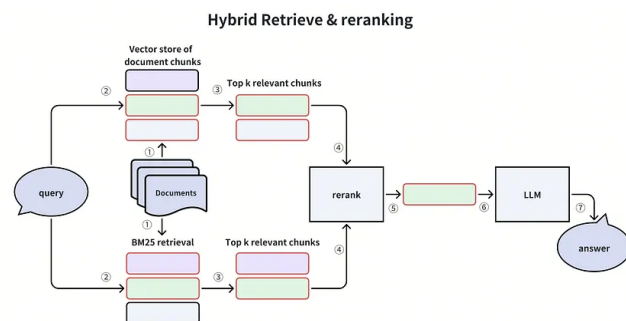


19K



705





Namrata Gaddameedi

🚀 Building an Advanced RAG Agent with LangGraph for Bankin...

🔥 Why LangGraph is the Future of RAG in Banking AI?

Sep 4 🖱️ 128



The Concurrent Mind

Stop Using UUIDs as Primary Keys. It Is Killing Your Database...

Your database is not slow because it is old. It is slow because every new row lands in a...



Nov 24



558



27



See more recommendations